

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU VỀ KIẾN TRÚC PHẦN MỀM	3
1.1. Khái niệm.....	3
1.1.1 Kiến trúc	4
1.1.2 Thành phần	5
1.1.3 Kết nối	6
1.1.4 Cấu hình.....	8
1.1.5 Kiểu kiến trúc	8
1.1.6 Mẫu kiến trúc.....	8
1.1.7 Mô hình kiến trúc	11
1.2. Vai trò	11
1.3. Các thuộc tính về chất lượng	13
1.4. Vai trò của thiết kế trong quy trình phát triển phần mềm.....	14
1.5. Chiến lược và phương pháp thiết kế.	16
1.5.1 Các chiến lược thiết kế phần mềm	16
1.5.2 Các phương pháp thiết kế phần mềm	19
Câu hỏi và bài tập cuối chương 1:	25
CHƯƠNG 2: MỘT SỐ KIỂU KIẾN TRÚC PHẦN MỀM	29
2.1. Kiến trúc ống và lọc.....	29
2.2. Kiến trúc hướng sự kiện	40
2.3. Kiến trúc phân lớp	52
2.4. Kiến trúc gọi và trả lời.....	58
2.5. Kiến trúc hướng đối tượng	68
2.6. Kiến trúc hướng dịch vụ	73
Câu hỏi và bài tập cuối chương 2:	80
CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC PHẦN MỀM	88
3.1. Tổng quan về thiết kế kiến trúc phần mềm	88

3.1.1 Khái niệm về thiết kế kiến trúc phần mềm.....	88
3.1.2 Tiến trình thiết kế kiến trúc	88
3.1.3 Các nguyên lý thiết kế kiến trúc	91
3.2. Vai trò của thiết kế kiến trúc phần mềm.....	95
3.3. Phương pháp thiết kế kiến trúc ADD	97
3.3.1 Tổng quát về phương pháp ADD	97
3.3.2 Các bước của phương pháp ADD:	98
3.3.3 Ưu điểm và hạn chế của phương pháp ADD:	102
3.3.4 Ứng dụng của phương pháp ADD:.....	103
3.3.5 So sánh ADD với các phương pháp thiết kế kiến trúc khác:	103
3.4 Bài tập áp dụng	104
Câu hỏi và bài tập	108
CHƯƠNG 4: THIẾT KẾ MẪU	112
4.1. Giới thiệu về mẫu thiết kế	112
4.2. Một số mẫu thiết kế cơ bản	116
4.2.1 Nhóm mẫu tạo dựng:	116
4.2.2 Nhóm mẫu cấu trúc	117
4.2.3 Nhóm mẫu hành vi	119
4.3. Bài tập thiết kế phần mềm trong bài toán thực.....	123
4.3.1 Các bước thiết kế phần mềm trong bài toán thực tế.....	123
4.3.2 Bài toán 1: Thiết kế hệ thống phần mềm quản lý thư viện trực tuyến dùng phương pháp sử dụng nhóm mẫu thiết kế tạo dựng (Factory Method).	125
4.3.3 Bài toán 2: Thiết kế hệ thống phần mềm đặt xe trực tuyến dùng phương pháp sử dụng nhóm mẫu thiết kế cấu trúc.....	134
4.3.4 Bài toán 3: Thiết kế hệ thống phần mềm game đua xe dùng phương pháp sử dụng nhóm mẫu thiết kế hành vi.	141
Câu hỏi và bài tập cuối chương 3:	155

CHƯƠNG 1: GIỚI THIỆU VỀ KIẾN TRÚC PHẦN MỀM

1.1. Khái niệm

Thuật ngữ *kiến trúc phần mềm* (software architecture) lần đầu tiên được sử dụng vào cuối những năm 1960 nhưng mãi đến những năm 1990 khái niệm này mới được xem xét, nghiên cứu rộng rãi trong công nghệ phần mềm. Kiến trúc phần mềm được hiểu như một khái niệm quan trọng bắt nguồn từ những nghiên cứu của Dijkstra vào năm 1968 và Parnas vào đầu những năm 1970. Các nghiên cứu này đã chỉ ra rằng *kiến trúc của hệ phần mềm đóng một vai trò quan trọng và làm thế nào có được một kiến trúc hợp lý là vấn đề cốt lõi trong quá trình phát triển phần mềm*. Những năm 1990 đã chứng kiến nhiều nỗ lực nghiên cứu nhằm đề xuất các kiểu kiến trúc phần mềm, các ngôn ngữ mô tả kiến trúc, các mẫu thiết kế, các mô hình thành phần phần mềm, các phương pháp hình thức để mô hình hóa kiến trúc... Động lực chính cho sự quan tâm mạnh mẽ này là mã nguồn của nhiều hệ phần mềm phổ biến đã lên đến hàng trăm nghìn hay hàng triệu dòng lệnh.

Kiến trúc hệ thống (system architecture) là một sự gắn kết các bộ phận trong một hệ thống với nhau bao gồm cấu trúc, giao diện và các cơ chế để phối hợp hoạt động giữa các bộ phận. Khi xác định được kiến trúc phù hợp, chúng ta sẽ dễ dàng chuyển đổi, tìm được vị trí của một chức năng hay chỉ ra được nơi có thể thêm một chức năng mới phù hợp với kiến trúc chung. Một kiến trúc tốt [18] phải được chi tiết hóa đủ để có thể ánh xạ thành mã nguồn thực sự và cho phép thêm những chức năng mới hay những khái niệm mới mà không ảnh hưởng đến phần còn lại của hệ thống.

Kiến trúc một hệ phần mềm được hiểu là một quá trình xác định một giải pháp để đưa ra một cấu trúc thích hợp nhằm đáp ứng được các yêu cầu về kỹ thuật và hoạt động của một tổ chức hay doanh nghiệp đồng thời có thể tối ưu hóa về mặt hiệu năng, an toàn và khả năng quản lý.

Quá trình này liên quan đến một loạt các quyết định và mỗi quyết định có thể có tác động thực sự đến chất lượng, hiệu năng, bảo trì, và thành công chung của một hệ ứng dụng. Để hiểu rõ hơn khái niệm kiến trúc, chúng ta sẽ xem xét một số định nghĩa đã được đưa ra gần đây [13]:

□ Booch đã định nghĩa kiến trúc phần mềm là một loạt các quyết định quan trọng về tổ chức một hệ thống phần mềm: Lựa chọn các thành phần của cấu trúc, giao diện của hệ thống; Xác định kiểu tương tác giữa các thành phần; Xác định các yếu tố cơ bản cấu thành hệ thống; Xác định kiểu kiến trúc của hệ thống; Khả năng tái xử dụng...

□ Martin Fowler cũng đã chỉ ra một số chủ đề phổ biến khi giải thích về kiến trúc như sau: việc phân rã hệ thống ở mức cao thành các bộ phận, những quyết định khó thay đổi, nhiều kiến trúc trong hệ thống, cái gì có thể thay đổi trong vòng đời của hệ thống...

□ Bass et al. [3] đã định nghĩa kiến trúc phần mềm là một hay nhiều cấu trúc của hệ thống bao gồm các thành phần phần mềm và các quan hệ giữa chúng.

Như vậy, kiến trúc một hệ thống là tìm cách xây dựng một liên kết giữa các yêu cầu nghiệp vụ và các yêu cầu kỹ thuật bằng việc hiểu các ca sử dụng (use case) và sau đó tìm cách cài đặt những ca sử dụng này vào trong phần mềm.

Mục tiêu của kiến trúc là chỉ ra được các yêu cầu có ảnh hưởng như thế nào đến cấu trúc của ứng dụng. Kiến trúc tốt có thể giảm được rủi ro về nghiệp vụ trong khi xây dựng một giải pháp kỹ thuật và có thể xử lý được các sự cố xảy ra liên quan đến phần cứng hay phần mềm trong vòng đời của nó. Kiến trúc sư phần mềm cần phải xem xét ảnh hưởng tổng thể của những quyết định thiết kế, sự cân bằng vốn có của các thuộc tính chất lượng (như hiệu năng và bảo mật) và sự cân bằng cần thiết của yêu cầu người dùng, hệ thống và yêu cầu nghiệp vụ. Sau đây là một số hướng dẫn khi kiến trúc một hệ thống:

□ Kiến trúc cần thể hiện được cấu trúc (structure) của hệ thống nhưng ẩn giấu các chi tiết cài đặt.

□ Kiến trúc phải hiện thực hóa được tất cả các ca sử dụng và kịch bản.

□ Kiến trúc phải thể hiện được sự quan tâm đến các yêu cầu của các bên liên quan.

□ Kiến trúc phải đáp ứng cả yêu cầu về chức năng lẫn yêu cầu về phi chức năng

Phần này nhằm trình bày chi tiết hơn một số định nghĩa về các thuật ngữ và những ý tưởng cơ bản về kiến trúc phần mềm để làm cơ sở cho trình bày các chương về sau.

1.1.1 Kiến trúc

Theo Taylor [18] *kiến trúc một hệ phần mềm là một tập các quyết định thiết kế chủ yếu được đưa ra cho hệ thống cần xây dựng*. Như vậy, kiến trúc phần mềm được xem là

những định hướng cho phát triển và tiến hóa một hệ phần mềm. Khái niệm *quyết định thiết kế* (design decision) là trung tâm của kiến trúc phần mềm nhằm kết nối các khái niệm khác liên quan đến kiến trúc. Sau đây là một số khái niệm liên quan đến quyết định thiết kế:

- Các quyết định liên quan đến *cấu trúc* (structure) của hệ thống. Ví dụ, các thành phần kiến trúc cần phải tổ chức và hợp thành các gói phân cấp theo kiểu 3 tầng.

- Các quyết định liên quan đến *hành vi chức năng* (functional behavior). Ví dụ, xử lý dữ liệu, lưu trữ và hiển thị sẽ được thực thi theo một thứ tự nghiêm ngặt.

- Các quyết định liên quan đến *tương tác* (interaction). Ví dụ, giao tiếp giữa các thành phần hệ thống chỉ xảy ra bằng cách sử dụng thông báo, sự kiện hay giao thức RMI trong java.

- Các quyết định liên quan đến *các tính chất phi chức năng* (nonfunctional properties) của hệ thống. Ví dụ, khả năng phụ thuộc của hệ thống sẽ được đảm bảo bằng cách phục hồi các mô đun xử lý.

- Các quyết định liên quan đến *cài đặt* (implementation) hệ thống. Ví dụ, thành phần giao diện người sử dụng sẽ được xây dựng bằng cách sử dụng Java Swing hay JSP.

Rõ ràng các quyết định quan trọng khác nhau sẽ dẫn đến các kiến trúc khác nhau. Tuy nhiên, một số quyết định không được xem là chủ yếu và không ảnh hưởng đến kiến trúc của hệ thống như chi tiết của thuật toán hay cấu trúc dữ liệu.

1.1.2 Thành phần

Các quyết định tạo nên kiến trúc một hệ phần mềm bao gồm việc kết hợp nhiều phần tử khác biệt nhưng có liên quan với nhau. Các phần tử này bao gồm những khía cạnh khác nhau:

- Xử lý: liên quan đến chức năng hay hành vi
- Trạng thái: liên quan đến thông tin hay dữ liệu
- Tương tác: liên quan đến cộng tác, phối hợp...

Những phần tử đóng gói xử lý và dữ liệu bên trong kiến trúc hệ thống gọi là *thành phần phần mềm* (software component). Theo Taylor [18]:

Thành phần phần mềm là một thực thể kiến trúc: (i) đóng gói một tập con chức năng và dữ liệu của hệ thống; (ii) chỉ có thể truy nhập vào thực thể thông qua một giao diện; (iii) xác định các phụ thuộc tùy theo ngữ cảnh mà nó yêu cầu.

Như vậy, thành phần phần mềm là sự tích hợp của tính toán và trạng thái trong hệ thống. Tùy theo kiểu kiến trúc, quan điểm người thiết kế hay yêu cầu của hệ thống, thành phần có thể đơn giản như một thao tác, một phương thức hay phức tạp như một phần hệ thống. Đặc trưng quan trọng nhất của thành phần là chỉ được nhìn thấy từ bên ngoài qua giao diện. Thành phần phần mềm được cho là thể hiện đầy đủ các nguyên lý *đóng gói* (encapsulation), *trừu tượng hóa* (abstraction) và *môđun hóa* (modularity) của kỹ thuật phần mềm. Như vậy, các thành phần hàm ý có khả năng kết hợp, sử dụng lại và tiến hóa.

Tuy nhiên, các khả năng này lại được thể hiện trong ngữ cảnh tạo ra thành phần như giao diện phần mềm, tính có sẵn của tài nguyên (dữ liệu...), thùng chứa thành phần, môi trường thực thi của chương trình, hệ điều hành, giao thức mạng, cấu hình phần cứng...

1.1.3 Kết nối

Một đặc trưng cơ bản quan trọng nữa của hệ phần mềm là *tương tác* (interaction) giữa các thành phần trong hệ thống. Nhiều hệ thống hiện đại được xây dựng với nhiều thành phần phức tạp, phân tán qua mạng và cập nhật liên tục suốt trong vòng đời của nó. Do đó, đối với người phát triển việc đảm bảo tương tác giữa các thành phần thậm chí quan trọng hơn chính chức năng của bản thân các thành phần.

Kết nối là một phần tử kiến trúc có nhiệm vụ thực thi tương tác giữa các thành phần.

Đối với hệ phần mềm trên máy để bàn truyền thống, các kết nối thường được thể hiện bởi lời *gọi thủ tục* hay *truy nhập dữ liệu chung*. Các kết nối này thường chỉ thể hiện sự tương tác giữa các cặp thành phần. Gọi thủ tục được cài đặt trực tiếp trong các ngôn ngữ lập trình để giúp trao đổi đồng bộ dữ liệu và điều khiển giữa các cặp thành phần. Khi đó, thành phần triệu gọi chuyển luồng điều khiển cũng như dữ liệu dưới dạng tham số đến thành phần được gọi và sau khi thực hiện yêu cầu, bên được gọi trả lại điều khiển cũng như kết quả tính toán cho bên gọi. Trong truy nhập dữ liệu, kiểu kết nối được thể hiện dưới dạng biến toàn cục hay bộ nhớ chung. Kiểu kết nối này cho phép nhiều thành phần tương tác bằng cách đọc hay ghi vào các phương tiện chung. Tương tác được phân bố không đồng

bộ theo thời gian nghĩa là việc đọc không phụ thuộc hay ràng buộc theo thời gian của việc ghi và ngược lại.

Tuy nhiên, trong các hệ phần mềm phức tạp trên mạng, các kết nối thường xảy ra cùng một lúc giữa một thành phần với nhiều thành phần dịch vụ khác nhau. Dựa vào cách thực thi dịch vụ tương tác, người ta thường phân các kết nối thành tám kiểu khác nhau [18]:

- ☐ *Kết nối gọi thủ tục*: ví dụ như phương thức trong lập trình hướng đối tượng.
- ☐ *Kết nối sự kiện*: ví dụ trong ứng dụng windows, các input GUI xem như sự kiện kích hoạt hệ thống.
- ☐ *Kết nối truy nhập dữ liệu*: ví dụ các cơ chế truy vấn trong cơ sở dữ liệu SQL
- ☐ *Kết nối liên kết*: được sử dụng để ràng buộc các thành phần cùng giữ một trạng thái suốt trong quá trình tương tác với nhau.
- ☐ *Kết nối luồng*: được sử dụng để truyền một khối lượng lớn dữ liệu giữa các tiến trình tự chủ riêng. Ví dụ, các socket TCP/UDP hay các giao thức client-server như RMI, CORBA, FTP, SOAP.
- ☐ *Kết nối trung gian*: được sử dụng khi các thành phần biết sự có mặt của các thành phần khác nhưng không thể biết được nhu cầu, trạng thái của nó. Khi đó, bộ phận trung gian sẽ cung cấp dịch vụ phối hợp hay giải quyết tranh chấp...
- ☐ *Kết nối thích nghi*: cung cấp phương tiện để hỗ trợ tương tác giữa các thành phần không được thiết kế với tương tác. Ví dụ, hệ phân tán có thể dựa vào gọi thủ tục từ xa (RPC: remote procedure call) cho mọi tương tác.
- ☐ *Kết nối theo tuyến*: được sử dụng để chỉ ra các tuyến tương tác và thực thi định tuyến truyền thông cũng như phối hợp giữa các thành phần theo tuyến này.

Thông thường các hệ phân tán sử dụng nhiều cách kết nối với nhau. Ví dụ, trong tính toán lưới, kết nối phân tán dữ liệu dựa trên hợp của bốn kiểu kết nối: gọi thủ tục, truy nhập dữ liệu, luồng và kết nối theo tuyến. Các kết nối này phân phối một khối lượng lớn dữ liệu giữa các thành phần trong môi trường này. Trong các hệ P2P, kết nối phân tán gồm bốn kiểu: trung gian, truy nhập dữ liệu, luồng và kết nối theo tuyến. Trong các hệ phân tán dựa trên mô hình client-server, kết nối gồm bốn kiểu: gọi thủ tục, truy nhập dữ liệu, luồng và kết nối theo tuyến.

1.1.4 Cấu hình

Các thành phần và kết nối thường được kết hợp với nhau theo cách nào đó trong cấu trúc để xác lập mục tiêu của hệ thống. Cách kết hợp này thể hiện cấu hình hay tô pô của hệ thống. Như vậy, *cấu hình của kiến trúc là tập các cách kết hợp giữa các thành phần và kết nối của kiến trúc hệ phần mềm.*

Cấu hình có thể biểu diễn như một đồ thị trong đó các đỉnh biểu diễn thành phần và kết nối, các cạnh biểu diễn sự kết hợp.

1.1.5 Kiểu kiến trúc

Kiểu kiến trúc là tập những quyết định thiết kế: (i) áp dụng được trong ngữ cảnh phát triển; (ii) ràng buộc những quyết định thiết kế cho hệ thống đó; (iii) thể hiện được chất lượng và lợi ích trong kiểu hệ thống như vậy.

Nhiều kinh nghiệm và nghiên cứu đã chỉ ra rằng trong các hệ phân tán, kiểu kiến trúc thỏa một số điều kiện sau đây sẽ đem nhiều lợi điểm:

- ☐ Tách biệt về mặt vật lý: các thành phần phần mềm yêu cầu dịch vụ tách biệt với các thành phần cung cấp dịch vụ. Điều này cho phép cấu trúc các thành phần một cách hợp lý, hiệu quả cả về số lượng yêu cầu và cung cấp dịch vụ.
- ☐ Trong suốt: làm cho nhà cung cấp không biết định danh của bên yêu cầu để dễ bề cung cấp dịch vụ một cách trong suốt kể cả khi thay đổi bên yêu cầu.
- ☐ Cô lập: tách biệt các bên yêu cầu với nhau để cho phép thêm, loại bỏ hay sửa đổi một cách độc lập.

1.1.6 Mẫu kiến trúc

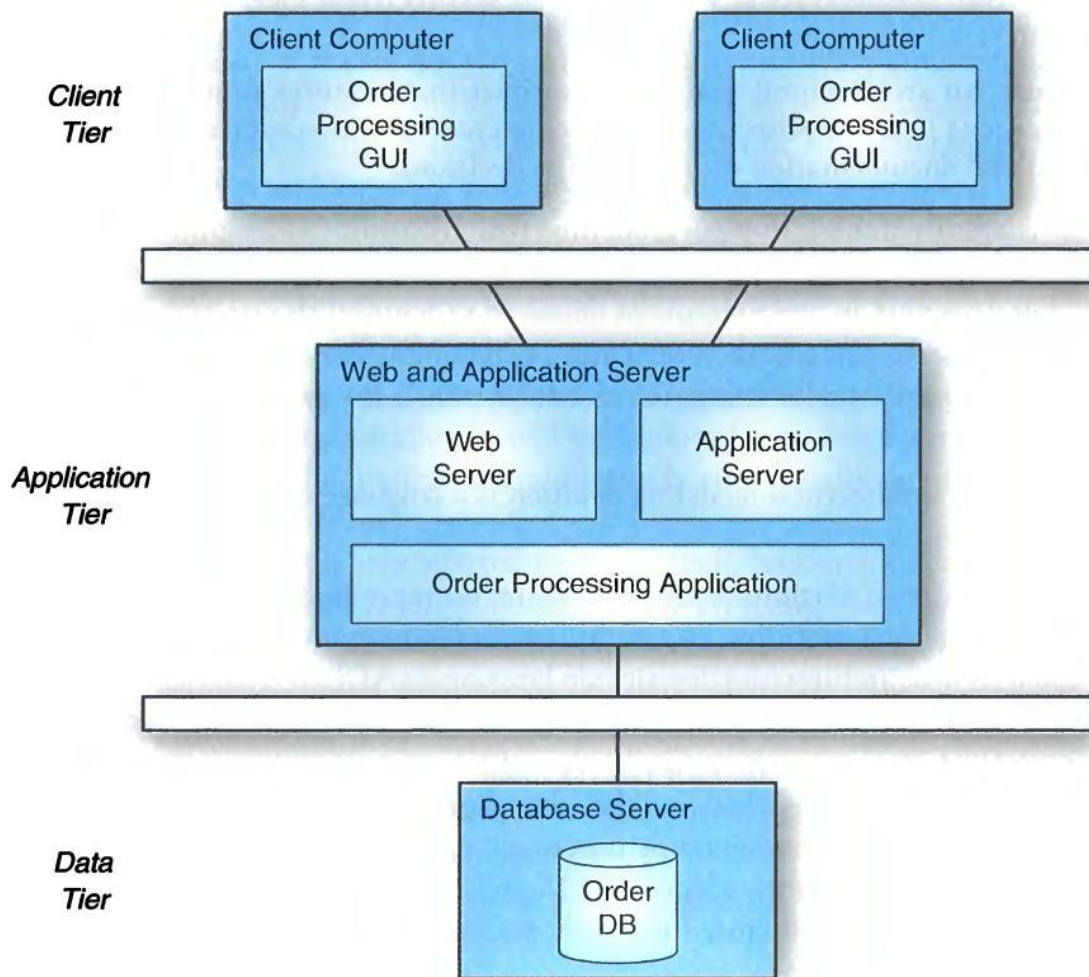
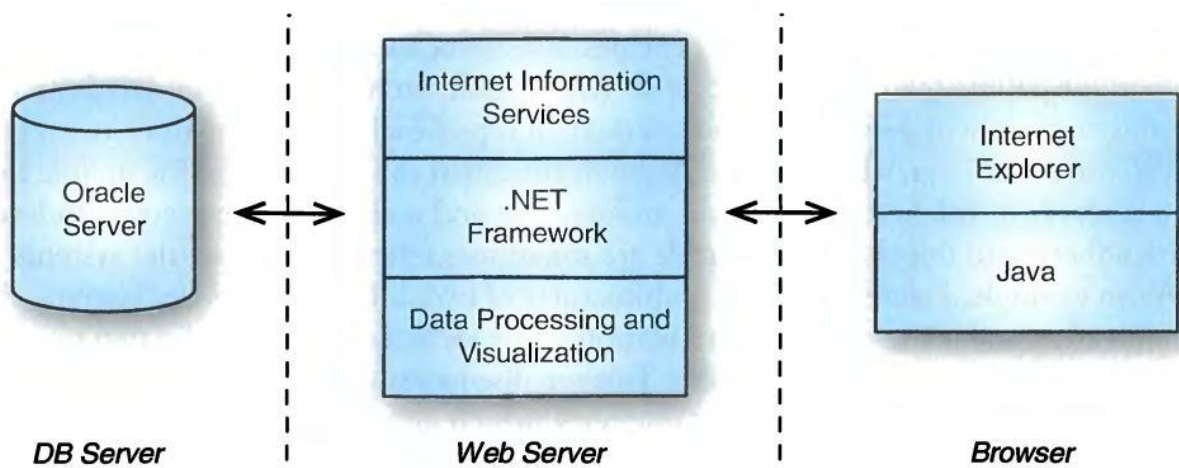
Các kiểu kiến trúc đưa ra những quyết định thiết kế và ràng buộc tổng quát. Chúng cần được mịn hóa thành các quyết định cụ thể dành riêng cho hệ thống cần xây dựng. Trong khi đó, mẫu kiến trúc (architectural pattern) cung cấp một tập các quyết định thiết kế liên quan đến các thành phần và kết nối với những đặc trưng cụ thể cho một lớp hệ phần mềm. Sự phân biệt giữa mẫu và kiểu kiến trúc không phải bao giờ cũng rõ ràng. Một số đặc trưng sau đây có thể giúp ta hiểu hơn sự khác biệt này:

□ *Phạm vi*: Kiểu kiến trúc áp dụng vào một ngữ cảnh phát triển như GUI, trong khi mẫu kiến trúc áp dụng vào một bài toán thiết kế đặc biệt như logic nghiệp vụ của hệ thống phải tách biệt với quản trị dữ liệu.

□ *Trừu tượng hóa*: Kiểu giúp ràng buộc các quyết định thiết kế kiến trúc về hệ thống và khá trừu tượng nên đòi hỏi sự thể hiện của con người. Trong khi đó, mẫu là phần kiến trúc đã được tham số hóa và có thể xem như một mẫu thiết kế cụ thể.

□ *Quan hệ*: Các mẫu không thể sử dụng ngay được mà phải được tham số hóa cho ngữ cảnh của bài toán cụ thể. Nghĩa là một mẫu cũng có thể áp dụng được cho những hệ thống với nhiều kiểu khác nhau. Ngược lại, một hệ thống được thiết kế theo một kiểu nào đó có thể sử dụng nhiều mẫu.

Ví dụ: Mẫu kiến trúc Hệ phân tán 3-tầng đã được sử dụng rộng rãi hiện nay cho các hệ thống phục vụ nghiên cứu khoa học (chẩn đoán ung thư, thi n văn, địa lý , dự đoán thời tiết..), các hệ thống dịch vụ ngân hàng, thương mại điện tử, đặt chỗ du lịch, chăm sóc y tế...Các hệ thống này có thể xử lý, lưu trữ, truy xuất một khối lượng lớn dữ liệu những người dùng khác nhau (Hình 1.1).



Hình 1.1: Biểu diễn kiến trúc 3 tầng [18]

Các hệ thống này gồm 3 tầng:

□ Tầng client hay trình diễn chứa chức năng cần thiết để người dùng truy nhập dịch vụ hệ thống. Tầng này chứa giao diện GUI của hệ thống và có thể lấy dữ liệu và thực thi một số xử lý nhỏ. Tầng client thông thường được triển khai trên các máy với khả năng tính toán và lưu trữ hạn chế như máy để bàn hay các thiết bị cầm tay, di động...

□ Tầng ứng dụng (application) hay logic nghiệp vụ hay còn gọi là tầng trung gian chứa chức năng chính của ứng dụng. Tầng này chịu trách nhiệm đáp ứng các yêu cầu dịch vụ hay xử lý yêu cầu từ tầng client và truy nhập, xử lý dữ liệu trong tầng dữ liệu. Tầng này được triển khai trên tập các máy chủ mạnh và lưu trữ lớn.

□ Tầng dữ liệu (data) chứa chức năng lưu trữ và truy nhập dữ liệu của ứng dụng. Tầng này chứa cơ sở dữ liệu lớn và có khả năng cung cấp nhiều yêu cầu truy nhập dữ liệu song song.

Tương tác giữa các tầng về nguyên lý là tuân theo khuôn dạng “yêu cầu – đáp ứng” nhưng không quy định chi tiết những dạng tương tác này. Nó có thể cài đặt theo kiểu một yêu cầu - một đáp ứng hay nhiều yêu cầu - một đáp ứng và cũng có thể một yêu cầu – nhiều đáp ứng hay đáp ứng dựa trên cập nhật yêu cầu theo chu kỳ.

1.1.7 Mô hình kiến trúc

Kiến trúc của một hệ phần mềm có thể được thể hiện bởi *mô hình kiến trúc* (architectural model) bằng cách sử dụng những ký hiệu mô hình hóa. Mô hình kiến trúc là sản phẩm thể hiện một số hay tất cả những quyết định thiết kế tạo nên kiến trúc của hệ thống. Tập ký hiệu mô hình kiến trúc được gọi là ngôn ngữ mô tả kiến trúc. Ngôn ngữ này có thể là văn bản, đồ họa, biểu đồ... Ví dụ, kiến trúc hệ 3 tầng có thể thể hiện bởi hai tập ký hiệu mô hình hóa khác nhau (Hình 1.1).

1.2. Vai trò

Kiến trúc phần mềm (Kiến trúc phần mềm) đóng vai trò cực kỳ quan trọng trong quá trình phát triển phần mềm. Nó được ví như bản thiết kế của một ngôi nhà, cung cấp một cái nhìn tổng thể về cấu trúc, các thành phần và cách chúng tương tác với nhau.

Các lợi ích cụ thể của kiến trúc phần mềm:

- Tăng chất lượng phần mềm: Giúp tạo ra phần mềm ổn định, đáng tin cậy và dễ sử dụng.

- Rút ngắn thời gian phát triển: Quá trình phát triển trở nên có tổ chức và hiệu quả hơn.
- Giảm chi phí: Giảm thiểu chi phí sửa lỗi và bảo trì.
- Tăng tính linh hoạt: Dễ dàng thích ứng với những thay đổi trong yêu cầu.

Kiến trúc phần mềm đóng vai trò quan trọng trong phát triển phần mềm vì nhiều lý do:

1. Cung cấp khung làm việc rõ ràng: Kiến trúc phần mềm giúp định hình cấu trúc cơ bản của ứng dụng, bao gồm cách các thành phần kỹ thuật tương tác với nhau. Nó như bản đồ cho các nhà phát triển, giúp họ hiểu cách xây dựng các thành phần của phần mềm để đạt được mục tiêu cụ thể.
2. Đảm bảo hiệu suất và khả năng mở rộng: Một kiến trúc tốt cho phép hệ thống xử lý tải lớn và mở rộng dễ dàng khi cần thiết. Nó xác định cách các yêu cầu về hiệu suất và khả năng mở rộng được đáp ứng.
3. Tối ưu hóa ngân sách và tài nguyên: Kiến trúc hợp lý giúp tránh lãng phí nguồn lực và thời gian phát triển, do có kế hoạch và cấu trúc rõ ràng từ đầu. Điều này cũng giúp kiểm soát chi phí phát triển.
4. Dễ dàng bảo trì và cập nhật: Kiến trúc rõ ràng và được tổ chức tốt giúp việc bảo trì và cập nhật phần mềm trở nên dễ dàng hơn. Nó cho phép sửa đổi hoặc thêm tính năng mới mà không làm ảnh hưởng đến toàn bộ hệ thống.
5. Giảm thiểu rủi ro: Một kiến trúc được thiết kế tốt giúp nhận diện và giảm thiểu rủi ro từ sớm trong quá trình phát triển, bao gồm vấn đề về hiệu suất, bảo mật và sự phụ thuộc giữa các thành phần.
6. Tăng cường bảo mật: Kiến trúc xác định cách bảo mật được tích hợp vào hệ thống, từ việc xác định các điểm truy cập quan trọng đến việc thiết lập các lớp bảo mật.
7. Hỗ trợ quyết định kỹ thuật: Nó cung cấp một khung tham chiếu để đưa ra các quyết định kỹ thuật, từ việc chọn công nghệ đến việc quyết định cách triển khai các tính năng.

8. Tăng khả năng hợp tác: Kiến trúc rõ ràng giúp các nhà phát triển và các bên liên quan khác làm việc cùng nhau một cách hiệu quả hơn, do mọi người đều hiểu cấu trúc tổng thể và mục tiêu của dự án.

Nhìn chung, kiến trúc phần mềm không chỉ ảnh hưởng đến chất lượng cuối cùng của sản phẩm phần mềm mà còn có tác động lớn đến hiệu quả của quá trình phát triển, khả năng bảo trì và mở rộng của phần mềm.

1.3. Các thuộc tính về chất lượng

Chất lượng phần mềm là một khái niệm rộng, bao gồm nhiều yếu tố khác nhau như độ tin cậy, hiệu suất, khả năng bảo trì, khả năng mở rộng, tính bảo mật... Trong kiến trúc phần mềm, các thuộc tính chất lượng này được xem xét kỹ lưỡng để đảm bảo phần mềm đáp ứng được các yêu cầu của người dùng và các tiêu chuẩn chất lượng.

Các thuộc tính chất lượng phần mềm quan trọng trong kiến trúc phần mềm:

- Độ tin cậy (Reliability): Phần mềm hoạt động đúng như mong đợi trong điều kiện đã xác định, trong một khoảng thời gian nhất định.
 - + Khả năng sẵn sàng: Phần mềm có sẵn để sử dụng khi cần thiết.
 - + Khả năng phục hồi: Phần mềm có thể phục hồi sau lỗi hoặc sự cố.
 - + Độ chính xác: Phần mềm cung cấp kết quả đúng đắn.
- Hiệu suất (Performance): Phần mềm đáp ứng được các yêu cầu về thời gian đáp ứng, dung lượng bộ nhớ và khả năng xử lý.
- Khả năng sử dụng (Usability): Phần mềm dễ học, dễ sử dụng và phù hợp với người dùng mục tiêu.
- Khả năng bảo trì (Maintainability): Phần mềm dễ dàng sửa lỗi, nâng cấp và mở rộng.
- Khả năng mở rộng (Scalability): Phần mềm có thể xử lý được khối lượng công việc tăng lên.
- Tính bảo mật (Security): Phần mềm bảo vệ dữ liệu và hệ thống khỏi các mối đe dọa.
- Tính tương thích (Compatibility): Phần mềm hoạt động tốt với các phần mềm và hệ thống khác.

- Khả năng tái sử dụng (Reusability): Các thành phần của phần mềm có thể được sử dụng lại trong các dự án khác.

Vai trò của kiến trúc phần mềm trong việc đảm bảo chất lượng

- Định hình các thuộc tính chất lượng: Kiến trúc phần mềm đặt nền móng cho các thuộc tính chất lượng của phần mềm. Các quyết định kiến trúc ban đầu sẽ ảnh hưởng trực tiếp đến các thuộc tính này.

- Cung cấp cơ sở để đánh giá chất lượng: Kiến trúc phần mềm cung cấp một khung để đánh giá chất lượng của phần mềm trong suốt quá trình phát triển.

- Hỗ trợ quá trình bảo trì và nâng cấp: Một kiến trúc phần mềm tốt sẽ làm cho việc bảo trì và nâng cấp phần mềm trở nên dễ dàng hơn.

Các yếu tố ảnh hưởng đến chất lượng phần mềm trong kiến trúc

- Lựa chọn các mẫu kiến trúc: Các mẫu kiến trúc khác nhau sẽ mang lại các thuộc tính chất lượng khác nhau.

- Công nghệ và công cụ: Các công nghệ và công cụ sử dụng trong quá trình phát triển cũng ảnh hưởng đến chất lượng phần mềm.

- Quy trình phát triển: Quy trình phát triển phần mềm có ảnh hưởng lớn đến chất lượng sản phẩm cuối cùng.

- Nhóm phát triển: Kỹ năng và kinh nghiệm của nhóm phát triển cũng là một yếu tố quan trọng.

Tóm lại, kiến trúc phần mềm đóng vai trò quan trọng trong việc đảm bảo chất lượng của phần mềm. Bằng cách lựa chọn kiến trúc phù hợp, sử dụng các công nghệ và công cụ thích hợp, và áp dụng các quy trình phát triển hiệu quả, các nhà phát triển có thể tạo ra các sản phẩm phần mềm có chất lượng cao, đáp ứng được nhu cầu của người dùng.

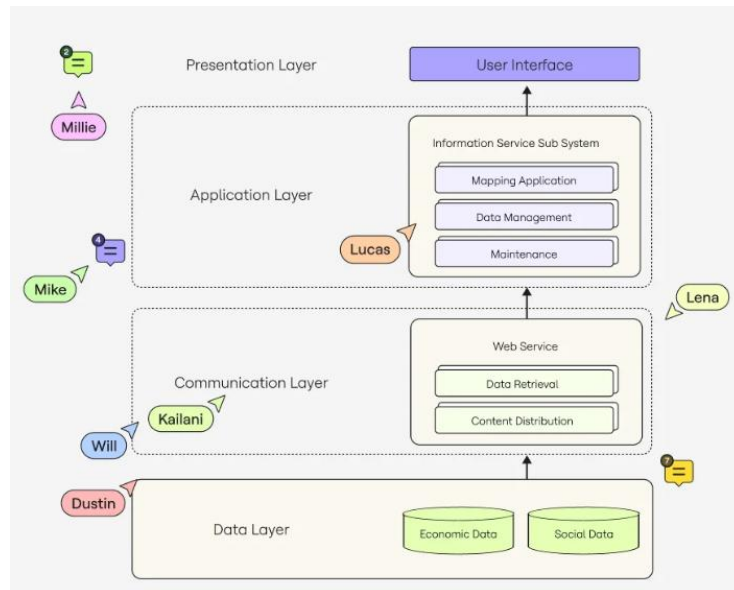
1.4. Vai trò của thiết kế trong quy trình phát triển phần mềm

Thiết kế đóng vai trò vô cùng quan trọng trong quy trình phát triển phần mềm, nó là cầu nối giữa ý tưởng và sản phẩm cuối cùng. Thiết kế không chỉ đơn thuần là tạo ra giao diện đẹp mắt mà còn đảm bảo phần mềm hoạt động hiệu quả, đáp ứng đúng nhu cầu của người dùng và dễ dàng bảo trì.

1. Định hình cấu trúc và chức năng của phần mềm:

Thiết kế giúp xác định rõ:

- Kiến trúc phần mềm: Cách tổ chức các thành phần của phần mềm, mối quan hệ giữa chúng.
- Luồng dữ liệu: Cách dữ liệu di chuyển và xử lý trong hệ thống.
- Giao diện người dùng (UI): Cách người dùng tương tác với phần mềm.
- Trải nghiệm người dùng (UX): Cảm nhận của người dùng khi sử dụng phần mềm.



Software architecture diagram

2. Cải thiện khả năng sử dụng:

- Giao diện trực quan, dễ hiểu: Giúp người dùng nhanh chóng làm quen và sử dụng phần mềm.
- Tương tác mượt mà: Đảm bảo các thao tác của người dùng được thực hiện một cách trơn tru.
- Trải nghiệm người dùng tốt: Tạo cảm giác hài lòng và thoải mái cho người dùng.

3. Giảm thiểu rủi ro và chi phí:

- Phát hiện lỗi sớm: Các vấn đề về thiết kế có thể được phát hiện và sửa chữa ở giai đoạn sớm, giúp giảm thiểu chi phí sửa chữa sau này.
- Cải thiện chất lượng: Một thiết kế tốt sẽ giúp tạo ra sản phẩm chất lượng cao, ít lỗi hơn.

- Tối ưu hóa quá trình phát triển: Thiết kế chi tiết giúp lập trình viên hiểu rõ yêu cầu và thực hiện công việc hiệu quả hơn.

4. Tăng tính bảo trì:

- Cấu trúc rõ ràng: Dễ dàng tìm kiếm và sửa lỗi, nâng cấp phần mềm.
- Mô tả chi tiết: Tài liệu thiết kế chi tiết giúp các lập trình viên mới dễ dàng tiếp cận và hiểu rõ hệ thống.



wellstructured codebase

5. Tăng tính linh hoạt và mở rộng:

- Thiết kế mô đun: Các thành phần của phần mềm được chia nhỏ, dễ dàng thay đổi hoặc thêm mới.
- Khả năng thích ứng: Phần mềm có thể dễ dàng thích ứng với các thay đổi trong yêu cầu của người dùng.

1.5. Chiến lược và phương pháp thiết kế.

1.5.1 Các chiến lược thiết kế phần mềm

Thiết kế phần mềm là một quá trình sáng tạo và có hệ thống, nhằm tạo ra các sản phẩm phần mềm đáp ứng nhu cầu và mong đợi của người dùng. Để đạt được điều này, các

nhà thiết kế phần mềm thường áp dụng những chiến lược khác nhau. Dưới đây là một số chiến lược phổ biến:

1. Thiết kế tập trung người dùng (User-centered design):

- Đặt người dùng làm trung tâm: Hiểu sâu sắc về người dùng, hành vi, nhu cầu và mong muốn của họ.
- Tạo ra các persona: Xây dựng các đại diện người dùng lý tưởng để giúp hình dung rõ hơn về đối tượng mục tiêu.
- Thử nghiệm liên tục: Thu thập phản hồi từ người dùng để cải thiện sản phẩm.

2. Thiết kế dựa trên dữ liệu (Data-driven design):

- Sử dụng dữ liệu để đưa ra quyết định: Dựa vào số liệu thống kê, hành vi người dùng để tối ưu hóa thiết kế.
- Phân tích dữ liệu: Tìm kiếm các xu hướng, mẫu hình để hiểu rõ hơn về cách người dùng tương tác với sản phẩm.
- Thử nghiệm A/B: So sánh các phiên bản thiết kế khác nhau để chọn ra phiên bản hiệu quả nhất.

3. Thiết kế linh hoạt (Agile design):

- Lặp đi lặp lại: Phát triển sản phẩm theo các vòng lặp ngắn, liên tục điều chỉnh dựa trên phản hồi.
- Ưu tiên tính năng: Tập trung vào việc phát triển các tính năng có giá trị nhất cho người dùng.
- Cộng tác chặt chẽ: Tạo ra một môi trường làm việc linh hoạt, khuyến khích sự hợp tác giữa các thành viên trong nhóm.

4. Thiết kế hệ thống (Systems design): Thiết kế hệ thống (Systems design) là quá trình xác định kiến trúc, thành phần và giao diện của một hệ thống. Mục tiêu chính của thiết kế hệ thống là tạo ra một hệ thống đáp ứng yêu cầu của người dùng và các bên liên quan.

- Xem xét toàn bộ hệ thống: Đảm bảo sản phẩm hoạt động tốt trong một hệ thống lớn hơn.

- Tích hợp với các hệ thống khác: Xây dựng các giao diện để kết nối với các hệ thống hiện có.
- Đảm bảo tính tương thích: Đảm bảo sản phẩm hoạt động tốt trên các nền tảng và thiết bị khác nhau.
- Các phương pháp thiết kế phổ biến:
 - Phương pháp Waterfall: Quy trình tuyến tính, các giai đoạn được thực hiện tuần tự.
 - Phương pháp Agile: Quy trình lặp đi lặp lại, linh hoạt, tập trung vào việc giao hàng nhanh chóng các tính năng có giá trị.
 - Phương pháp Design Thinking: Phương pháp tập trung vào việc hiểu người dùng, tạo ý tưởng, xây dựng nguyên mẫu và thử nghiệm.
 - Phương pháp Lean UX: Áp dụng các nguyên tắc của Lean để tối ưu hóa quy trình thiết kế, giảm thiểu lãng phí và tăng tốc độ phát triển.

Các yếu tố quan trọng khác trong thiết kế:

- Hiểu rõ yêu cầu: Xác định rõ các yêu cầu chức năng và phi chức năng của sản phẩm.
- Tạo wireframe và prototype: Tạo các bản phác thảo và mô hình tương tác để trực quan hóa ý tưởng.
- Đảm bảo tính khả dụng: Đảm bảo sản phẩm dễ sử dụng và truy cập.
- Đảm bảo tính bảo mật: Bảo vệ dữ liệu của người dùng.

Lựa chọn chiến lược thiết kế phù hợp:

Việc lựa chọn chiến lược thiết kế phụ thuộc vào nhiều yếu tố, bao gồm:

- Quy mô và tính phức tạp của dự án:
- Ngân sách:
- Thời gian:
- Nhu cầu của khách hàng:

Thiết kế phần mềm là một quá trình sáng tạo và không ngừng phát triển. Bằng cách áp dụng các chiến lược và phương pháp thiết kế phù hợp, các nhà phát triển có thể tạo ra

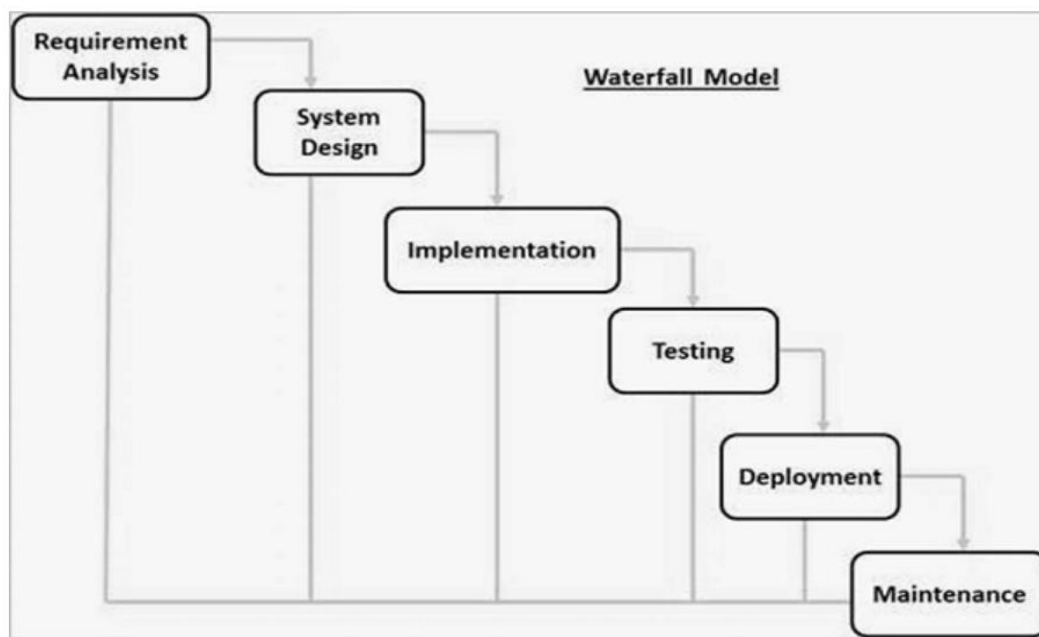
các sản phẩm phần mềm chất lượng cao, đáp ứng nhu cầu của người dùng và đạt được thành công.

1.5.2 Các phương pháp thiết kế phần mềm

Thiết kế phần mềm là một trong những bước quan trọng nhất trong quá trình phát triển phần mềm. Để đảm bảo rằng phần mềm được phát triển hiệu quả và đáp ứng được các yêu cầu của người dùng, việc lựa chọn phương pháp thiết kế phần mềm phù hợp là vô cùng quan trọng. Hiện nay, có nhiều phương pháp thiết kế phần mềm khác nhau, mỗi phương pháp đều có ưu điểm và nhược điểm riêng. Trong tài liệu này, chúng ta sẽ cùng tìm hiểu về các phương pháp thiết kế phần mềm phổ biến, phân tích, đánh giá và so sánh giữa các phương pháp để giúp bạn hiểu rõ hơn và lựa chọn phương pháp phù hợp cho doanh nghiệp của mình.

1. Phương pháp Waterfall (Mô hình thác nước)

Phương pháp thác nước hay Waterfall là một trong những phương pháp thiết kế phần mềm truyền thống và ra đời lâu nhất. Hình dung một cách trực quan, phương pháp thiết kế phần mềm này có cấu trúc giống như thác nước được chảy từ trên xuống và không sự thay đổi hay quay lại giữa các giai đoạn. Các giai đoạn chính của phương pháp thác nước bao gồm: Phân tích yêu cầu -> thiết kế -> kiểm thử -> triển khai và bảo trì. Về cơ bản



đây là phương pháp phù hợp với cơ cấu của một mô hình truyền thống nơi mà sự quản lý như một dòng chảy từ trên xuống dưới các phòng ban.

Ưu điểm:

- Dễ hiểu và dễ thực hiện do có cấu trúc rõ ràng.
- Dễ dàng quản lý tiến độ và chi phí do các giai đoạn được định rõ từ đầu.
- Phù hợp với các dự án có yêu cầu rõ ràng và không thay đổi nhiều.
- Nhược điểm:
 - Thiếu linh hoạt, khó khăn trong việc thay đổi yêu cầu khi dự án đã bắt đầu.
 - Khả năng phát hiện lỗi muộn, gây tốn kém chi phí sửa chữa.

Mô hình thác nước phù hợp với các dự án có yêu cầu cố định, ít thay đổi và không đòi hỏi sự linh hoạt cao. Nó thường được sử dụng trong các dự án phần mềm cho các ngành công nghiệp có yêu cầu an toàn và bảo mật cao như hàng không, quân sự và y tế.

2. Phương pháp Agile (Linh hoạt)



Agile là một phương pháp thiết kế phần mềm linh hoạt, tập trung vào việc phát triển phần mềm theo từng giai đoạn nhỏ gọi là sprint (chu kỳ phát triển ngắn). Agile khuyến khích sự hợp tác chặt chẽ giữa các thành viên trong nhóm và khách hàng, cho phép thay đổi yêu cầu trong quá trình phát triển để đáp ứng tốt hơn các nhu cầu của người dùng.

So với mô hình thác nước trên thì Agile nhấn mạnh vào sự linh hoạt. Sự ra đời của nó gắn với thời kỳ khi các công ty khởi nghiệp mới ở Mỹ phải thích ứng làn sóng thay đổi, cập nhật và cải tiến về công nghệ bắt buộc quá trình thiết kế phần mềm này có các nguyên tắc về sự linh hoạt. Linh hoạt cũng là nguyên tắc quan trọng nhất để ưu tiên việc ra được một sản phẩm (phần mềm) giúp khách hàng tối ưu về chi phí tiền bạc, thời gian kèm theo khả năng mở rộng

Ưu điểm:

- Linh hoạt và dễ dàng thay đổi yêu cầu khi cần thiết.
- Tăng cường sự hợp tác và giao tiếp giữa các thành viên trong nhóm.
- Khả năng phát hiện lỗi sớm và cải thiện chất lượng phần mềm liên tục.

Nhược điểm:

- Khó quản lý tiến độ và chi phí do tính linh hoạt cao.
- Đòi hỏi sự cam kết và kỷ luật từ tất cả các thành viên trong nhóm.

Agile phù hợp với các dự án có yêu cầu thay đổi liên tục và đòi hỏi sự phản ứng nhanh chóng từ nhóm phát triển. Nó thường được sử dụng trong các dự án phát triển phần mềm thương mại, ứng dụng web và các dự án khởi nghiệp.

3. Phương pháp Scrum

Scrum là một trong những khung làm việc (framework) phổ biến của Agile. Scrum tổ chức công việc thành các sprint ngắn, thường kéo dài từ 2 đến 4 tuần. Trong mỗi sprint, nhóm phát triển sẽ hoàn thành một phần chức năng cụ thể của phần mềm và trình bày cho khách hàng để nhận phản hồi. Scrum phù hợp với các dự án có yêu cầu thay đổi thường xuyên và cần phản hồi nhanh từ khách hàng. Nó thường được sử dụng trong các dự án phát triển phần mềm thương mại, ứng dụng web và các dự án khởi nghiệp.



Ưu điểm của phương pháp Scrum là tập trung vào việc giao tiếp và hợp tác chặt chẽ giữa các thành viên trong nhóm từ đó cho phép phản hồi nhanh chóng từ khách hàng và điều chỉnh kịp thời. Scrum thường đi đôi với kế hoạch và chiến lược của hàng, để ví dụ cho việc này có thể so sánh nó với thác nước. Phương pháp thác nước đòi hỏi phần mềm phải là sản phẩm đầu cuối và sẵn sàng hoạt động một một cách hoàn thiện thì ngược lại Scrum có thể linh hoạt đi theo từng giai đoạn chiến lược mở rộng kinh doanh của khách hàng. Ví dụ, phương pháp Scrum sẽ tạo ra các phần mềm hoạt sớm và liên tục update theo chiến lược kinh doanh của doanh nghiệp.

Ưu điểm và như vậy nhưng ngược lại nó cũng có những nhược điểm lớn nhất liên quan tới con người như: đòi hỏi sự cam kết cao từ các thành viên trong nhóm và khách hàng; khó khăn trong việc duy trì tập trung và kỷ luật trong nhóm khách hàng.

4. Phương pháp Kanban



Kanban là một phương pháp thiết kế phần mềm tập trung vào việc tối ưu hóa quy trình làm việc và giảm thiểu lãng phí. Kanban sử dụng bảng Kanban để trực quan hóa quy trình làm việc, giúp nhóm phát triển theo dõi tiến độ và phát hiện các tắc nghẽn trong quy trình. Kanban phù hợp với các dự án có quy trình làm việc phức tạp và cần tối ưu hóa quy trình. Nó thường được sử dụng trong các dự án phát triển phần mềm thương mại, ứng dụng web và các dự án khởi nghiệp.

Ưu điểm:

- Linh hoạt và dễ dàng thích nghi với các thay đổi trong quy trình làm việc.
- Tăng cường sự minh bạch và hiển thị tiến độ công việc.
- Giảm thiểu lãng phí và tối ưu hóa quy trình làm việc.
- Nhược điểm:
 - Đòi hỏi sự cam kết và kỷ luật từ tất cả các thành viên trong nhóm.
 - Khó khăn trong việc duy trì sự cân bằng giữa các công việc và tránh tình trạng quá tải.

5. Phương pháp Lean

Lean là một phương pháp thiết kế phần mềm tập trung vào việc giảm thiểu lãng phí và tối ưu hóa quy trình làm việc. Lean áp dụng các nguyên tắc từ sản xuất tinh gọn (lean manufacturing) vào phát triển phần mềm, bao gồm việc loại bỏ các hoạt động không giá trị gia tăng và tập trung vào việc cung cấp giá trị cho khách hàng.



Lean phù hợp với các dự án cần tối ưu hóa quy trình làm việc và giảm thiểu lãng phí. Nó thường được sử dụng trong các dự án phát triển phần mềm thương mại, ứng dụng web và các dự án khởi nghiệp. Ưu điểm của Lean được thấy là giảm thiểu lãng phí và tập trung vào tối ưu quy trình làm việc; tăng cường sự tập trung vào giá trị của khách hàng; cải thiện sự hiệu quả và tập trung vào chất lượng của phần mềm. Ngược lại nhược điểm của Lean là khó khăn trong việc cân bằng giữa các nhóm công việc;

6. Vậy phương pháp thiết kế phần mềm nào phù hợp nhất với bối cảnh hiện tại?

Doanh nghiệp đều biết rằng các biến động gần đây về đại dịch Covid và cuộc suy thoái vẫn đang kéo dài đã và vẫn đang làm thay đổi chúng ta nhận thức về các vấn đề như tính linh hoạt, tính hiệu suất và sự cắt giảm chi phí. Khi lựa chọn phương pháp thiết kế

phần mềm chúng ta có thể dựa trên các nguyên tắc ưu tiên cho tình trạng chung trên nên có thể thấy phương pháp thiết kế phần mềm phù hợp nhất hiện nay cho doanh nghiệp là Agile và Scrum đã phân tích phía trên. Nếu doanh nghiệp của bạn cần sự linh hoạt và khả năng phản ứng nhanh với thay đổi: Agile hoặc Scrum là những phương pháp phù hợp. Scrum đặc biệt hiệu quả nếu bạn muốn chia nhỏ chiến lược kinh doanh kết hợp với công nghệ theo từng giai đoạn.

Nếu doanh nghiệp của bạn muốn tối ưu hóa quy trình làm việc và giảm thiểu lãng phí thì Lean và Kanban là những lựa chọn tốt – tốt nhất đối với những doanh nghiệp tập trung vào khía cạnh sản xuất và nâng cao chất lượng cạnh tranh. Kanban giúp bạn trực quan hóa quy trình và phát hiện các tắc nghẽn, trong khi Lean tập trung vào việc loại bỏ các hoạt động không giá trị gia tăng.

Câu hỏi và bài tập cuối chương 1:

Câu hỏi trắc nghiệm:

1. Mục tiêu chính của kiến trúc phần mềm là gì?

- a) Tạo ra mã nguồn phức tạp và khó hiểu.
- b) Giảm thiểu công sức bảo trì và nâng cấp phần mềm.
- c) Tối ưu hóa hiệu suất phần cứng.
- d) Tập trung vào tính thẩm mỹ của giao diện.

2. Kiến trúc phần mềm tốt giúp:

- a) Tạo ra các thành phần không thể tái sử dụng.
- b) Tăng thời gian phát triển.
- c) Tạo ra các thành phần có thể tái sử dụng và dễ dàng thích ứng với thay đổi.
- d) Làm giảm khả năng thích ứng của hệ thống.

3. Trong quá trình thiết kế một hệ thống đặt vé trực tuyến, nhóm thiết kế nhận thấy người dùng gặp khó khăn trong việc chọn chỗ ngồi. Giải pháp UCD nào sau đây sẽ hiệu quả nhất?

- a) Loại bỏ tính năng chọn chỗ ngồi để đơn giản hóa quy trình.
- b) Cung cấp sơ đồ chỗ ngồi trực quan và dễ tương tác.

- c) Tăng giá vé cho các chỗ ngồi đẹp.
- d) Tự động chọn chỗ ngồi cho người dùng.

4. Một trang web tin tức muốn tăng tỷ lệ nhấp chuột (click-through rate) vào các bài viết. Dữ liệu nào sau đây sẽ hữu ích nhất để phân tích?

- a) Dữ liệu về thời gian đọc trung bình của người dùng.
- b) Dữ liệu về các tiêu đề bài viết và hình ảnh thu nhỏ (thumbnail) có hiệu quả cao.
- c) Dữ liệu về số lượng bình luận trên mỗi bài viết.
- d) Dữ liệu về số lượng người theo dõi trang web trên mạng xã hội.

5. Trong một ứng dụng web, kết nối nào cho phép trình duyệt web giao tiếp với máy chủ web?

- a) SQL.
- b) HTTP.
- c) XML.
- d) JSON.

6. Kiến trúc phần mềm ảnh hưởng chủ yếu đến những yếu tố nào của hệ thống?

- a) Hiệu năng, khả năng bảo trì, khả năng mở rộng, độ tin cậy.
- b) Màu sắc chủ đạo của giao diện người dùng.
- c) Ngôn ngữ lập trình được sử dụng.
- d) Hệ điều hành mà phần mềm chạy trên đó.

7. Kiến trúc phần mềm giúp phát hiện và giải quyết vấn đề thiết kế khi nào?

- a) Chỉ sau khi triển khai phần mềm.
- b) Chỉ trong giai đoạn kiểm thử.
- c) Sớm trong quá trình phát triển.
- d) Không giúp phát hiện vấn đề thiết kế.

8. Thuộc tính chất lượng nào liên quan đến khả năng phần mềm bảo vệ dữ liệu và chống lại các cuộc tấn công?

- a) Khả năng sử dụng
- b) Khả năng bảo mật

- c) Khả năng kiểm thử
- d) Khả năng tái sử dụng

9. Một ứng dụng di động muốn tăng thời gian tương tác của người dùng. Chỉ số nào sau đây quan trọng nhất cần theo dõi?

- a) Số lượt tải xuống ứng dụng.
- b) Thời gian trung bình người dùng sử dụng ứng dụng mỗi ngày.
- c) Số lượng quảng cáo được hiển thị.
- d) Số lượng nhân viên hỗ trợ khách hàng.

10. Một ứng dụng học ngoại ngữ muốn tăng tỷ lệ người dùng hoàn thành khóa học. Dữ liệu nào sau đây sẽ hữu ích nhất để phân tích?

- a) Dữ liệu về thời gian người dùng dành cho mỗi bài học và các bài học mà họ thường bỏ dở.
- b) Dữ liệu về số lượng người dùng tải xuống ứng dụng.
- c) Dữ liệu về các ngôn ngữ được người dùng yêu thích nhất.
- d) Dữ liệu về số lượng nhân viên của công ty.

Câu hỏi lý thuyết:

1. Định nghĩa kiến trúc phần mềm là gì?
2. Tại sao kiến trúc phần mềm lại quan trọng trong quá trình phát triển phần mềm?
3. Quality Attributes (Các thuộc tính chất lượng) của phần mềm là gì? Hãy liệt kê một số thuộc tính chất lượng quan trọng?
4. Sự khác biệt giữa kiến trúc phần mềm và thiết kế phần mềm là gì?
5. Hãy giải thích sự khác biệt giữa chiến lược và phương pháp thiết kế phần mềm?
6. Nêu ưu điểm và nhược điểm của phương pháp Waterfall và Agile?
7. Design Thinking khác với các phương pháp khác như thế nào?
8. Scrum và Kanban có điểm gì giống và khác nhau?

9. So sánh và đối chiếu giữa phương pháp Waterfall và Agile trong các dự án khác nhau.
10. Nêu những trường hợp nên áp dụng phương pháp Design Thinking.
11. Trong một dự án phát triển phần mềm, khi nào bạn nên sử dụng phương pháp Waterfall và khi nào nên sử dụng Agile?
12. Làm thế nào để áp dụng Design Thinking vào quá trình thiết kế giao diện người dùng?

Bài tập vận dụng:

1. Lập kế hoạch cho một dự án phần mềm nhỏ, sử dụng một trong các phương pháp đã học.

CHƯƠNG 2: MỘT SỐ KIỂU KIẾN TRÚC PHẦN MỀM

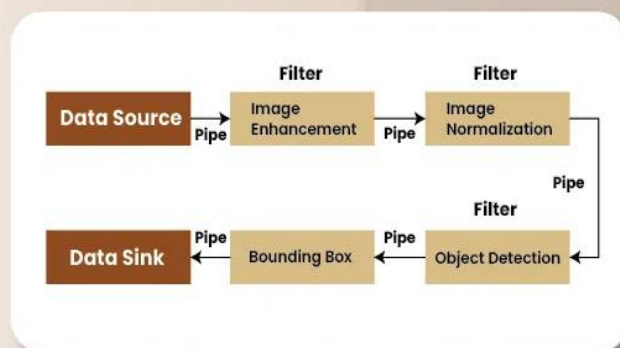
Kiến trúc phần mềm là bản thiết kế tổng quan của một hệ thống phần mềm, mô tả các thành phần, mối quan hệ giữa chúng và cách chúng tương tác với nhau. Việc lựa chọn kiến trúc phù hợp đóng vai trò quan trọng trong việc đảm bảo hiệu suất, khả năng mở rộng và dễ bảo trì của hệ thống. Dưới đây là một số kiểu kiến trúc phần mềm phổ biến:

- Kiến trúc ống và lọc
- Kiến trúc hướng sự kiện
- Kiến trúc phân lớp
- Kiến trúc gọi và trả lời
- Kiến trúc hướng đối tượng
- Kiến trúc hướng dịch vụ

2.1. Kiến trúc ống và lọc

Kiến trúc ống và lọc (Pipe and Filter) là một trong những mô hình kiến trúc phần mềm cơ bản và phổ biến. Trong mô hình này, dữ liệu được xử lý tuần tự qua một loạt các thành phần độc lập gọi là "ống" (pipe), mỗi ống thực hiện một phép biến đổi cụ thể trên dữ liệu. Dữ liệu đầu ra của một ống sẽ trở thành dữ liệu đầu vào của ống tiếp theo.

Pipe & Filter Architecture System Design



Pipe and Filter Architecture

Trong thiết kế hệ thống, kiến trúc Pipe và Filter cấu trúc các tác vụ xử lý thành một chuỗi các giai đoạn được gọi là "pipe", trong đó mỗi giai đoạn lọc và chuyển đổi dữ liệu theo từng bước. Khung mô-đun này cho phép các bộ lọc hoạt động độc lập, cải thiện khả năng mở rộng và khả năng tái sử dụng. Mỗi bộ lọc được giao các hoạt động cụ thể, chẳng hạn như xác thực hoặc định dạng dữ liệu và chuyển đầu ra của nó sang giai đoạn tiếp theo thông qua các pipe được kết nối với nhau. Kiến trúc này thúc đẩy tính linh hoạt và dễ bảo trì bằng cách cô lập các mối quan tâm và cho phép các thành phần được tái sử dụng trên nhiều hệ thống khác nhau.

Kiến trúc ống và bộ lọc là gì?

Kiến trúc Pipe and Filter là một mẫu thiết kế chia một quy trình thành một loạt các bước riêng biệt, được gọi là bộ lọc, được liên kết bằng các kênh được gọi là ống. Mỗi bộ lọc được dành riêng cho một chức năng xử lý cụ thể, cho dù nó liên quan đến việc chuyển đổi, xác thực hay tổng hợp dữ liệu. Dữ liệu chảy qua các bộ lọc này thông qua các ống, mang đầu ra của một bộ lọc đến đầu vào của một bộ lọc khác.

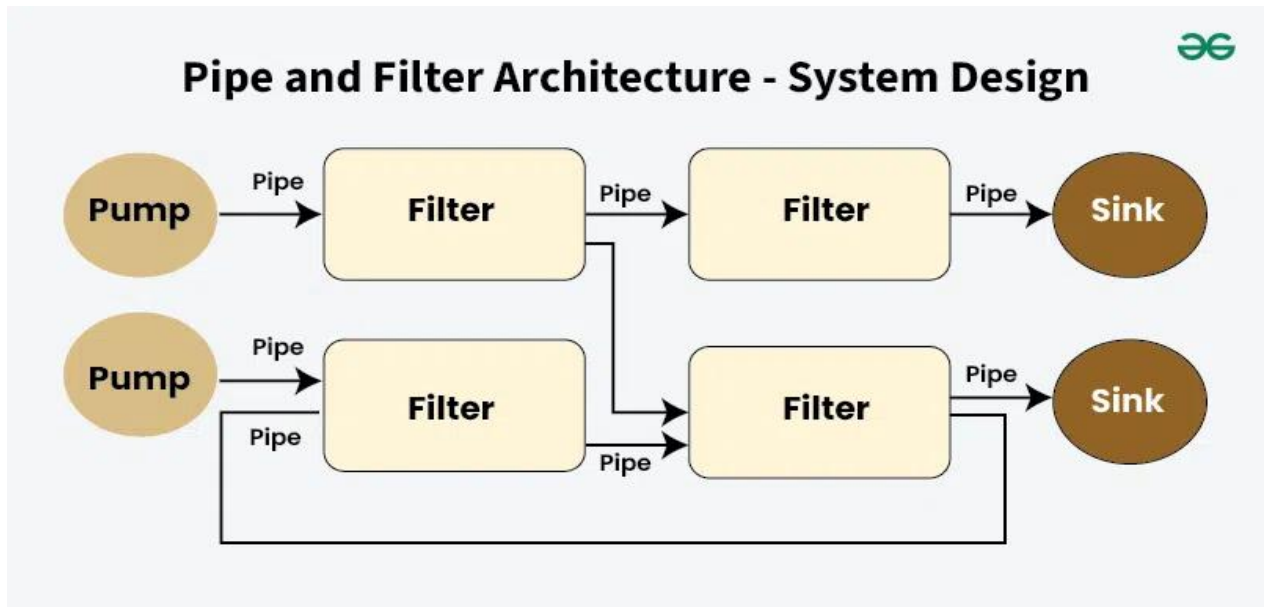
- Kiến trúc này tăng cường tính mô-đun, vì mỗi bộ lọc hoạt động độc lập và tập trung vào một chức năng riêng biệt. Nó cũng thúc đẩy khả năng tái sử dụng, cho phép sử dụng bộ lọc trên các hệ thống hoặc ứng dụng khác nhau.

- Hơn nữa, nó vừa linh hoạt vừa có khả năng mở rộng; các bộ lọc có thể được thêm vào, xóa đi hoặc sắp xếp lại mà không ảnh hưởng nhiều đến toàn bộ hệ thống và nhiều phiên bản bộ lọc có thể hoạt động đồng thời để quản lý các tập dữ liệu lớn hơn.

Cấu trúc có tổ chức này khiến kiến trúc Pipe và Filter trở thành lựa chọn ưu tiên cho các tác vụ như xử lý dữ liệu, trình biên dịch và các ứng dụng yêu cầu chuyển đổi dữ liệu theo thứ tự và tuần tự.

Thành phần của Kiến trúc ống và bộ lọc

Kiến trúc có tính mô-đun cao, với mỗi bộ lọc hoạt động độc lập, đơn giản hóa việc hiểu hệ thống, bảo trì và mở rộng. Bộ lọc có thể được thêm vào, xóa hoặc sắp xếp lại mà không ảnh hưởng đáng kể đến toàn bộ hệ thống và các đường ống song song có thể tăng thông lượng cho khối lượng dữ liệu lớn hơn.



Kiến trúc ống và bộ lọc

- Máy bơm: Các thành phần này khởi tạo quy trình bằng cách hoạt động như nguồn dữ liệu, đưa dữ liệu vào hệ thống và bắt đầu lưu thông qua đường ống.
- Bộ lọc: Mỗi bộ lọc thực hiện một nhiệm vụ cụ thể, độc lập, có thể là chuyển đổi, xác thực hoặc xử lý dữ liệu trước khi chuyển tiếp. Trong thiết lập, có hai cấp độ bộ lọc; cấp độ đầu tiên xử lý dữ liệu từ máy bơm và chuyển đến bộ lọc thứ hai, tiếp tục xử lý trước khi chuyển tiếp.
- Ống: Ống đóng vai trò là kênh để di chuyển dữ liệu giữa các bộ lọc, liên kết từng thành phần theo trình tự và đảm bảo truyền dữ liệu trơn tru. Trong sơ đồ, ống được mô tả là các mũi tên kết nối các thành phần.
- Sink: Đây là điểm cuối nơi dữ liệu đã xử lý cuối cùng được thu thập hoặc sử dụng. Sau khi đi qua tất cả các bộ lọc, dữ liệu sẽ đến sink, hoàn thành hành trình pipeline của nó.
- Xử lý song song: Kiến trúc cũng hỗ trợ cấu trúc song song, trong đó hai đường ống độc lập hoạt động song song. Mỗi đường ống bắt đầu bằng máy bơm riêng, xử lý dữ liệu thông qua một loạt các bộ lọc và kết thúc tại các bồn chứa riêng biệt. Điều này cho biết khả năng xử lý đồng thời các luồng dữ liệu khác nhau mà không có xung đột

Đặc điểm của kiến trúc ống và bộ lọc

Kiến trúc Pipe and Filter trong thiết kế hệ thống sở hữu một số đặc điểm chính khiến nó trở thành một mẫu thiết kế hiệu quả và phổ biến cho nhiều ứng dụng. Sau đây là những đặc điểm chính của nó:

- Tính mô-đun: Mỗi bộ lọc là một thành phần độc lập thực hiện một nhiệm vụ cụ thể. Sự tách biệt này cho phép dễ dàng hiểu, phát triển và bảo trì từng bộ lọc mà không ảnh hưởng đến toàn bộ hệ thống.
- Khả năng tái sử dụng: Bộ lọc có thể được tái sử dụng trên các hệ thống khác nhau hoặc trong các phần khác nhau của cùng một hệ thống. Điều này làm giảm sự trùng lặp công sức và thúc đẩy việc sử dụng hiệu quả tài nguyên.
- Khả năng kết hợp: Bộ lọc có thể được kết hợp theo nhiều trình tự khác nhau để tạo ra các đường ống xử lý phức tạp. Tính linh hoạt này cho phép các nhà thiết kế xây dựng quy trình làm việc tùy chỉnh chỉ bằng cách sắp xếp lại hoặc kết hợp các bộ lọc.
- Khả năng mở rộng: Kiến trúc hỗ trợ xử lý song song bằng cách chạy nhiều phiên bản bộ lọc. Điều này nâng cao khả năng xử lý khối lượng dữ liệu lớn hơn của hệ thống và cải thiện hiệu suất.
- Khả năng bảo trì: Việc cô lập các chức năng thành các bộ lọc riêng biệt giúp đơn giản hóa việc gỡ lỗi và bảo trì. Các thay đổi đối với một bộ lọc không ảnh hưởng đến các bộ lọc khác, giúp việc cập nhật và sửa lỗi dễ quản lý hơn.

Nguyên tắc thiết kế cho kiến trúc ống và bộ lọc

Kiến trúc Pipe and Filter tuân thủ một số nguyên tắc thiết kế cơ bản đảm bảo tính hiệu quả, độ bền và khả năng bảo trì. Dưới đây là giải thích chi tiết về các nguyên tắc này:

- Tách biệt các mối quan tâm: Mỗi bộ lọc chịu trách nhiệm cho một nhiệm vụ cụ thể. Bằng cách tách biệt các nhiệm vụ, hệ thống có thể được phát triển, thử nghiệm và bảo trì dễ dàng hơn. Nguyên tắc này đảm bảo rằng những thay đổi trong một phần của hệ thống có tác động tối thiểu đến các phần khác.
- Tính mô-đun: Hệ thống được chia thành các mô-đun riêng biệt gọi là bộ lọc. Mỗi bộ lọc là một đơn vị xử lý độc lập có thể được phát triển, thử nghiệm và bảo trì riêng biệt. Tính mô-đun này giúp đơn giản hóa việc gỡ lỗi và cho phép nâng cấp hoặc thay thế dễ dàng hơn.

- Pipeline Parallelism: Kiến trúc hỗ trợ xử lý song song bằng cách cho phép nhiều phiên bản bộ lọc chạy đồng thời. Điều này tăng cường khả năng xử lý khối lượng dữ liệu lớn hơn của hệ thống và cải thiện hiệu suất, đặc biệt là đối với các ứng dụng dữ liệu chuyên sâu.

- Bộ lọc không trạng thái: Bộ lọc thường không trạng thái, nghĩa là chúng không lưu giữ dữ liệu giữa các bước xử lý. Điều này đơn giản hóa việc thiết kế và triển khai bộ lọc và tăng cường khả năng mở rộng. Bộ lọc không trạng thái có thể dễ dàng được sao chép để xử lý song song.

- Xử lý lỗi và cô lập lỗi: Mỗi bộ lọc phải xử lý lỗi nội bộ và đảm bảo chỉ có dữ liệu hợp lệ mới được chuyển đến giai đoạn tiếp theo. Lỗi trong một bộ lọc không được lan truyền qua đường ống, đảm bảo hệ thống vẫn mạnh mẽ và có khả năng chịu lỗi. Việc cô lập lỗi này giúp tăng cường độ tin cậy của hệ thống.

Lợi ích của kiến trúc ống và bộ lọc

Kiến trúc Pipe and Filter mang lại nhiều lợi ích khiến nó trở thành lựa chọn hấp dẫn để thiết kế các hệ thống phức tạp. Sau đây là một số lợi ích chính:

- Xử lý dữ liệu nâng cao: Kiến trúc này phù hợp với các ứng dụng yêu cầu xử lý dữ liệu tuần tự, chẳng hạn như chuyển đổi dữ liệu, xác thực và tổng hợp. Mỗi bộ lọc xử lý một bước cụ thể trong quy trình xử lý, đảm bảo luồng dữ liệu có trật tự và hiệu quả.

- Dễ hiểu: Luồng dữ liệu rõ ràng và tuyến tính từ bộ lọc này sang bộ lọc khác giúp hệ thống dễ hiểu và dễ hình dung. Sự đơn giản này hỗ trợ cho việc thiết kế, lập tài liệu và truyền đạt cấu trúc và chức năng của hệ thống.

- Cô lập lỗi: Lỗi trong một bộ lọc được cô lập và không lan truyền qua đường ống, đảm bảo hệ thống vẫn mạnh mẽ và có khả năng chịu lỗi. Mỗi bộ lọc có thể xử lý lỗi nội bộ, nâng cao độ tin cậy chung của hệ thống.

- Kiểm tra được cải thiện: Mỗi bộ lọc có thể được kiểm tra độc lập, giúp xác định và khắc phục sự cố dễ dàng hơn. Điều này cải thiện chất lượng hệ thống và giảm thời gian cần thiết để kiểm tra và gỡ lỗi.

- Chuẩn hóa: Giao diện thống nhất cho bộ lọc và đường ống thúc đẩy tính nhất quán trong thiết kế và triển khai. Chuẩn hóa này làm giảm sự phức tạp

- Tối ưu hóa tài nguyên: Bằng cách chia nhỏ quá trình xử lý thành các tác vụ nhỏ hơn, dễ quản lý hơn, hệ thống có thể tối ưu hóa việc sử dụng tài nguyên. Bộ lọc có thể được phân bổ tài nguyên dựa trên nhu cầu cụ thể của chúng, cải thiện hiệu quả chung của hệ thống.

Những thách thức của kiến trúc ống và bộ lọc

Trong khi kiến trúc Pipe and Filter mang lại nhiều lợi ích, nó cũng đi kèm với một số thách thức cần được giải quyết để triển khai hiệu quả. Sau đây là một số thách thức chính:

- Chi phí hiệu suất: Việc truyền dữ liệu giữa các bộ lọc qua đường ống có thể gây ra chi phí hiệu suất, đặc biệt là nếu có nhiều bộ lọc hoặc nếu dữ liệu cần chuyển đổi thường xuyên. Điều này có thể làm chậm tốc độ xử lý chung.
- Độ trễ: Bản chất tuần tự của đường ống xử lý có thể gây ra độ trễ, đặc biệt là trong các ứng dụng thời gian thực hoặc độ trễ thấp. Mỗi bộ lọc sẽ làm tăng thời gian xử lý chung, có thể không phù hợp với các tác vụ nhạy cảm về thời gian.
- Xử lý lỗi phức tạp: Mặc dù việc cô lập lỗi là một lợi ích, nhưng việc quản lý lỗi trên nhiều bộ lọc có thể trở nên phức tạp. Đảm bảo rằng mỗi bộ lọc xử lý và truyền đạt lỗi đúng cách có thể đòi hỏi thêm nỗ lực và sự phối hợp.
- Quản lý trạng thái: Bộ lọc không trạng thái dễ triển khai hơn nhưng có thể không phù hợp với mọi ứng dụng. Khi cần quản lý trạng thái, nó có thể làm phức tạp việc thiết kế và triển khai bộ lọc, đòi hỏi phải xử lý cẩn thận để duy trì tính nhất quán và tính chính xác.
- Sử dụng tài nguyên: Quản lý hiệu quả các tài nguyên, chẳng hạn như bộ nhớ và CPU, có thể là một thách thức. Các bộ lọc có thể có các yêu cầu tài nguyên khác nhau và việc cân bằng các yêu cầu này trên toàn hệ thống để tránh tắc nghẽn và đảm bảo sử dụng hiệu quả có thể phức tạp.

Chiến lược thực hiện

Việc triển khai kiến trúc Pipe and Filter đòi hỏi một cách tiếp cận chiến lược để đảm bảo hệ thống hiệu quả, có thể bảo trì và có thể mở rộng. Sau đây là các chiến lược chi tiết để triển khai kiến trúc này:

- Xác định giao diện rõ ràng:
 - Đầu vào/Đầu ra thống nhất: Thiết lập định dạng đầu vào và đầu ra thống nhất cho mỗi bộ lọc để đảm bảo dữ liệu lưu chuyển trơn tru giữa các bộ lọc.
 - Giao thức chuẩn hóa: Sử dụng giao thức truyền thông chuẩn hóa (ví dụ: HTTP, gRPC) để truyền thông giữa các tiến trình.
 - Thiết kế bộ lọc mô-đun:
 - Nguyên tắc trách nhiệm duy nhất: Mỗi bộ lọc phải thực hiện một nhiệm vụ cụ thể, giúp hệ thống dễ quản lý và gỡ lỗi hơn.
 - Đóng gói: Giữ logic bên trong của mỗi bộ lọc được ẩn đi, chỉ hiển thị những giao diện cần thiết.
- Bộ lọc không trạng thái:
 - Tính không trạng thái: Thiết kế bộ lọc ở chế độ không trạng thái bất cứ khi nào có thể để đơn giản hóa việc mở rộng quy mô và xử lý song song.
 - Quản lý trạng thái: Nếu cần quản lý trạng thái, hãy quản lý trạng thái bên ngoài hoặc đảm bảo trạng thái được cô lập và không ảnh hưởng đến các bộ lọc khác.
- Xử lý lỗi mạnh mẽ:
 - Ghi nhật ký lỗi: Đảm bảo rằng mỗi bộ lọc ghi lại lỗi theo cách nhất quán.
 - Suy giảm nhẹ nhàng: Thiết kế đường ống để xử lý lỗi nhẹ nhàng, chẳng hạn như bỏ qua dữ liệu có vấn đề hoặc sử dụng cơ chế dự phòng.
- Kiểm tra và xác nhận:
 - Kiểm tra đơn vị: Kiểm tra kỹ lưỡng từng bộ lọc một cách độc lập để đảm bảo nó thực hiện đúng chức năng mong muốn.
 - Kiểm thử tích hợp: Xác thực toàn bộ quy trình để đảm bảo các bộ lọc hoạt động liền mạch và dữ liệu chảy chính xác.
- Những cân nhắc về bảo mật:
 - Mã hóa dữ liệu: Đảm bảo dữ liệu được mã hóa khi truyền và khi lưu trữ để bảo vệ thông tin nhạy cảm.

- Kiểm soát truy cập: Triển khai kiểm soát truy cập chặt chẽ để ngăn chặn truy cập trái phép vào bộ lọc và dữ liệu.
- Quản lý phiên bản và triển khai:
- Kiểm soát phiên bản: Sử dụng hệ thống kiểm soát phiên bản để quản lý các thay đổi đối với bộ lọc và cấu hình đường ống.
- Triển khai liên tục: Triển khai các hoạt động triển khai liên tục để đảm bảo cập nhật và khôi phục liên mạch với sự gián đoạn tối thiểu.

Các trường hợp sử dụng và ứng dụng phổ biến

Kiến trúc Pipe and Filter là một mẫu thiết kế đa năng có thể áp dụng trong nhiều lĩnh vực và ứng dụng khác nhau. Sau đây là một số trường hợp sử dụng và ứng dụng phổ biến cho kiến trúc này:

- Đường ống xử lý dữ liệu
 - Xử lý văn bản: Các đường ống Unix (ví dụ: grep, awk, sed) cho phép nối các lệnh để xử lý và chuyển đổi dữ liệu văn bản một cách hiệu quả.
 - Trình biên dịch: Sử dụng một loạt các bộ lọc để phân tích từ vựng, phân tích cú pháp, phân tích ngữ nghĩa, tối ưu hóa và tạo mã.
- Xử lý luồng
 - Phân tích thời gian thực: Các hệ thống như Apache Flink, Apache Storm và Apache Kafka Streams xử lý các luồng dữ liệu liên tục theo thời gian thực.
 - Xử lý phương tiện: Các khung như GStreamer xử lý các luồng âm thanh và video, thực hiện các hoạt động như giải mã, lọc và mã hóa.
- Quy trình ETL (Trích xuất, Chuyển đổi, Tải)
 - Tích hợp dữ liệu: Các công cụ như Apache NiFi và Talend thực hiện trích xuất, chuyển đổi và tải dữ liệu giữa các nguồn dữ liệu và đích khác nhau.
 - Làm sạch dữ liệu: Chuyển đổi và làm sạch dữ liệu qua nhiều giai đoạn trước khi tải dữ liệu vào cơ sở dữ liệu hoặc kho dữ liệu.
- Dịch vụ vi mô và Kiến trúc hướng dịch vụ (SOA)
 - Tự động hóa quy trình làm việc: Các dịch vụ vi mô hoạt động như các bộ lọc xử lý và chuyển đổi dữ liệu khi dữ liệu đi qua một loạt các dịch vụ.

- Quản lý quy trình kinh doanh (BPM): Triển khai quy trình công việc như một chuỗi các bước xử lý được kết nối bằng hàng đợi tin nhắn hoặc API.

Ví dụ thực tế

Kiến trúc Pipe and Filter được sử dụng trong nhiều hệ thống thực tế trên nhiều miền khác nhau. Sau đây là một số ví dụ đáng chú ý:

- Dòng lệnh Unix/Linux:
 - Shell Pipelines: Unix và Linux shell (ví dụ: Bash) cho phép người dùng nối các lệnh lại với nhau bằng cách sử dụng các đường ống. Ví dụ: `cat file.txt | grep "pattern" | sort | uniq` xử lý một tệp thông qua một loạt lệnh để lọc, sắp xếp và xóa các mục trùng lặp.
- Trình biên dịch:
 - GCC (GNU Compiler Collection): GCC xử lý mã nguồn qua nhiều giai đoạn bao gồm tiền xử lý, phân tích cú pháp, tối ưu hóa và tạo mã. Mỗi giai đoạn là một bộ lọc chuyển đổi mã từ dạng này sang dạng khác.
- Khung xử lý dữ liệu:
 - Apache Flink và Apache Storm: Các khung này xử lý luồng dữ liệu theo thời gian thực. Mỗi thành phần trong cấu trúc xử lý (map, filter, reduce) hoạt động như một bộ lọc trong đường ống.
 - Apache NiFi: Một công cụ tích hợp dữ liệu tự động hóa luồng dữ liệu giữa các hệ thống, sử dụng bộ xử lý (bộ lọc) để chuyển đổi, định tuyến và quản lý luồng dữ liệu.
- Xử lý phương tiện:
 - GStreamer: Một khuôn khổ đa phương tiện xử lý các luồng âm thanh và video thông qua một đường ống các phân tử (bộ lọc) cho các tác vụ như giải mã, mã hóa và lọc.
- Khung phát triển web:
 - Express.js (Node.js): Middleware trong Express.js hoạt động như bộ lọc xử lý các yêu cầu và phản hồi HTTP. Ví dụ, việc ghi nhật ký, xác thực và phân tích yêu cầu được xử lý bởi các hàm middleware riêng biệt.

- ASP.NET Core Middleware: Tương tự như Express.js, ASP.NET Core sử dụng các thành phần phần mềm trung gian để xử lý các yêu cầu HTTP trong một đường ống.
- Công cụ ETL (Trích xuất, Chuyển đổi, Tải):
 - Talend: Một công cụ ETL sử dụng một loạt các thành phần để trích xuất dữ liệu từ nhiều nguồn khác nhau, chuyển đổi dữ liệu theo các quy tắc kinh doanh và tải dữ liệu vào hệ thống mục tiêu.
 - Apache Hop: Nền tảng tích hợp dữ liệu nguồn mở xử lý dữ liệu thông qua một loạt các bước chuyển đổi, cho phép thực hiện quy trình làm việc ETL phức tạp.

Các thư viện và khung phổ biến hỗ trợ Pipe và Filter

Một số thư viện và khuôn khổ phổ biến hỗ trợ kiến trúc Pipe và Filter, tạo điều kiện thuận lợi cho việc phát triển các ứng dụng có khả năng mở rộng và mô-đun. Sau đây là một số ứng dụng đáng chú ý:

- Apache NiFi:
 - Apache NiFi là một công cụ tích hợp dữ liệu nguồn mở cho phép tự động hóa luồng dữ liệu giữa các hệ thống.
 - Nó sử dụng giao diện đồ họa để thiết kế các đường ống dữ liệu bao gồm các bộ xử lý (bộ lọc) được kết nối bằng các luồng dữ liệu (đường ống).
 - Hỗ trợ thu thập, chuyển đổi, định tuyến và phân phối dữ liệu với bộ xử lý tích hợp để xử lý nhiều định dạng và giao thức dữ liệu khác nhau.
- Apache Flink:
 - Apache Flink là một nền tảng xử lý luồng mã nguồn mở hỗ trợ các ứng dụng truyền dữ liệu phân tán, thông lượng cao và độ trễ thấp.
 - Nó tổ chức logic xử lý thành các luồng dữ liệu và hoạt động, giống như kiến trúc Pipe và Filter trong đó các hoạt động đóng vai trò như bộ lọc.
 - Cung cấp hỗ trợ cho việc xử lý thời gian sự kiện, tính toán trạng thái, hoạt động tạo cửa sổ và tích hợp với nhiều nguồn dữ liệu và bộ thu thập dữ liệu khác nhau.

- Bảo Apache:
 - Apache Storm là hệ thống xử lý luồng thời gian thực có khả năng xử lý khối lượng dữ liệu lớn với độ trễ thấp.
 - Nó sử dụng kiến trúc dựa trên cấu trúc liên kết, trong đó các vòi và bu lông lần lượt đại diện cho các nguồn dữ liệu và đơn vị xử lý (bộ lọc).
 - Cung cấp khả năng chịu lỗi, khả năng mở rộng và hỗ trợ xử lý sự kiện phức tạp với ngữ nghĩa xử lý tin nhắn được đảm bảo.
- Phần mềm trung gian ASP.NET Core:
 - ASP.NET Core là một nền tảng web đa nền tảng để xây dựng các ứng dụng hiện đại dựa trên nền tảng đám mây.
 - Nó sử dụng các thành phần phần mềm trung gian có thể được cấu hình trong đường ống để xử lý các yêu cầu và phản hồi HTTP.
 - Các thành phần phần mềm trung gian hoạt động như bộ lọc để thực hiện các tác vụ như xác thực, ghi nhật ký, định tuyến và xử lý ngoại lệ trong quy trình xử lý yêu cầu.
- Phần mềm trung gian Express.js:
 - Express.js là một nền tảng web tối giản dành cho Node.js hỗ trợ phần mềm trung gian.
 - Các hàm phần mềm trung gian hoạt động như bộ lọc trong chu trình yêu cầu-phản hồi, xử lý các yêu cầu đến và phản hồi đi.
 - Cho phép các nhà phát triển phân loại và tùy chỉnh logic xử lý yêu cầu bằng cách biên soạn các hàm phần mềm trung gian trong một đường ống.

Tóm lại, kiến trúc Pipe and Filter cung cấp một cách mạnh mẽ để thiết kế hệ thống bằng cách chia nhỏ các tác vụ thành các thành phần có thể tái sử dụng (bộ lọc) được kết nối thông qua các kênh (ống). Nó thúc đẩy tính mô-đun, cho phép các nhà phát triển độc lập phát triển, thử nghiệm và bảo trì từng bộ lọc. Cách tiếp cận này tăng cường khả năng mở rộng, tính linh hoạt và khả năng bảo trì trên nhiều miền khác nhau như xử lý dữ liệu, xử lý luồng, phát triển web, v.v.

Kiến trúc ống và lọc là một mô hình thiết kế phần mềm đơn giản nhưng hiệu quả. Nó cung cấp một cách tiếp cận rõ ràng và linh hoạt để xây dựng các hệ thống xử lý dữ liệu. Tuy nhiên, như mọi kiến trúc khác, nó cũng có những ưu và nhược điểm riêng. Việc lựa chọn kiến trúc ống và lọc phụ thuộc vào yêu cầu cụ thể của từng hệ thống.

2.2. Kiến trúc hướng sự kiện

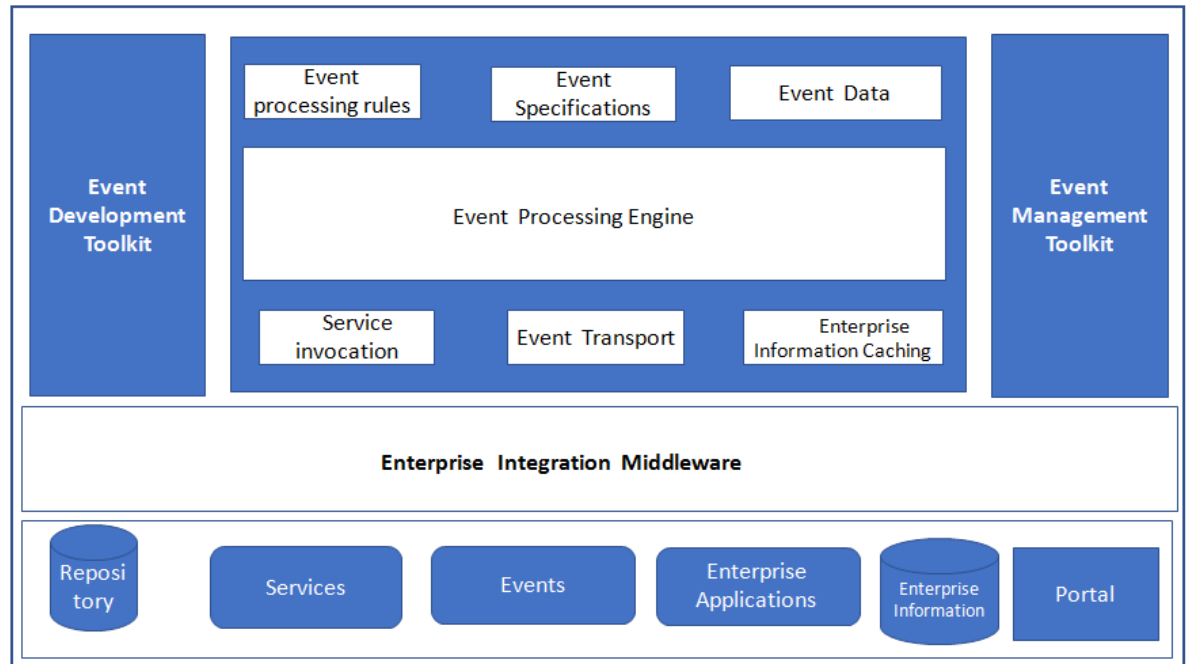
Kiến trúc hướng sự kiện (Event-Driven Architecture - EDA) là một mô hình thiết kế phần mềm trong đó các thành phần hoặc dịch vụ của hệ thống giao tiếp với nhau chủ yếu thông qua việc sản xuất và tiêu thụ các sự kiện. Một sự kiện đại diện cho một sự thay đổi về trạng thái hoặc một cập nhật.

Tại sao sử dụng kiến trúc hướng sự kiện?

- Linh hoạt: Các thành phần có thể được thêm vào hoặc loại bỏ mà không ảnh hưởng đến toàn bộ hệ thống.
- Mở rộng: Hệ thống có thể dễ dàng mở rộng để xử lý các loại sự kiện mới.
- Phân tán: Các thành phần có thể được triển khai trên nhiều máy chủ khác nhau.
- Hiệu suất: EDA có thể xử lý lượng lớn sự kiện một cách hiệu quả.

Các thành phần chính trong EDA

- Producer: Thành phần tạo ra các sự kiện.
- Event: Sự kiện được tạo ra, chứa thông tin về sự thay đổi.
- Broker: Trung gian truyền đạt các sự kiện giữa các thành phần.
- Consumer: Thành phần lắng nghe và xử lý các sự kiện.



EDA components

Các loại kiến trúc hướng sự kiện

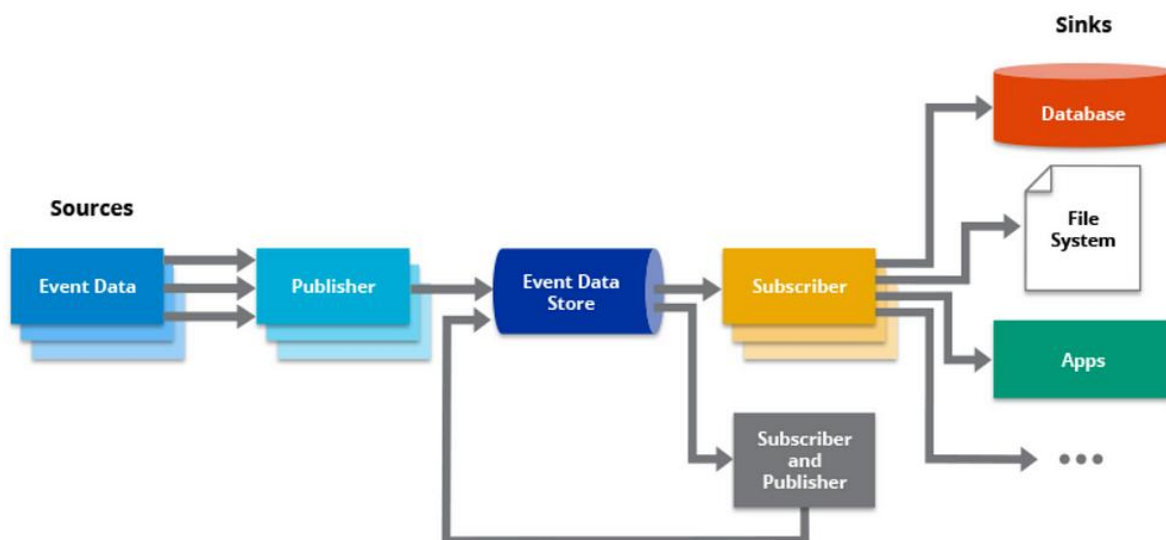
- Broker topology: Mỗi sự kiện được gửi đến một broker, broker chịu trách nhiệm phân phối sự kiện đến các consumer phù hợp.
- Mediator topology: Một mediator chịu trách nhiệm trung gian giữa các producer và consumer.

Kiến trúc hướng sự kiện cho phép các ứng dụng hành động theo các sự kiện này khi chúng xảy ra. Điều này trái ngược với kiến trúc truyền thống, chủ yếu xử lý dữ liệu dưới dạng các lô thông tin được chèn và cập nhật theo một khoảng thời gian xác định và phản hồi các yêu cầu do người dùng khởi tạo thay vì thông tin mới.

Kiến trúc hướng sự kiện tạo, phát hiện, tiêu thụ và phản ứng với các sự kiện. Các khái niệm phổ biến trong kiến trúc hướng sự kiện bao gồm publisher, subscriber, sources và sinks. Publisher là thành phần thu thập dữ liệu sự kiện và lưu trữ trong kho lưu trữ. Subscriber sau đó tiêu thụ và có khả năng phản hồi sự kiện. Sources là nơi dữ liệu bắt nguồn và sinks là đích đến nơi subscriber gửi dữ liệu.

Với kiến trúc hướng sự kiện có thể mở rộng, bạn có thể tạo và phản hồi một số lượng lớn sự kiện theo thời gian thực. Kiến trúc hướng sự kiện đặc biệt phù hợp với phần mềm

liên kết lỏng lẻo, chẳng hạn như dịch vụ vi mô. Kiến trúc hướng sự kiện hoạt động tốt với các sự kiện không thể đoán trước, phi tuyến tính, do đó chúng rất linh hoạt.



This diagram represents a common event-driven architecture design pattern, in which events feed actions in applications and repositories. The event is generated by some action, is fed through a connector or broker, then is processed, usually in a stream processing platform, and finally, a result is delivered to an application or repository.

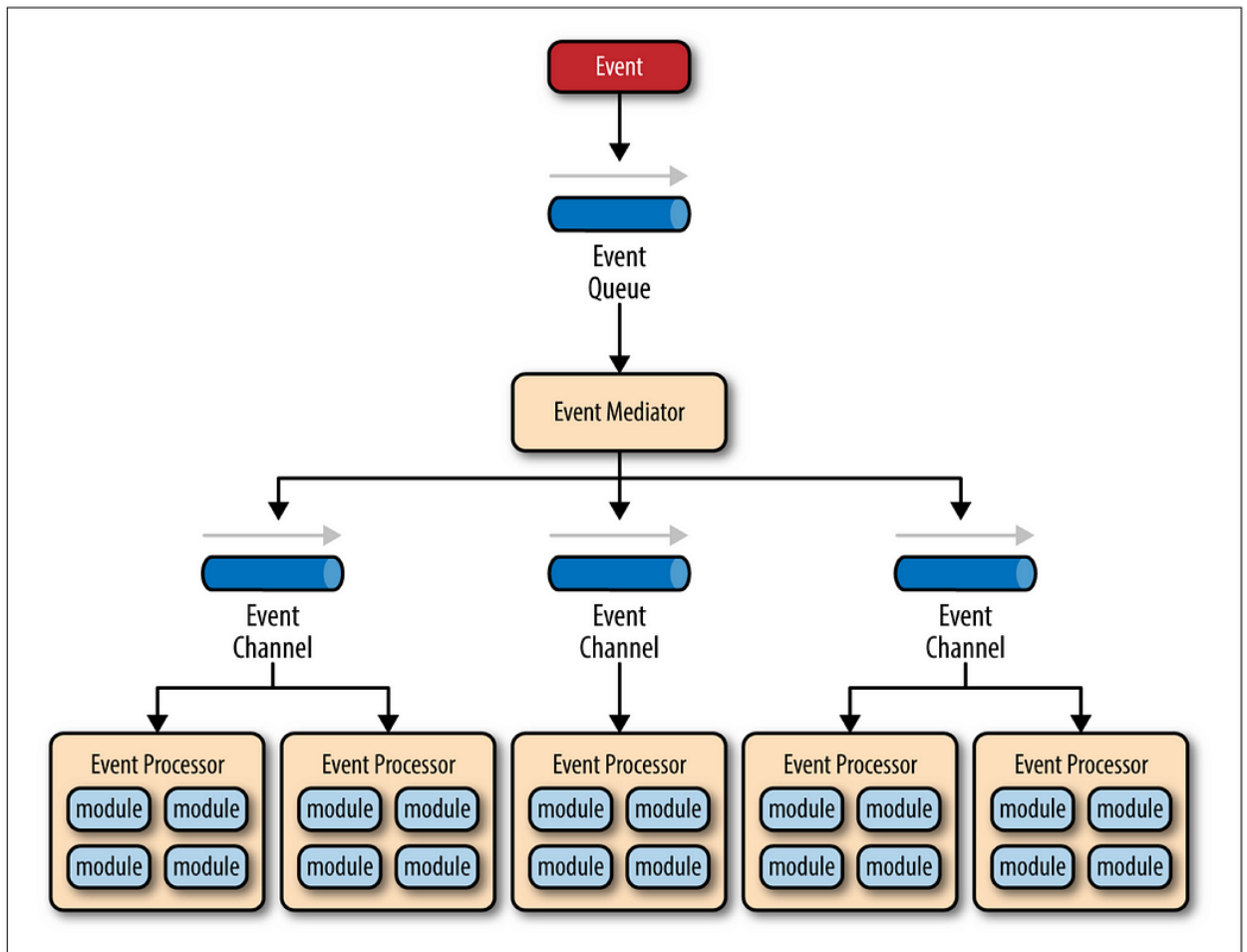
Mẫu kiến trúc hướng sự kiện bao gồm hai cấu trúc chính, trung gian và môi giới. Cấu trúc trung gian thường được sử dụng khi bạn cần sắp xếp nhiều bước trong một sự kiện thông qua một trung gian trung tâm, trong khi cấu trúc môi giới được sử dụng khi bạn muốn kết nối các sự kiện lại với nhau mà không cần sử dụng trung gian trung tâm. Vì đặc điểm kiến trúc và chiến lược triển khai khác nhau giữa hai cấu trúc này, nên điều quan trọng là phải hiểu từng cấu trúc để biết cấu trúc nào phù hợp nhất với tình huống cụ thể của bạn.

Cấu trúc trung gian:

Cấu trúc trung gian hữu ích cho các sự kiện có nhiều bước và yêu cầu một số mức độ phối hợp để xử lý sự kiện. Ví dụ, một sự kiện duy nhất để đặt giao dịch chứng khoán có thể yêu cầu bạn trước tiên phải xác thực giao dịch, sau đó kiểm tra tính tuân thủ của giao dịch chứng khoán đó so với các quy tắc tuân thủ khác nhau, chỉ định giao dịch cho một nhà môi giới, tính toán hoa hồng và cuối cùng là đặt giao dịch với nhà môi giới đó. Tất cả các

bước này sẽ yêu cầu một số mức độ phối hợp để xác định thứ tự các bước và những bước nào có thể được thực hiện tuần tự và song song.

Có bốn loại thành phần kiến trúc chính trong cấu trúc trung gian: hàng đợi sự kiện, một trung gian sự kiện, kênh sự kiện và bộ xử lý sự kiện. Luồng sự kiện bắt đầu bằng việc máy khách gửi sự kiện đến *hàng đợi sự kiện*, được sử dụng để vận chuyển sự kiện đến trung gian sự kiện. *Trung gian sự kiện* nhận sự kiện ban đầu và sắp xếp sự kiện đó bằng cách gửi các sự kiện không đồng bộ bổ sung đến *các kênh sự kiện* để thực hiện từng bước của quy trình. *Các quy trình sự kiện*, lắng nghe trên các kênh sự kiện, nhận sự kiện từ trung gian sự kiện và thực hiện logic nghiệp vụ cụ thể để xử lý sự kiện.

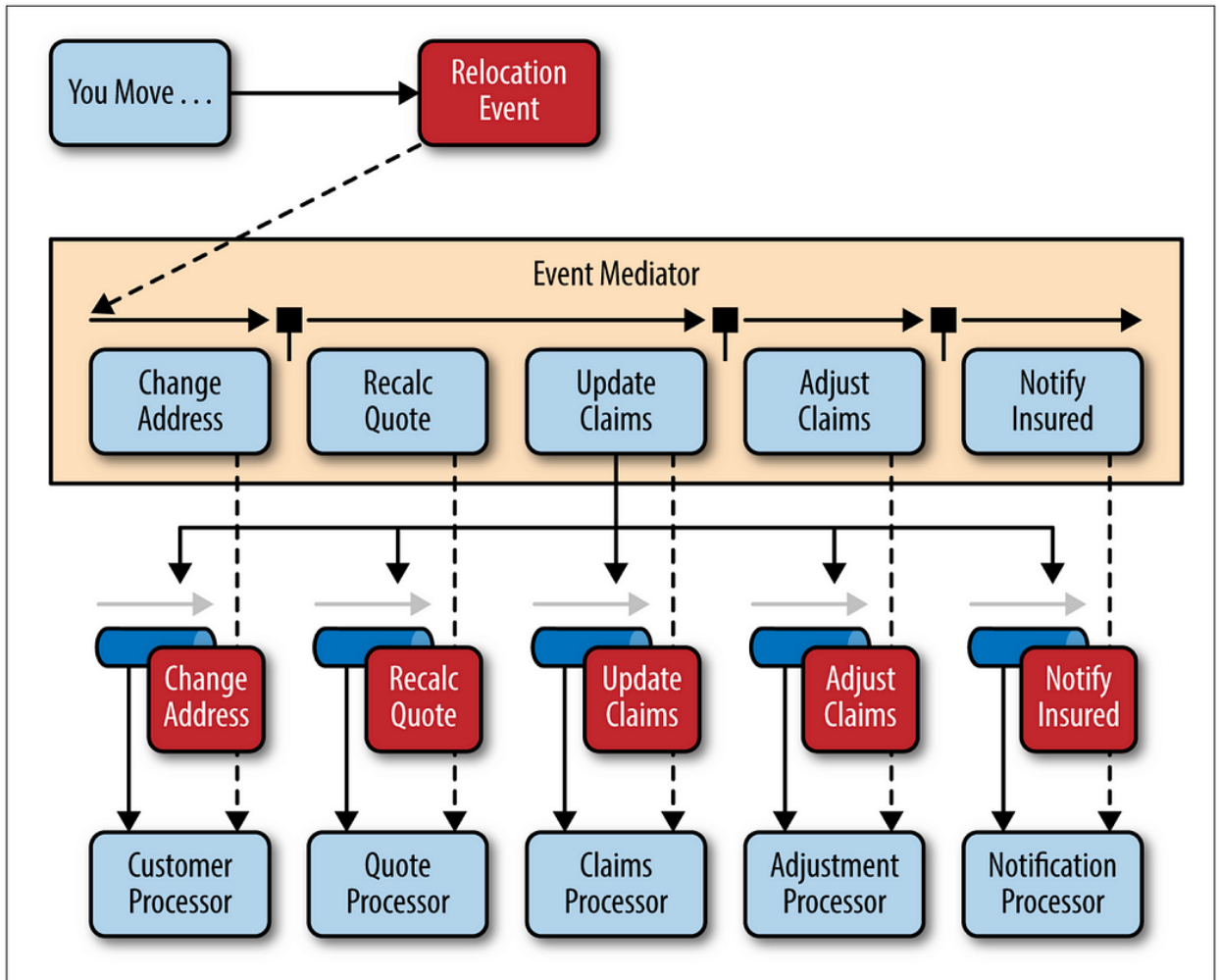


Topology môi giới

cấu trúc môi giới khác với cấu trúc trung gian ở chỗ không có trung gian sự kiện trung tâm; thay vào đó, luồng tin nhắn được phân bổ trên các thành phần bộ xử lý sự kiện

theo kiểu chuỗi thông qua một môi giới tin nhắn nhẹ (ví dụ: ActiveMQ, HornetQ, v.v.). Cấu trúc này hữu ích khi bạn có luồng xử lý sự kiện tương đối đơn giản và bạn không muốn (hoặc không cần) điều phối sự kiện trung tâm.

Có hai loại thành phần kiến trúc chính trong cấu trúc môi giới: thành phần *môi giới* và thành phần *bộ xử lý sự kiện*. Thành phần môi giới có thể tập trung hoặc liên kết và chứa tất cả các kênh sự kiện được sử dụng trong luồng sự kiện.

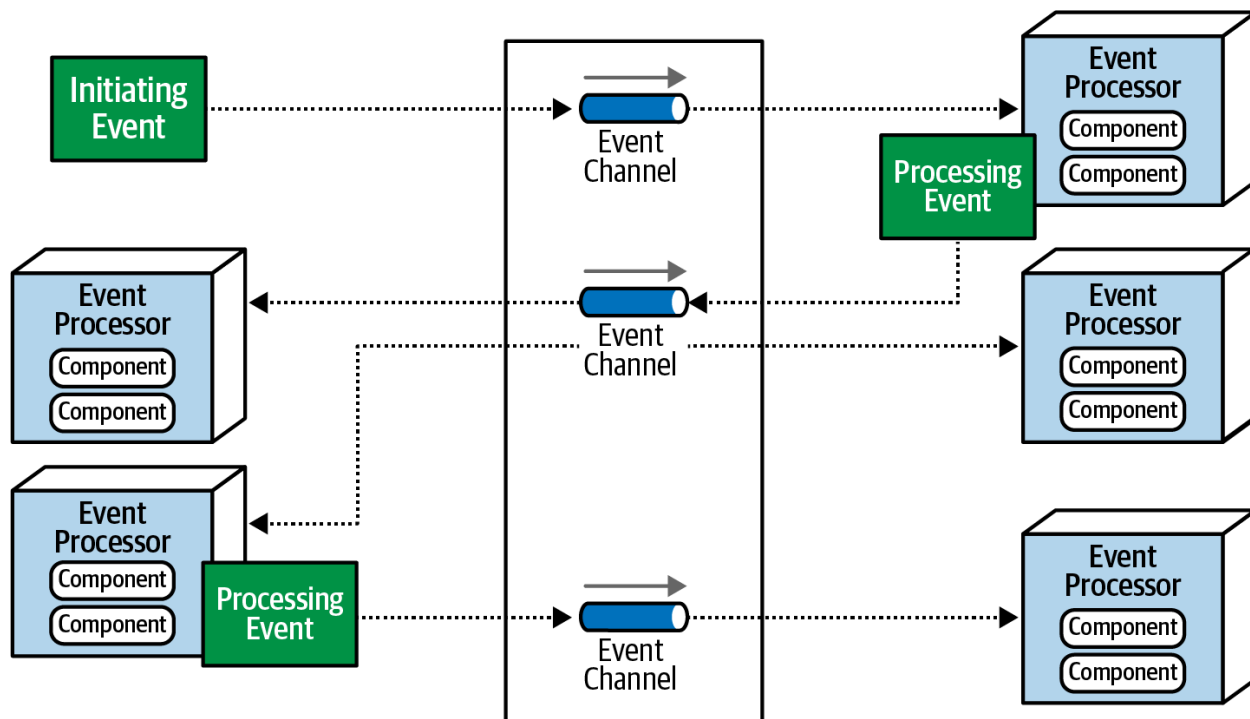


Phân loại kiến trúc hướng sự kiện

EDA được phân loại dựa trên cấu trúc liên kết (topology) giữa các component trong hệ thống với nhau, bao gồm hai mô hình phổ biến là **Broker topology** và **Mediator topology**.

2.2.1. Broker topology

Trong Broker topology, luồng tin nhắn sẽ phân phối đều tới các event processor qua message broker mà không cần tới một trung tâm điều phối event.



Broker topology - Nguồn: Fundamentals of Software Architecture - Mark Richards

2.2.1.1. Các thành phần chính của broker topology

Các thành phần chính của Broker topology bao gồm:

- Initiating event: sự kiện ban đầu bắt đầu toàn bộ luồng sự kiện
- Event channel: được sử dụng để lưu trữ các sự kiện được tạo ra và phân phối các sự kiện đó đến một service phản hồi chúng. Event channel có thể ở dạng topic hoặc queue.
- Event broker: chứa các event channel tham gia vào một luồng sự kiện.
- Event processor: service chịu trách nhiệm xử lý sự kiện
- Processing event: là sự kiện được tạo khi trạng thái của một số service thay đổi và gửi thông báo cho phần còn lại của hệ thống về sự thay đổi trạng thái đó. Event này được gửi theo cơ chế fire-and-forget broadcasting, tức là gửi và sau đó không cần xác nhận việc gửi có thành công hay không.

2.2.1.2. Luồng tin nhắn trong broker topology

Luồng tin nhắn trong broker topology hoạt động theo thứ tự sau:

- Một initiating event được gửi tới một event channel nằm trong event broker để xử lý.
- Một event processor lấy event đó về từ event channel để xử lý và thực hiện nhiệm vụ cụ thể liên quan đến việc xử lý event đó.
- Khi xử lý xong, event processor gửi một processing event tới event channel nhằm thông báo rằng event đã xử lý xong.
- Các event processor khác sẽ lắng nghe processing event này và phản ứng lại bằng cách tạo ra event processing khác. Quá trình này lặp lại cho tới khi event processor cuối xử lý xong.

Khi xử lý xong event, event processor sẽ luôn gửi tới broker một processing event để thông báo đã xử lý xong dù chúng có được tiêu thụ hay không. Nếu một event không có ai lắng nghe thì nó sẽ bị bỏ qua. Đây là thiết kế đảm bảo khả năng mở rộng hệ thống. Lý do là nếu hệ thống phát sinh nghiệp vụ mới, cần tới event này thì người dùng chỉ cần cho Event Processor lắng nghe thay vì phải sửa logic trong code.

2.2.1.3. Trường hợp nên dùng broker topology

- Cần giảm sự phụ thuộc lẫn nhau (loose coupling): Đối với các hệ thống mà các thành phần cần hoạt động độc lập, không cần biết về nhau, broker topology cung cấp khả năng tách rời cần thiết.
- Định tuyến đơn giản: các event processor giao tiếp với nhau bằng event mà không cần thông qua một trung gian quản lý nào nên phù hợp với hệ thống có định tuyến đơn giản, không cần đảm bảo đúng trình tự.

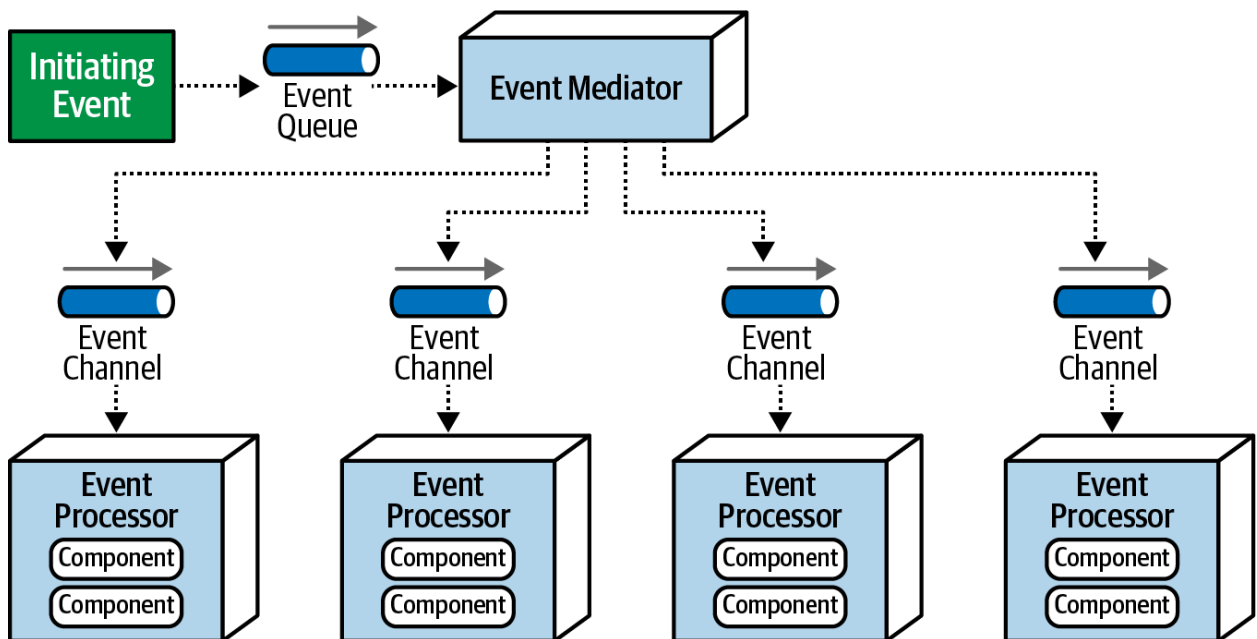
2.2.1.4. Trường hợp không nên dùng broker topology

- Xử lý logic phức tạp giữa các thành phần: Nếu hệ thống yêu cầu xử lý logic phức tạp, biến đổi dữ liệu, hoặc điều phối quy trình làm việc giữa các nhà sản xuất và người tiêu dùng, broker topology có thể không đủ linh hoạt. mediator topology thường phù hợp hơn trong trường hợp này.
- Tích hợp và chuẩn hóa dữ liệu từ nhiều nguồn: Trong trường hợp cần tích hợp dữ liệu từ nhiều nguồn khác nhau với các định dạng và giao thức không đồng nhất, Broker Topology có thể không cung cấp đủ khả năng xử lý và chuẩn hóa dữ liệu cần thiết.

- Cần kiểm soát trung tâm và quản lý workflow: Nếu hệ thống cần một mức độ kiểm soát trung tâm cao đối với luồng sự kiện và quản lý luồng, Broker topology có thể không phù hợp. Mediator Topology, với khả năng điều phối và quản lý trung tâm, có thể là lựa chọn tốt hơn trong trường hợp này.

2.2.2. Mediator topology

Trong Mediator topology có một thành phần trung gian, thường được gọi là "**mediator**", được sử dụng để điều phối và quản lý luồng sự kiện giữa các service hoặc component khác nhau trong hệ thống.

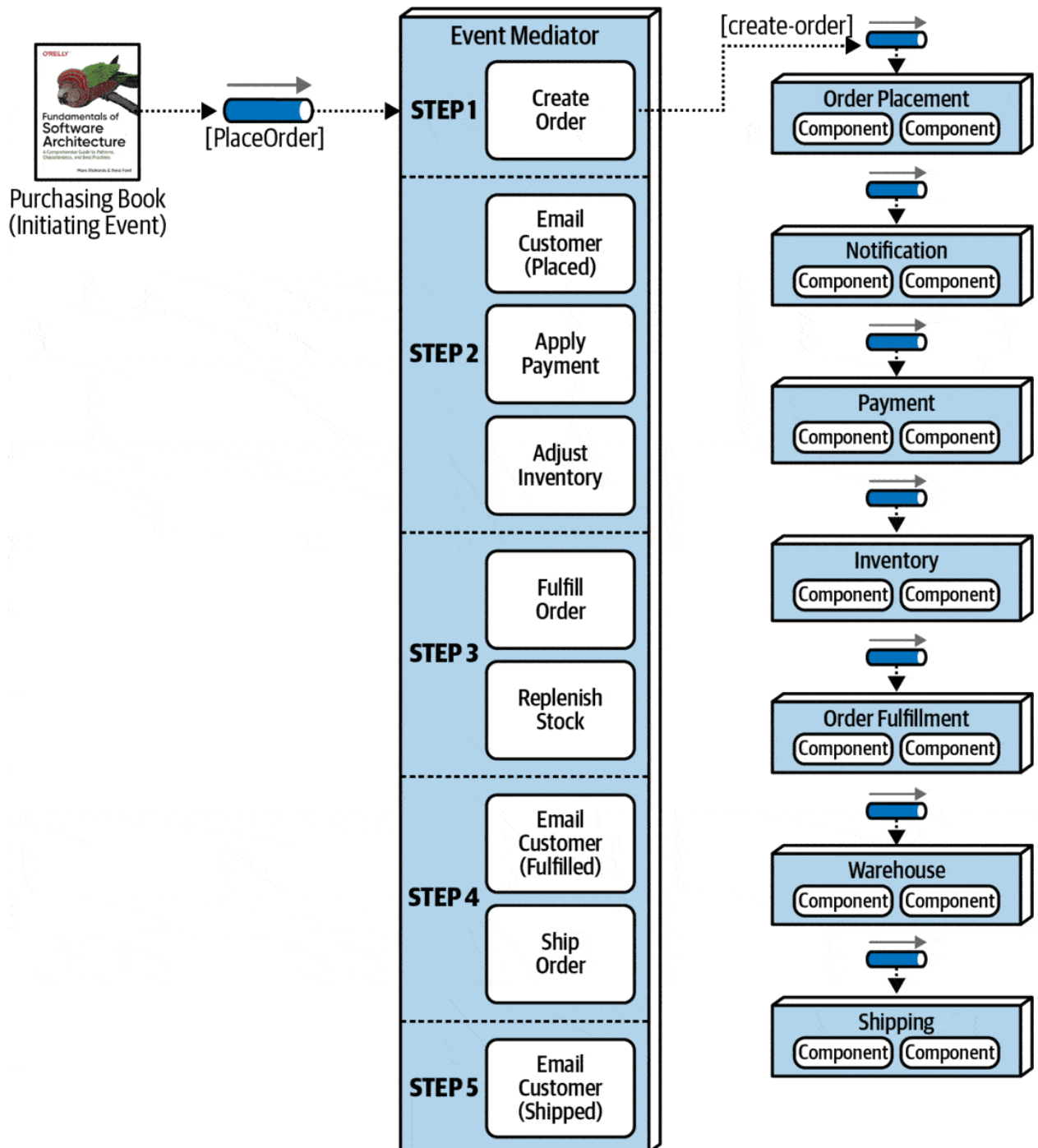


Mediator topology - Nguồn: Fundamentals of Software Architecture - Mark Richards

2.2.2.1. Các thành phần chính của Mediator topology bao gồm:

- Initiating event: sự kiện ban đầu bắt đầu toàn bộ luồng sự kiện
- Event queue: lưu trữ các sự kiện ban đầu, đảm bảo chúng không bị mất mát trong quá trình xử lý. Queue giúp xử lý các sự kiện tuần tự, đảm bảo tính nhất quán trước khi event được chuyển tới Mediator.
- Event mediator: nơi điều phối và quản lý luồng sự kiện, là trung tâm của mô hình Mediator.
- Event channel: tương tự broker topology
- Event processor: tương tự broker topology

2.2.2.2. Cách hoạt động của Mediator topology được mô tả thông qua ví dụ về hệ thống bán lẻ sau:



Ví dụ về mediator trong hệ thống bán lẻ - Nguồn: Fundamentals of Software Architecture
- Mark Richards

- **Bước 1:** Tạo đơn hàng

Sau khi nhận được event khởi tạo PlaceOrder, Mediator tạo ra event create-order, rồi gửi nó tới order-placement queue. OrderPlacement event processor sẽ nhận event này, xác thực thông tin và tạo order, trả cho Mediator thông báo đã xử lý xong kèm ID của order (acknowledgment). Tùy vào nghiệp vụ cụ thể, Mediator có thể gửi lại event cho client để thông báo đã tạo order.

- **Bước 2: Xử lý đơn hàng**

Mediator tạo và gửi 3 event đồng thời là email-customer, apply-payment và adjust-inventory tới các queue tương ứng là notification, payment và inventory. Mediator cần phải chờ cho các event processor xử lý thành công tất cả event trước khi chuyển sang bước 3.

- **Bước 3: Thực hiện đơn hàng**

Tương tự bước 2, các event fulfill-order và order-stock được gửi tới các queue tương ứng là order-fulfillment và warehouse.

- **Bước 4: Giao hàng**

Mediator thông báo trạng thái sẵn sàng giao hàng cho khách hàng bằng gửi event email-customer và ship-order tới các queue notification và shipping.

- **Bước 5: Thông báo cho khách hàng**

Thông báo cho khách hàng trạng thái đơn hàng qua email bằng event email-customer.

2.2.2.3. Trường hợp nên dùng mediator topology:

- Cần quản lý tập trung: Phù hợp với các hệ thống cần quản lý tập trung, cần sự tích hợp chặt chẽ để đảm bảo các sự kiện được điều phối và xử lý theo đúng trình tự. Nếu hệ thống yêu cầu kiểm soát chặt chẽ về quyền truy cập và bảo mật dữ liệu, mediator topology có thể cung cấp một lớp kiểm soát bổ sung.
- Cần xử lý logic phức tạp: Khi hệ thống yêu cầu xử lý logic nâng cao hoặc biến đổi sự kiện trước khi chúng được gửi đến người tiêu dùng cuối cùng. Mediator có thể thực hiện các tác vụ như lọc, biến đổi dữ liệu, hoặc ánh xạ sự kiện.

- Tích hợp hệ thống đa dạng: Khi cần tích hợp nhiều hệ thống khác nhau với các giao thức và định dạng tin nhắn không đồng nhất, mediator topology cho phép biến đổi và chuẩn hóa dữ liệu giữa các hệ thống.

2.2.2.4. Trường hợp không nên dùng mediator topology:

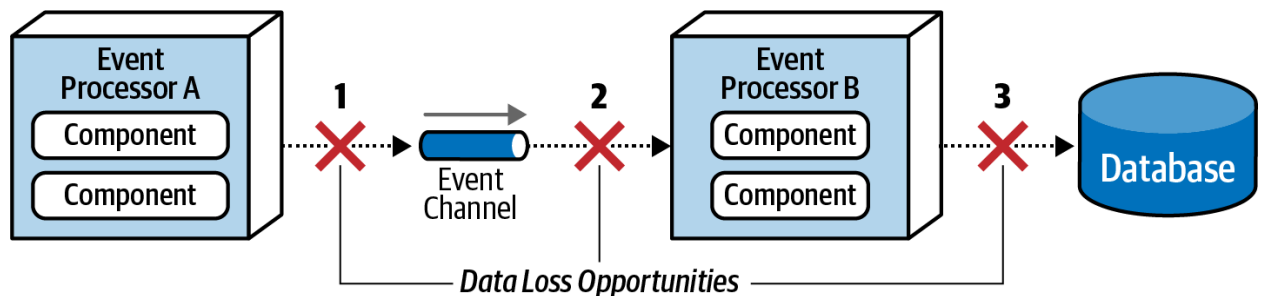
- Yêu cầu hiệu suất cao và độ trễ thấp: Mediator Topology có thể tạo ra độ trễ bổ sung do xử lý trung gian. Trong các hệ thống cần xử lý tin nhắn cực kỳ nhanh và độ trễ thấp, việc thêm một mediator có thể không phù hợp.

- Cần tính đơn giản và dễ mở rộng: Nếu mục tiêu là xây dựng một hệ thống đơn giản, dễ hiểu và dễ mở rộng, việc thêm một mediator có thể làm tăng độ phức tạp không cần thiết.

- Tách biệt và độc lập giữa các thành phần: Trong các hệ thống yêu cầu mức độ tách biệt cao giữa các thành phần, việc sử dụng mediator có thể tạo ra sự phụ thuộc trung tâm, làm giảm khả năng tách biệt và độc lập của các thành phần.

- Khả năng chịu lỗi và phục hồi: Mediator trở thành điểm trung tâm có thể gây ra sự cố. Nếu mediator gặp sự cố, toàn bộ hệ thống có thể bị ảnh hưởng. Trong các hệ thống cần khả năng chịu lỗi cao, việc phụ thuộc vào một điểm trung tâm có thể không phải là lựa chọn tốt nhất.

2.2.3. Xử lý mất dữ liệu trong kiến trúc hướng sự kiện



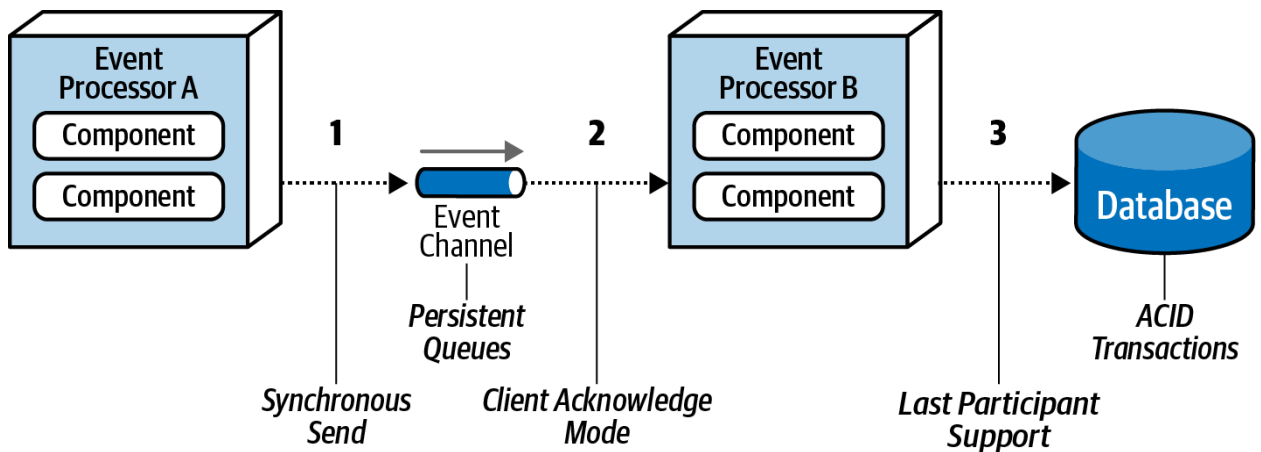
Mất dữ liệu trong EDA - Nguồn: Fundamentals of Software Architecture - Mark Richards

Một trong những vấn đề trọng tâm trong EDA là *mất dữ liệu*. Lỗi này có thể xảy ra ở các thời điểm sau:

- Event processor đẩy event vào Event channel: Broker lỗi khiến cho dữ liệu bị mất.

- Event channel đẩy event tới Event Processor: Event processor lỗi trước khi nhận được event, gây mất dữ liệu.
- Event processor lưu trữ dữ liệu đã xử lý vào database: dữ liệu bị lỗi không lưu vào database được hoặc database bị lỗi không nhận được dữ liệu.

Việc mất dữ liệu ở các thời điểm có thể giải quyết bằng các biện pháp sau:



Biện pháp xử lý mất dữ liệu trong EDA - Nguồn: Fundamentals of Software Architecture

- Mark Richards

- Synchronous send (Gửi đồng bộ): Triển khai Persisted message queues hỗ trợ tính năng guaranteed delivery. Khi nhận một event, broker sẽ lưu dữ liệu cả ở memory và ở kho chứa vật lý như filesystem hoặc database. Nhờ vậy, nếu broker bị lỗi, khi trở lại hoạt động bình thường, nó hoàn toàn có thể lấy event ở kho chứa vật lý để xử lý tiếp.
- Client acknowledge mode (Chế độ xác nhận): Khi một event được lấy ra từ queue, nó sẽ bị xoá khỏi queue. Tuy nhiên, với chế độ client acknowledge mode, event sẽ không bị xoá khỏi queue và được gắn client ID, đánh dấu nó đã được tiêu thụ, nhằm không consumer nào có thể đọc nó. Nếu Event processor A bị lỗi khi đang xử lý event X, thì X vẫn còn tồn tại trong queue, đảm bảo dữ liệu không bị mất.
- Last participant support (LPS): Việc data lỗi không lưu được vào database có thể xử lý bằng cách tận dụng ACID. Nếu có lỗi xảy ra khi transaction đã được commit thì dữ liệu sẽ được. Các event được xác nhận sau khi data được lưu thành công vào database.

2.3. Kiến trúc phân lớp

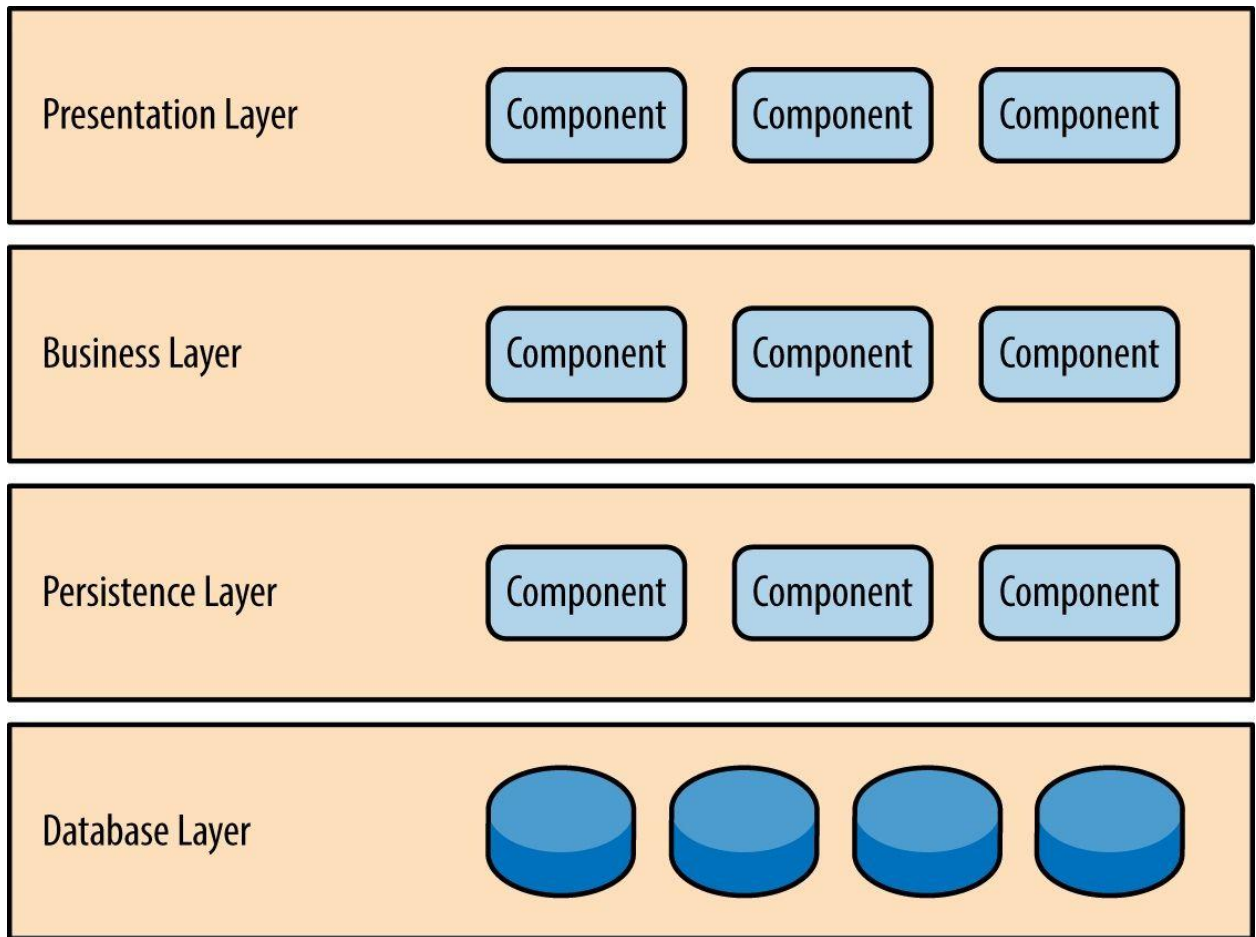
Kiến trúc lớp, còn được gọi là kiến trúc nhiều lớp, là một mẫu thiết kế phần mềm được áp dụng rộng rãi trong công nghệ phần mềm hiện đại để tạo điều kiện thuận lợi cho việc tổ chức, phân tách các mối quan tâm và khả năng bảo trì của ứng dụng. Nó đề cập đến cách thức sắp xếp và cấu trúc các thành phần phần mềm để xây dựng các ứng dụng thể hiện tính mô đun, khả năng mở rộng và tính linh hoạt, cũng như thúc đẩy việc giảm mã trùng lặp, khả năng thích ứng với các thay đổi và khả năng mở rộng cho việc triển khai các tính năng và yêu cầu mới.

Nói chung, một kiến trúc phân lớp bao gồm nhiều lớp, mỗi lớp có một bộ trách nhiệm cụ thể trong ngữ cảnh của một ứng dụng. Các lớp này được tổ chức theo hệ thống phân cấp dọc, trong đó các lớp trên phụ thuộc vào các lớp dưới về chức năng và dịch vụ; đây được gọi là mối quan hệ phụ thuộc. Mỗi lớp về cơ bản là một mức độ trừu tượng trong ứng dụng bao gồm các nhiệm vụ liên quan, thực hiện các vai trò được chỉ định của chúng đồng thời giảm thiểu sự tương tác với các lớp khác trừ khi thực sự cần thiết. Theo cách tiếp cận kiến trúc này, việc triển khai được gói gọn trong các lớp tương ứng, góp phần bảo trì dễ dàng hơn, cải thiện khả năng sử dụng lại và khả năng quản lý hệ thống phần mềm tốt hơn.

Thành phần của kiến trúc phân lớp

Các thành phần (component) trong kiến trúc phân lớp được tổ chức thành các lớp logic ngang, mỗi lớp thực hiện một vai trò cụ thể trong ứng dụng (ví dụ: logic trình bày - User interface hoặc logic nghiệp vụ - Business).

Mặc dù không có con số cụ thể về số lượng và loại lớp phải có, kiến trúc phân lớp thường bao gồm bốn lớp tiêu chuẩn: **presentation**, **business**, **persistence**, và **database**.

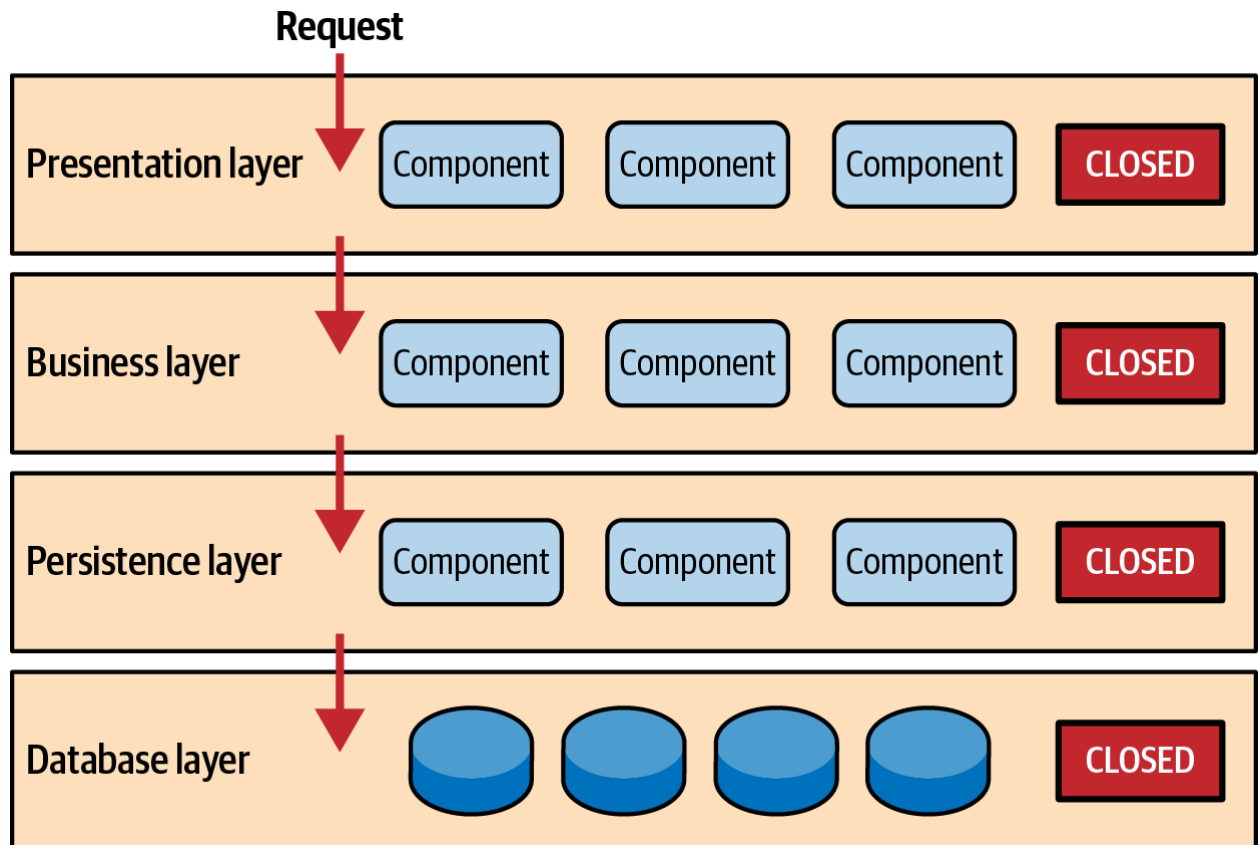


Kiến trúc phân lớp tiêu chuẩn - Nguồn: Software Architecture Patterns - Mark Richards

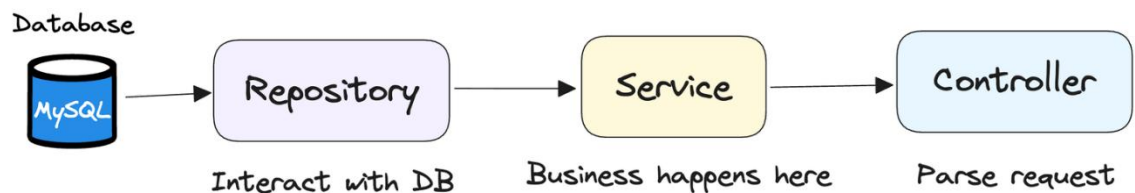
1. **Lớp Presentation:** Gồm các thành phần như giao diện người dùng, trình duyệt, ứng dụng di động, hoặc các thành phần tương tác với người dùng khác. Có chức năng hiển thị thông tin và kết quả từ lớp Logic cho người dùng và chuyển dữ liệu người dùng nhập vào lớp Logic.
2. **Lớp Business:** Đôi khi còn được gọi là lớp Service. Có nhiệm vụ xử lý các logic nghiệp vụ, nhận yêu cầu từ lớp Presentation, xử lý nó và gửi yêu cầu tương ứng đến lớp Persistence.
3. **Lớp Persistence:** Đảm nhiệm việc gửi các yêu cầu đến lớp Database để thực hiện các thao tác liên quan đến dữ liệu.
4. **Lớp Database:** Bao gồm cơ sở dữ liệu và các thành phần liên quan như hệ quản trị cơ sở dữ liệu. Có nhiệm vụ quản lý cơ sở dữ liệu, thực hiện thao tác

đọc và ghi dữ liệu và triển khai các truy vấn và lưu trữ dữ liệu theo cách được định nghĩa từ lớp Persistence.

Trong kiến trúc phân lớp, các lớp có thể ở trạng thái mở hoặc đóng. Hình minh họa ở dưới thể hiện kiến trúc phân lớp đóng. Các **lớp cô lập** (*layers of isolation*) được tạo ra nhằm đảm bảo sự độc lập giữa các lớp với nhau, thay đổi của lớp này không ảnh hưởng tới lớp khác.



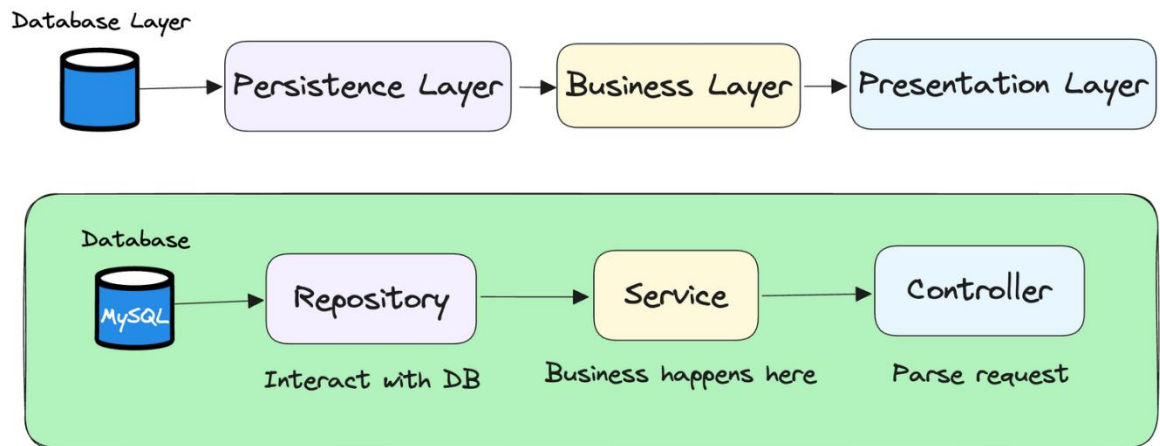
Để làm rõ luận điểm này, chúng ta hãy cùng nhìn vào mô hình API được thiết kế theo kiến trúc quốc dân được sử dụng phổ biến sau:



Kiến trúc phân lớp trong thực tế

Chúng ta có bốn lớp với các nhiệm vụ cụ thể:

1. **Controller** (Presentation Layer): Parse các request từ client
2. **Service** (Business Layer): Xử lý logic nghiệp vụ
3. **Repository** (Persistence Layer): Mapping dữ liệu từ service với database (DAO - Data access object hoặc DTO - Data transfer object)
4. **Database** (Database Layer): Cập nhật dữ liệu trong database



Kiến trúc phân lớp trong thực tế

Dù có ưu điểm tách biệt logic giữa các tầng, kiến trúc phân lớp vẫn có một yếu điểm dễ thấy là logic phải được truyền theo thứ tự các tầng, bất kể tầng đó có xử lý dữ liệu hay không. Điều này gây ra sự lãng phí tài nguyên.

Trường hợp nên dùng kiến trúc phân lớp

Kiến trúc phân lớp đã được chứng minh là linh hoạt và hiệu quả trong việc cung cấp một kế hoạch chi tiết hợp lý để phát triển các loại ứng dụng khác nhau, cho dù chúng hướng đến thiết bị di động, web hay máy tính để bàn. Ví dụ: với AppMaster, các ứng dụng phụ trợ, web và di động được tạo bằng nền tảng này được cấu trúc trong kiến trúc phân lớp, tạo điều kiện dễ dàng quản lý, bảo trì và khả năng thích ứng cho người dùng.

Nếu dự án có những hạn chế nhất định về thời gian và ngân sách, kiến trúc phân lớp là một sự lựa chọn thích hợp vì các lý do chính sau:

- Kiến trúc phân lớp thuộc loại kiến trúc monolithic nên nó không mang sự phức tạp như kiến trúc phân tán.
- Dựa trên mức độ phổ biến của kiến trúc phân lớp, hầu hết các nhà phát triển điều quen thuộc với nó, kiến việc vận hành, phát triển dự án trở nên dễ dàng hơn.
- Kiến trúc phân lớp thuộc nhóm kiến trúc phân vùng kỹ thuật nên việc sử dụng kiến trúc này phù hợp với nhiều tổ chức khác nhau do các nhóm được chia thành nhóm chuyên trách kỹ thuật như nhóm FE, nhóm BE, nhóm database, ...

4. Trường hợp không nên dùng kiến trúc phân lớp

- Nếu dự án của bạn đòi hỏi khả năng mở rộng, độ đàn hồi, khả năng chịu lỗi và hiệu năng thì kiến trúc phân lớp không phải là sự lựa chọn phù hợp. Nguyên nhân chủ yếu do bản chất của loại kiến trúc monolithic là khó mở rộng và khả năng chịu lỗi thấp. Một component trong hệ thống bị lỗi có thể dẫn tới lỗi cả ứng dụng.
- Nếu dự án của bạn có nhiều sự thay đổi về domain. Ví dụ, ứng dụng thương mại điện tử cần thêm tính năng livestream. Để hoàn thành yêu cầu này, bạn cần sửa nhiều lớp khác nhau như sửa schema của database (lớp database) gồm thêm bảng mới hoặc cột mới hoặc cả hai, sửa SQL trong lớp persistence, sửa logic của lớp business và sửa cả lớp presentation (giả sử cần thêm API mới). Việc thay đổi này có thể kém linh hoạt và mất nhiều thời gian vì scope phải sửa nhiều.
- Nếu team của bạn tổ chức thành các nhóm đa chức năng xử lý chung domain, nghĩa là trong cùng một team sẽ có nhiều thành viên có chuyên môn trên nhiều mảng khác nhau như FE, BE, designer, DA, ... thì việc này không hề phù hợp với bản chất của kiến trúc phân lớp (do thuộc phân vùng kỹ thuật).

Ưu điểm của việc áp dụng kiến trúc phân lớp trong bối cảnh phát triển phần mềm bao gồm:

- Tính mô-đun: Việc tách biệt các mối quan tâm cho phép các nhà phát triển tập trung vào các khía cạnh cụ thể của ứng dụng, làm cho cơ sở mã trở nên dễ hiểu, có tổ chức và dễ quản lý hơn. Tính mô-đun này cũng thúc đẩy việc tái sử dụng các thành phần trên các ứng dụng khác nhau.

- Khả năng mở rộng: Kiến trúc phân lớp vốn đã hỗ trợ khả năng mở rộng, vì mỗi lớp có thể được mở rộng và tối ưu hóa độc lập theo các yêu cầu thay đổi và mức tăng hiệu suất được nhắm mục tiêu. Tính linh hoạt này đặc biệt có lợi trong các ứng dụng tải cao hoặc cấp doanh nghiệp, chẳng hạn như các ứng dụng được tạo bởi AppMaster.

- Khả năng bảo trì: Nhờ cách ly các khu vực chức năng và cách triển khai tương ứng của chúng trong một ứng dụng, kiến trúc phân lớp thúc đẩy khả năng bảo trì tốt hơn và cung cấp các sửa đổi hoặc cập nhật dễ dàng hơn cho các thành phần mà không ảnh hưởng đáng kể đến toàn bộ hệ thống.

- Khả năng kiểm tra: Mỗi lớp có thể được kiểm tra độc lập, đảm bảo rằng các thành phần riêng lẻ và ứng dụng tổng thể đều mạnh mẽ và đáng tin cậy. Khía cạnh khả năng kiểm tra này được nhấn mạnh khi AppMaster tập trung vào việc tạo ra các ứng dụng không có nợ kỹ thuật, vì những thay đổi về yêu cầu được giải quyết bằng cách tạo lại toàn bộ ứng dụng từ đầu.

- Khả năng tương tác: Kiến trúc phân lớp tạo điều kiện thuận lợi cho việc tích hợp và liên lạc giữa các hệ thống và dịch vụ khác nhau. Ví dụ: AppMaster tạo tài liệu Swagger và API RESTful cho các ứng dụng của mình, cho phép tương tác liền mạch với các hệ thống và dịch vụ khác.

Kiến trúc phân lớp (Layered architecture) chia hệ thống ra thành các lớp thực hiện chức năng riêng biệt. Mỗi lớp sẽ tương tác với lớp ở trên hoặc ở dưới nó.

Ưu điểm của kiến trúc phân lớp là các layer độc lập, đảm bảo logic không bị chồng chéo lẫn nhau. Tuy nhiên, vì phải đi theo thứ tự từng lớp một nên có thể dẫn tới hiệu năng không tốt do dữ liệu bị truyền qua những lớp không cần thiết.

Kiến trúc phân lớp sẽ phù hợp với những hệ thống đơn giản chưa cần nhiều khả năng mở rộng, khả năng chịu lỗi và hiệu năng. Ngoài ra, những dự án bị giới hạn bởi thời gian và ngân sách sẽ phù hợp với kiến trúc đơn giản này.

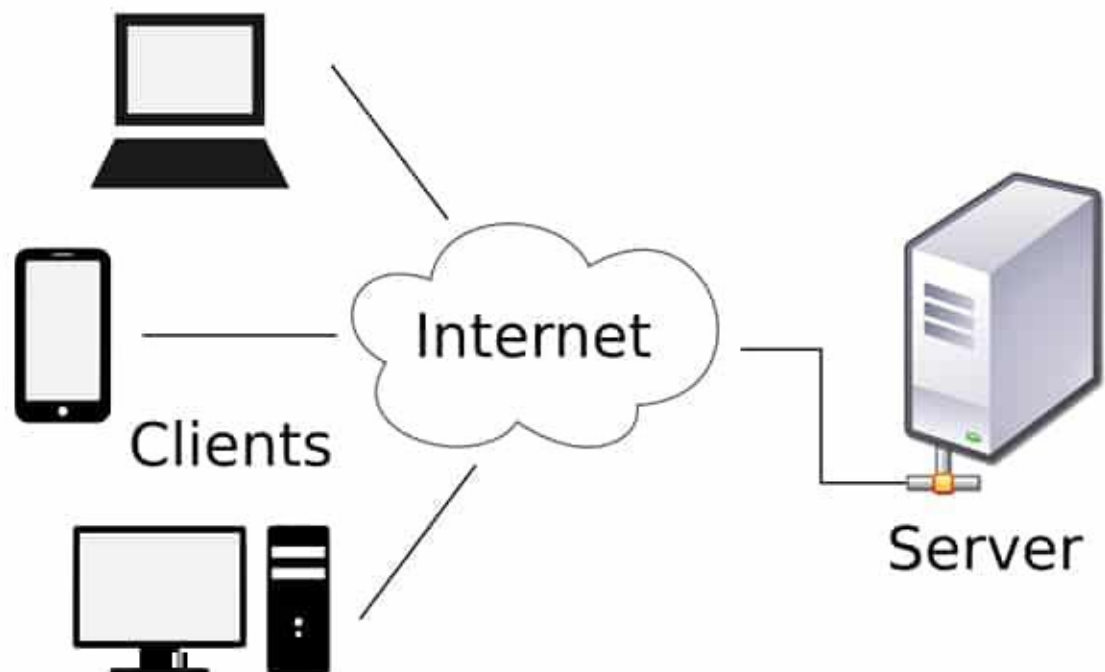
2.4. Kiến trúc gọi và trả lời

Kiến trúc gọi và trả lời (Client-Server) là một mô hình thiết kế phần mềm trong đó Client (máy khách) gửi yêu cầu đến Server (máy chủ), máy chủ sẽ xử lý yêu cầu và trả về kết quả cho máy khách.

Mô hình này thường được áp dụng trong hệ thống mạng, ứng dụng web, ứng dụng doanh nghiệp và cơ sở dữ liệu. Ý tưởng của mô hình này như sau: Client (máy khách) sẽ gửi yêu cầu (request) đến Server (máy chủ, cung ứng các dịch vụ cho các máy khách). Server sẽ xử lý dữ liệu và gửi kết quả về cho Client.

Ứng dụng thực tế của Kiến trúc Client-Server

- Web Application (Ứng dụng Web): Gmail, Facebook, YouTube.
- Hệ thống quản lý doanh nghiệp (ERP, CRM).
- Ứng dụng ngân hàng trực tuyến, ví điện tử.
- Trò chơi trực tuyến (Online Games, Game Server).
- Dịch vụ Streaming (Spotify).



Client (Máy khách) là thiết bị hoặc ứng dụng mà người dùng sử dụng để gửi yêu cầu đến máy chủ. Vai trò của client là khởi tạo các truy vấn, hiển thị thông tin và tương tác

với người dùng cuối. Các ví dụ điển hình của client bao gồm trình duyệt web như Chrome, Firefox; các ứng dụng di động trên Android hoặc iOS; phần mềm máy tính như ứng dụng email hoặc phần mềm quản lý dữ liệu.

Server (Máy chủ) là hệ thống có nhiệm vụ xử lý các yêu cầu được gửi từ client. Server sẽ thực hiện các tác vụ như xử lý logic nghiệp vụ, truy xuất cơ sở dữ liệu, phân tích dữ liệu, và gửi kết quả phản hồi lại cho client. Một số loại server phổ biến trong kiến trúc này bao gồm: Web Server (ví dụ: Apache, Nginx), Database Server (ví dụ: MySQL, PostgreSQL), và Application Server (ví dụ: Tomcat, Node.js).

Network (Mạng kết nối) là thành phần trung gian giúp kết nối giữa client và server. Đây có thể là mạng cục bộ (LAN), mạng diện rộng (WAN), hoặc mạng Internet toàn cầu. Việc truyền tải dữ liệu giữa client và server được thực hiện thông qua các giao thức truyền thông mạng như HTTP, HTTPS, hoặc TCP/IP, đảm bảo dữ liệu được gửi đi và nhận lại một cách đáng tin cậy và bảo mật.

Mô hình Client-Server là kiến trúc phổ biến trong nhiều hệ thống ứng dụng hiện nay, đặc biệt trong các hệ thống web và dịch vụ mạng, nhờ khả năng phân chia rõ ràng vai trò, tối ưu hóa tài nguyên, và hỗ trợ mở rộng linh hoạt.

Client/Server là mô hình tổng quát nhất, trên thực tế thì một server có thể được nối tới nhiều server khác nhằm làm việc hiệu quả và nhanh hơn. Khi nhận được 1 yêu cầu từ client, server này có thể gửi tiếp yêu cầu vừa nhận được cho server khác ví dụ như database server vì bản thân nó không thể xử lý yêu cầu này được. Máy server có thể thi hành các nhiệm vụ đơn giản hoặc phức tạp.

Ví dụ :

Mail Server : Ở bên phía Client, người dùng soạn thảo Email và sẽ gửi đến Mail Server, phía bên Mail Server sẽ tiếp nhận và lưu trữ, tìm kiếm địa chỉ của mail được gửi đến và gửi đi.

Web Server : Lưu trữ các trang Web. khi người dùng ở phía máy Client nhập địa chỉ của trang web, Client sẽ gửi yêu cầu đến máy chủ Web và máy chủ Web sẽ gửi toàn bộ nội dung của trang web về cho phía Client.

Cách hoạt động của Kiến trúc Client-Server

Cách thức hoạt động của kiến trúc Client-Server được tổ chức theo một chuỗi các bước tương tác giữa máy khách và máy chủ thông qua mạng kết nối. Quá trình này đảm bảo rằng dữ liệu và dịch vụ được cung cấp một cách hiệu quả, nhất quán và có thể mở rộng trong môi trường phân tán.

Cụ thể, quy trình hoạt động của mô hình Client-Server diễn ra như sau:

Bước 1: Client (máy khách) khởi tạo một yêu cầu và gửi nó đến Server (máy chủ) thông qua hệ thống mạng, sử dụng các giao thức truyền thông như HTTP, HTTPS, hoặc TCP/IP. Yêu cầu này có thể là truy vấn dữ liệu, gửi thông tin người dùng, hoặc thực hiện một tác vụ cụ thể.

Bước 2: Server nhận yêu cầu từ client, sau đó tiến hành xử lý dựa trên logic nghiệp vụ đã được lập trình sẵn. Các hoạt động xử lý có thể bao gồm truy xuất dữ liệu từ cơ sở dữ liệu, thực hiện các phép tính, xác thực thông tin người dùng, hoặc tương tác với các hệ thống khác.

Bước 3: Sau khi xử lý xong, server tạo ra một phản hồi phù hợp (chẳng hạn như dữ liệu được tìm thấy, thông báo trạng thái, hoặc kết quả tính toán) và gửi lại cho client thông qua cùng một kênh mạng.

Bước 4: Client nhận phản hồi từ server và thực hiện các hành động tiếp theo như hiển thị dữ liệu cho người dùng, lưu trữ thông tin, hoặc gửi tiếp các yêu cầu khác tùy theo ngữ cảnh của ứng dụng.

Cơ chế hoạt động theo từng bước như trên giúp phân tách rõ ràng trách nhiệm giữa client và server, đồng thời đảm bảo rằng hệ thống có thể vận hành ổn định và dễ dàng mở rộng khi cần thiết. Mô hình này là nền tảng cho nhiều ứng dụng hiện đại như hệ thống web, dịch vụ đám mây và các ứng dụng phân tán.

Ví dụ:

- **Truy cập website:**
 - Người dùng nhập URL vào trình duyệt (Client).
 - Trình duyệt gửi yêu cầu HTTP đến Web Server.
 - Web Server lấy dữ liệu từ Database và trả về trang web cho trình duyệt.
- **Ứng dụng ngân hàng online:**

- Người dùng nhập thông tin tài khoản và gửi yêu cầu đăng nhập.
- Server kiểm tra thông tin và trả về kết quả đăng nhập thành công hay thất bại.

Ưu điểm và nhược điểm của kiến trúc Client-Server

Kiến trúc Client-Server là một mô hình phổ biến trong phát triển hệ thống phần mềm nhờ khả năng tổ chức linh hoạt và phân chia rõ ràng giữa các thành phần. Tuy nhiên, mô hình này cũng tồn tại những hạn chế nhất định bên cạnh những lợi ích nổi bật.

Ưu điểm:

Một trong những lợi thế nổi bật của mô hình Client-Server là khả năng truy cập tài nguyên dữ liệu từ xa một cách an toàn. Thông qua các giao thức bảo mật như HTTPS hoặc các cơ chế xác thực, máy khách có thể truy cập dữ liệu từ máy chủ mà không cần phải trực tiếp lưu trữ hay xử lý dữ liệu tại phía người dùng, từ đó hỗ trợ hiệu quả cho việc chia sẻ thông tin giữa các vị trí địa lý khác nhau.

Ngoài ra, kiến trúc này không phụ thuộc vào một công nghệ lập trình cụ thể, do đó các nhà phát triển có thể sử dụng bất kỳ ngôn ngữ lập trình nào phù hợp với yêu cầu của từng phía (client hoặc server), miễn là chúng hỗ trợ giao tiếp qua giao thức chung. Việc phân chia rõ ràng vai trò giữa máy khách và máy chủ cũng giúp quá trình bảo trì và nâng cấp hệ thống trở nên đơn giản hơn. Các lỗi hoặc thay đổi tại phía server không nhất thiết ảnh hưởng đến toàn bộ hệ thống client, và ngược lại, điều này giúp tiết kiệm thời gian và nguồn lực trong quản lý hệ thống.

Nhược điểm:

Tuy nhiên, một trong những hạn chế lớn nhất của mô hình Client-Server là vấn đề khả năng chịu tải của máy chủ. Khi có quá nhiều máy khách gửi yêu cầu đồng thời, server có thể bị quá tải, dẫn đến hiện tượng chậm phản hồi hoặc thậm chí gián đoạn dịch vụ. Do đó, việc xây dựng một kiến trúc hạ tầng đủ mạnh và có khả năng mở rộng là điều bắt buộc trong các hệ thống lớn.

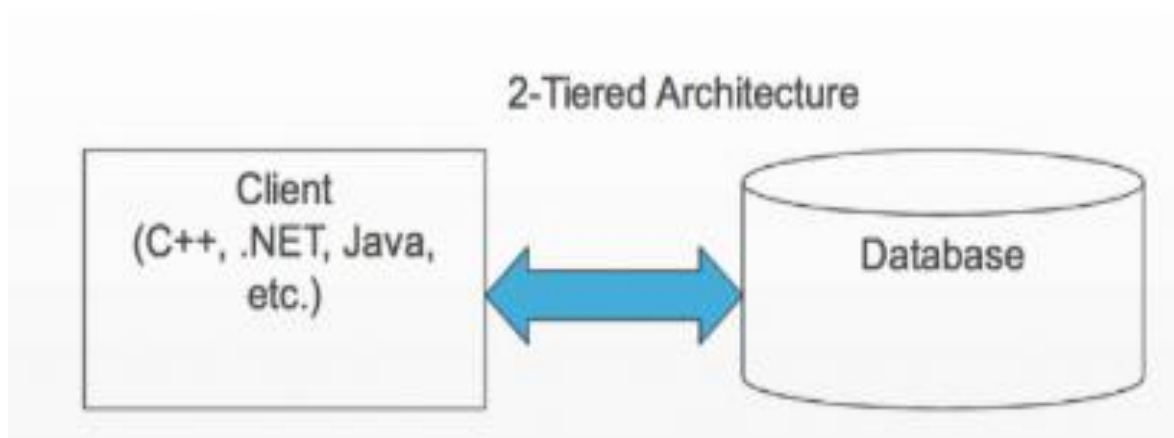
Ngoài ra, nếu kênh truyền thông giữa client và server không được đảm bảo an toàn, dữ liệu trao đổi qua mạng có thể bị rò rỉ hoặc tấn công bởi các đối tượng xấu, đặc biệt trong các hệ thống không áp dụng đầy đủ các biện pháp mã hóa, xác thực và kiểm soát truy cập.

Các mô hình Client-Server phổ biến

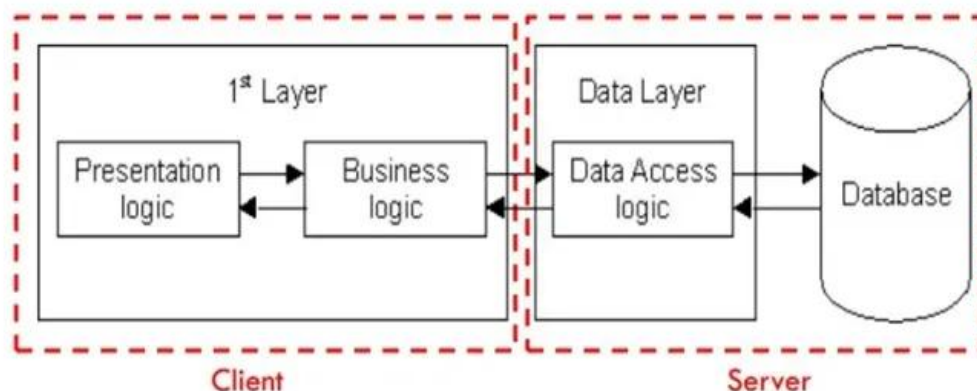
- Mô hình 2 lớp (Two-tier architecture)
- Mô hình 3 lớp (Three-tier architecture)
- Mô hình N lớp (N-tier architecture)

Mô hình 2 lớp (Two-Tier Architecture)

- là một dạng của mô hình Client-Server, trong đó hệ thống được chia thành hai thành phần chính:
 - Lớp Client (Tầng giao diện - Presentation Layer)
 - Lớp Server (Tầng dữ liệu - Data Layer)
- Trong mô hình này, Client giao tiếp trực tiếp với Server mà không có tầng trung gian (Middleware), thường được sử dụng trong các hệ thống nhỏ hoặc ứng dụng nội bộ



2-Tier Architecture



- **Lớp 1 - Client (Tầng giao diện - Presentation Layer)**
 - Chạy trên máy người dùng (PC, laptop, smartphone).
 - Chứa giao diện người dùng (UI) để nhập và hiển thị dữ liệu.
 - Gửi yêu cầu truy vấn dữ liệu đến Server.
- **Lớp 2 - Server (Tầng dữ liệu - Data Layer)**
 - Chứa cơ sở dữ liệu và xử lý truy vấn dữ liệu.
 - Nhận yêu cầu từ Client, xử lý, và trả kết quả.
 - Thường chạy trên một máy chủ trung tâm (Database Server).
- **Ví dụ:** Ứng dụng nhắn tin văn bản (SMS).

Khi bạn gửi tin nhắn, ứng dụng trên điện thoại (**client**) sẽ gửi yêu cầu đến máy chủ của nhà mạng (**server**), sau đó máy chủ sẽ chuyển tin nhắn đến người nhận.

Một ví dụ khác là các ứng dụng **quản lý cơ sở dữ liệu**, trong đó **client** kết nối đến **database server** để truy vấn và quản lý dữ liệu.

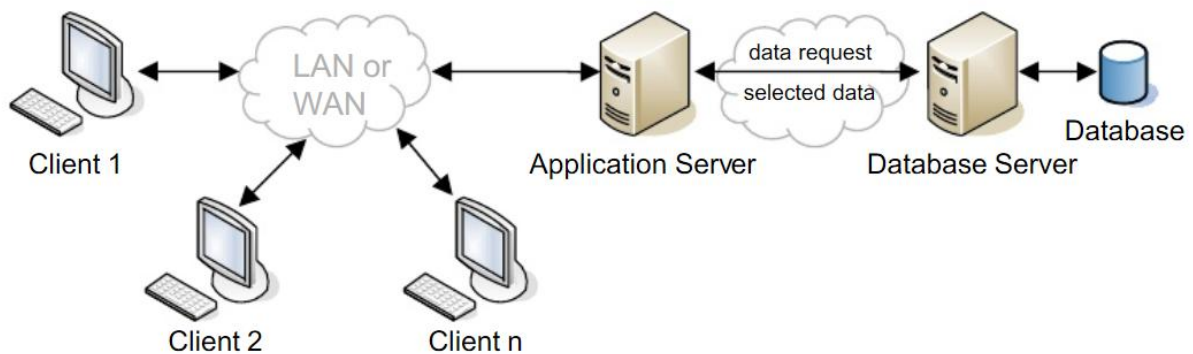
- **Ứng dụng quản lý thư viện**
 - Người dùng nhập tên sách vào phần mềm trên máy tính (Client).
 - Client gửi truy vấn SQL đến Database Server.
 - Server tìm kiếm sách và gửi kết quả về Client.
 - Client hiển thị danh sách sách phù hợp.
- **Ứng dụng kế toán nội bộ**
 - Nhân viên kế toán nhập dữ liệu hóa đơn vào ứng dụng.
 - Client gửi dữ liệu đến Database Server để lưu trữ.
 - Khi cần tra cứu, Client gửi truy vấn đến Server để lấy dữ liệu.
- **Ưu và nhược điểm của kiến trúc 2-Tier**

Kiến trúc 2-Tier phù hợp với các ứng dụng nhỏ gọn, không yêu cầu xử lý phức tạp hoặc bảo mật cao. Với các hệ thống lớn hơn và phức tạp, mô hình ba tầng (3-Tier) hoặc n tầng (N-Tier) sẽ tối ưu hơn để đảm bảo tính linh hoạt và khả năng mở rộng.

Ưu điểm	Nhược điểm
Giao tiếp nhanh và giảm thiểu độ trễ do kết nối trực tiếp giữa Client và Server.	Khả năng mở rộng hạn chế, khó đáp ứng nếu số lượng người dùng tăng nhanh.
Chi phí phát triển và bảo trì thấp do hệ thống ít tầng và đơn giản.	Dễ bị giới hạn trong giao diện người dùng, khó triển khai tính năng phức tạp.
Dễ quản lý và kiểm soát, phù hợp với các ứng dụng đơn giản.	Thiếu các tính năng bảo mật nâng cao, dễ bị tấn công khi ứng dụng phát triển.

Mô hình 3 lớp (Three-Tier Architecture) là một mô hình Client-Server nâng cao, trong đó hệ thống được chia thành ba tầng riêng biệt:

Three-Tier Client-Server Architecture



- **Mô hình 3 lớp** (Three-Tier Architecture) là một mô hình Client-Server nâng cao, trong đó hệ thống được chia thành ba tầng riêng biệt:
 - Lớp giao diện (Presentation Layer - Client)
 - Lớp xử lý nghiệp vụ (Business Logic Layer - Application Server)
 - Lớp dữ liệu (Data Layer - Database Server)
- **Mô hình 3 lớp** (Three-Tier Architecture)

- Điểm khác biệt so với kiến trúc 2 lớp:
 - Trong mô hình 2 lớp, Client giao tiếp trực tiếp với Database Server.
 - Trong mô hình 3 lớp, có thêm tầng trung gian Business Logic Layer giúp xử lý nghiệp vụ và bảo vệ dữ liệu.
- **Lớp 1 - Presentation Layer (Giao diện - Client)**
 - Là phần giao diện người dùng, chạy trên máy tính, điện thoại, trình duyệt web.
 - Hiển thị dữ liệu, nhận thao tác từ người dùng và gửi yêu cầu đến Application Server.
- **Lớp 2 - Business Logic Layer (Xử lý nghiệp vụ - Application Server)**
 - Là tầng trung gian chịu trách nhiệm xử lý nghiệp vụ.
 - Nhận yêu cầu từ Client, xử lý logic, truy xuất dữ liệu từ Database Server.
 - Bảo mật tốt hơn vì Client không trực tiếp truy cập Database.
- **Lớp 3 - Data Layer (Lưu trữ dữ liệu - Database Server)**
 - Chứa cơ sở dữ liệu, thực hiện truy vấn SQL, lưu trữ và cung cấp dữ liệu.
 - Chỉ nhận yêu cầu từ Application Server, không cho phép Client truy cập trực tiếp.

Cách thức hoạt động của mô hình 3 lớp

- Client gửi yêu cầu đến Application Server.
- Application Server xử lý nghiệp vụ và gửi truy vấn đến Database Server.
- Database Server tìm dữ liệu, gửi kết quả về Application Server.
- Application Server xử lý kết quả, trả dữ liệu về Client.
- Client hiển thị dữ liệu lên giao diện người dùng.

Ví dụ thực tế về Kiến trúc 3 lớp

- **Ứng dụng thương mại điện tử (Website bán hàng)**
 - Người dùng đặt hàng trên trang web (Client).
 - Application Server kiểm tra giỏ hàng, tính toán giá tiền, xác thực thanh toán.
 - Database Server lưu đơn hàng vào hệ thống.
 - Kết quả được gửi lại cho người dùng.

- **Ứng dụng ngân hàng trực tuyến**
 - Người dùng đăng nhập vào ứng dụng ngân hàng.
 - Application Server kiểm tra thông tin đăng nhập, xác thực người dùng.
 - Database Server truy vấn số dư tài khoản và gửi dữ liệu về Application Server.
 - Ứng dụng hiển thị số dư cho người dùng.
- **Ưu điểm:**
 - Bảo mật cao hơn: Client không trực tiếp truy cập Database.
 - Dễ mở rộng: Có thể nâng cấp từng lớp mà không ảnh hưởng đến toàn bộ hệ thống.
 - Dễ bảo trì: Code được chia thành từng phần riêng biệt, dễ sửa chữa và nâng cấp.
 - Hiệu suất tốt hơn: Xử lý nghiệp vụ tách biệt giúp phân tán tải tốt hơn.
- **Nhược điểm:**
 - Độ phức tạp cao hơn so với kiến trúc 2 lớp.
 - Cần thêm tài nguyên để vận hành Application Server.
 - Hiệu suất thấp hơn nếu không tối ưu tốt, vì có thêm một lớp trung gian.

Khi nào nên sử dụng Kiến trúc 3 lớp?

- Khi hệ thống có quy mô lớn, cần mở rộng dễ dàng.
- Khi bảo mật là yếu tố quan trọng, ví dụ như hệ thống ngân hàng, thương mại điện tử.
- Khi cần tách biệt xử lý nghiệp vụ, giảm tải cho Database Server.
- Khi phát triển ứng dụng web hoặc ứng dụng di động cần backend mạnh.

So sánh giữa mô hình 2 lớp và mô hình 3 lớp

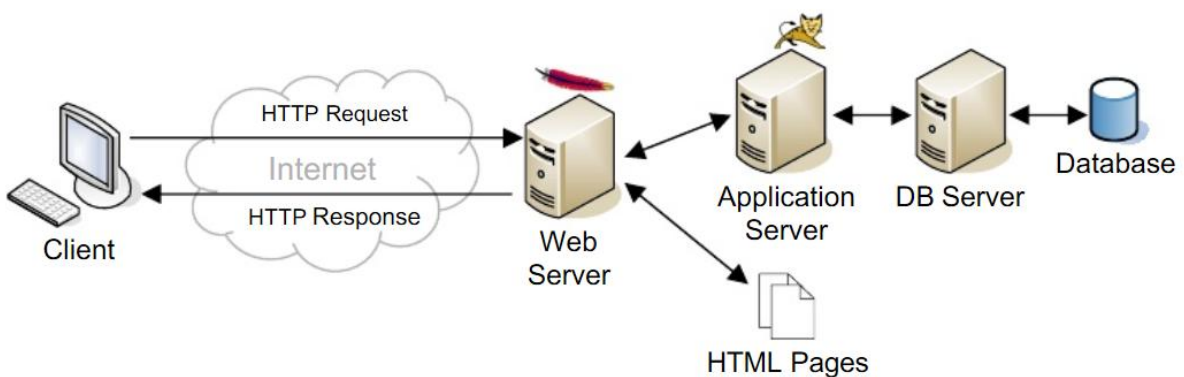
Tiêu chí	Kiến trúc 2 lớp	Kiến trúc 3 lớp
Số tầng	2 (Client - Server)	3 (Client - Application - Database)
Tầng trung gian	Không có	Có tầng Business Logic (Server xử lý nghiệp vụ)
Bảo mật	Thấp hơn do Client truy cập trực tiếp Database	Cao hơn vì Database chỉ nhận dữ liệu từ Business Logic
Khả năng mở rộng	Khó mở rộng khi có nhiều Client	Dễ mở rộng hơn do xử lý nghiệp vụ được tách riêng
Hiệu suất	Nhanh trong hệ thống nhỏ	Tốt hơn cho hệ thống lớn, tải được phân bổ tốt hơn



Mô hình N lớp (N-Tier Architecture) là một mô hình mở rộng của kiến trúc 3 lớp, trong đó hệ thống được chia thành nhiều tầng (N) để tăng tính linh hoạt, bảo mật và khả năng mở rộng.

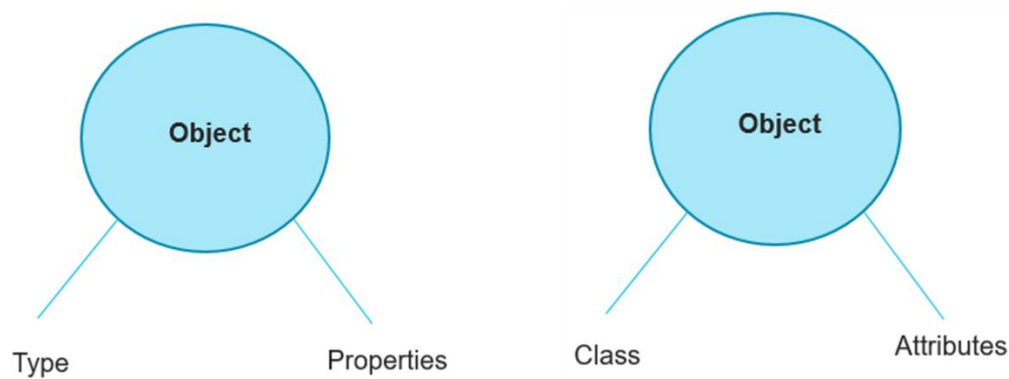
- $N \geq 3$, tức là có ít nhất 3 tầng (thường là Presentation, Business Logic, và Data).
- Các hệ thống lớn, phức tạp như Microservices, Cloud Computing thường áp dụng mô hình này.

N-Tier Architecture



2.5. Kiến trúc hướng đối tượng

Kiến trúc hướng đối tượng là một mô hình thiết kế dựa trên sự phân chia trách nhiệm một ứng dụng hoặc hệ thống cho các thành phần tái sử dụng. Một thiết kế hướng đối tượng xem như một hệ thống với các đối tượng cộng tác với nhau, thay vì một bộ các chương trình hoặc hướng dẫn thủ tục. Đối tượng là rời rạc, độc lập, và liên kết lỏng lẻo. Chúng giao tiếp thông qua các giao diện bằng cách gọi phương thức hoặc truy cập vào thuộc tính của các đối tượng khác.



Kiến trúc hướng đối tượng là một mô hình thiết kế dựa trên sự phân chia trách nhiệm một ứng dụng hoặc hệ thống cho các thành phần tái sử dụng. Một thiết kế hướng đối tượng xem như một hệ thống với các đối tượng cộng tác với nhau, thay vì một bộ các chương trình hoặc hướng dẫn thủ tục.

Đối tượng là rời rạc, độc lập, và liên kết lỏng lẻo. Chúng giao tiếp thông qua các giao diện bằng cách gọi phương thức hoặc truy cập vào thuộc tính của các đối tượng khác

Các khái niệm cốt lõi:

- Đối tượng (Object): Một thực thể cụ thể trong hệ thống, có trạng thái (dữ liệu) và hành vi (các thao tác có thể thực hiện).

Ví dụ, trong một ứng dụng quản lý thư viện:

"sách", "người đọc", "phiếu mượn" có thể là các đối tượng.

- Lớp (Class): Khuôn mẫu hoặc bản thiết kế cho một hoặc nhiều đối tượng có cùng kiểu, thuộc tính và hành vi.

Ví dụ, lớp "Sách" định nghĩa các thuộc tính chung như: tiêu đề, tác giả, ISBN và các hành vi như hiển thị thông tin, cho mượn.

Các đối tượng sách cụ thể (ví dụ: "Truyện Kiều", "Đắc Nhân Tâm") là các thể hiện (instance) của lớp "Sách".

- Tính đóng gói (Encapsulation): Việc gói gọn dữ liệu và các phương thức thao tác trên dữ liệu đó vào bên trong một đối tượng. Điều này giúp bảo vệ dữ liệu khỏi sự truy cập và sửa đổi trực tiếp từ bên ngoài, tăng tính bảo mật và giảm sự phụ thuộc lẫn nhau giữa các thành phần.

- Tính trừu tượng (Abstraction): Ẩn đi các chi tiết phức tạp bên trong đối tượng và chỉ hiển thị những thông tin và thao tác cần thiết cho người sử dụng. Ví dụ, khi sử dụng đối tượng "Xe hơi", người dùng không cần biết cơ chế hoạt động phức tạp của động cơ mà chỉ cần quan tâm đến các thao tác như tăng tốc, phanh, lái.

- Tính kế thừa (Inheritance): Cho phép một lớp (lớp con) kế thừa các thuộc tính và phương thức của một lớp khác (lớp cha). Điều này giúp tái sử dụng mã nguồn, giảm sự trùng lặp và xây dựng hệ thống theo cấu trúc phân cấp. Ví dụ, lớp "Xe tải" và "Xe con" có thể kế thừa các thuộc tính chung từ lớp "Xe".

- Tính đa hình (Polymorphism): Cho phép các đối tượng thuộc các lớp khác nhau phản ứng khác nhau với cùng một thông điệp (gọi phương thức). Ví dụ, phương thức "di chuyển" có thể được thực hiện khác nhau trên đối tượng "Xe đạp" và đối tượng "Máy bay".

- Giao tiếp bằng thông điệp (Message Passing): Các đối tượng tương tác với nhau bằng cách gửi và nhận thông điệp (gọi các phương thức của nhau).

Các nguyên tắc cơ bản của kiểu kiến trúc hướng đối tượng là:

- Trừu tượng: Điều này cho phép chúng ta giảm độ phức tạp của hành động mà vẫn giữ được các đặc điểm cơ bản của hành động. Ví dụ, một giao diện trừu tượng có thể hỗ trợ các hoạt động truy cập dữ liệu bằng cách sử dụng phương thức đơn giản như get() và set().

- Thành phần: Đối tượng có thể được lắp ráp từ các đối tượng khác, và có thể chọn để ẩn các đối tượng nội bộ từ các lớp khác hoặc tiếp xúc với chúng thông qua giao diện.

- Kế thừa: Đối tượng có thể kế thừa từ các đối tượng khác và sử dụng chức năng trong các đối tượng cơ bản hoặc ghi đè lên nó để thực hiện hành vi mới. Hơn nữa, kế thừa làm cho bảo trì và cập nhật dễ dàng hơn, như thay đổi các đối tượng cơ sở được truyền tự động cho các đối tượng kế thừa.

- Đóng gói: Đối tượng tiếp xúc với đối tượng khác chỉ thông qua các phương thức, thuộc tính, và các sự kiện, và ẩn các chi tiết nội bộ như các biến từ các đối tượng khác và trạng thái. Điều này làm cho nó dễ dàng cập nhật hoặc thay thế các đối tượng, miễn là giao diện tương thích không ảnh hưởng các đối tượng và mã khác.

- Đa hình: Kỹ thuật này cho phép ghi đè lên hành vi của lớp cơ sở bằng cách thực hiện hành vi mới được thay đổi cho đối tượng hiện tại.

- Tách biệt: Đối tượng có thể được tách biệt từ người dùng bằng cách định nghĩa một giao diện trừu tượng mà các đối tượng cụ thể và người dùng có thể hiểu được.

Kiểu hướng đối tượng được sử dụng phổ biến trong mô hình đối tượng ở các lĩnh vực hoạt động khoa học, nghiệp vụ phức tạp (chẳng hạn như thông tin khách hàng hoặc đơn đặt hàng).

Những ưu điểm của kiểu kiến trúc hướng đối tượng

- Dễ hiểu: mô hình đối tượng có liên hệ chặt chẽ với các đối tượng trong thế giới thực nên làm cho dễ hiểu kiến trúc hơn.

- Tái sử dụng: cung cấp cho việc sử dụng lại thông qua tính đa hình và trừu tượng.

- Dễ kiểm chứng: cung cấp cách để cải thiện khả năng kiểm tra thông qua đóng gói.

- Dễ mở rộng: đóng gói, đa hình, và trừu tượng đảm bảo cho một sự thay đổi trong dữ liệu mà không ảnh hưởng đến giao diện và các đối tượng khác.

- Tính kết dính cao: bằng cách định vị các phương thức và các thuộc tính của một đối tượng, các đối tượng khác nhau có thể sử dụng phương thức và các thuộc tính của nhau và như vậy đạt được sự kết dính cao.

Chọn kiểu kiến trúc hướng đối tượng nếu chúng ta muốn mô hình ứng dụng dựa trên các đối tượng thế giới thực, hoặc đã có những đối tượng và các lớp phù hợp với thiết kế và các yêu cầu hoạt động. Kiểu hướng đối tượng cũng thích hợp nếu chúng ta cần phải

đóng gói logic và dữ liệu với nhau thành các thành phần tái sử dụng hoặc có logic nghiệp vụ phức tạp đòi hỏi trừu tượng hóa và mô hình.

Các ứng dụng phổ biến của OOA:

- Phát triển phần mềm: Được sử dụng rộng rãi trong nhiều ngôn ngữ lập trình như Java, C++, Python, C#, v.v.
- Phát triển trò chơi: Tạo các đối tượng trong trò chơi như nhân vật, vật phẩm và môi trường.
- Phát triển web: Xây dựng các ứng dụng web phức tạp với các thành phần mô-đun.
- Trí tuệ nhân tạo: Mô hình hóa các thực thể trong thế giới thực và sự tương tác của chúng.

Ví dụ: Hệ thống quản lý thư viện. Trong OOA, bạn có thể có các đối tượng như:

- **Sách:**
 - Thuộc tính: Tiêu đề, Tác giả, ISBN, Tính khả dụng
 - Phương pháp: Mượn, Trả lại, Kiểm tra tính khả dụng
- **Thành viên:**
 - Thuộc tính: Tên, ID, MembershipType
 - Phương pháp: BorrowBook, ReturnBook, ViewBorrowHistory

Các đối tượng này tương tác với nhau để thể hiện chức năng của thư viện.

Tóm lại, OOA là một mô hình mạnh mẽ để thiết kế các hệ thống phần mềm có tính mô-đun, có thể tái sử dụng và có thể bảo trì. Bằng cách tập trung vào các đối tượng và tương tác của chúng, các nhà phát triển có thể tạo ra các ứng dụng mạnh mẽ và thích ứng hơn.

Ví dụ 2: Game nhập vai

- **Đối tượng:** Nhân vật, Quái vật, Vật phẩm, Bản đồ
- **Thuộc tính:**
 - Nhân vật: Tên, Lớp nhân vật, Mức độ, Sức mạnh, Máu
 - Quái vật: Tên, Loại quái vật, Mức độ, Sức mạnh, Máu
 - Vật phẩm: Tên, Loại vật phẩm, Hiệu ứng

- Bản đồ: Kích thước, Các ô, Các đối tượng trên bản đồ
- **Phương thức:**
 - Nhân vật: Di chuyển, Tấn công, Sử dụng vật phẩm
 - Quái vật: Di chuyển, Tấn công
 - Vật phẩm: Sử dụng
- **Quan hệ:**
 - Nhân vật có thể tương tác với quái vật và vật phẩm.
 - Quái vật xuất hiện trên bản đồ.

Ví dụ 3: Ứng dụng đặt vé máy bay

- **Đối tượng:** Hành khách, Chuyến bay, Máy bay, Sân bay
- **Thuộc tính:**
 - Hành khách: Tên, Số điện thoại, Email, Giới tính, Quốc tịch
 - Chuyến bay: Mã chuyến bay, Sân bay đi, Sân bay đến, Thời gian khởi hành, Thời gian đến, Giá vé
 - Máy bay: Mã máy bay, Số ghế, Hãng sản xuất
 - Sân bay: Tên sân bay, Mã sân bay, Địa chỉ
- **Phương thức:**
 - Hành khách: Đặt vé, Hủy vé, Thay đổi thông tin
 - Chuyến bay: Kiểm tra tình trạng chuyến bay
 - Máy bay: Kiểm tra số ghế trống
- **Quan hệ:**
 - Một chuyến bay thuộc về một máy bay.
 - Một chuyến bay khởi hành từ một sân bay và đến một sân bay khác.
 - Một hành khách có thể đặt nhiều chuyến bay.

Bài tập:

Hãy chọn một trong các ví dụ trên hoặc tự nghĩ ra một ví dụ khác (ví dụ: hệ thống quản lý bán hàng, ứng dụng hẹn hò,...) và:

1. **Xác định các đối tượng:** Liệt kê các đối tượng chính trong hệ thống của bạn.

2. **Xác định các thuộc tính:** Xác định các thuộc tính của mỗi đối tượng.
3. **Xác định các phương thức:** Xác định các hành động mà mỗi đối tượng có thể thực hiện.
4. **Xây dựng mối quan hệ:** Mô tả mối quan hệ giữa các đối tượng.
5. **Vẽ sơ đồ lớp:** Sử dụng UML (Unified Modeling Language) để vẽ sơ đồ lớp mô tả cấu trúc của hệ thống.

2.6. Kiến trúc hướng dịch vụ

Kiến trúc hướng dịch vụ là gì?

Kiến trúc hướng dịch vụ (SOA) là một phương pháp phát triển phần mềm sử dụng các thành phần của phần mềm được gọi là dịch vụ để tạo ra các ứng dụng dành cho doanh nghiệp. Mỗi dịch vụ cung cấp một tính năng doanh nghiệp, đồng thời các dịch vụ cũng có thể giao tiếp với nhau giữa nhiều nền tảng và ngôn ngữ. Nhà phát triển tận dụng SOA để tái sử dụng các dịch vụ trong nhiều hệ thống khác nhau hoặc kết hợp một số dịch vụ độc lập để thực hiện các tác vụ phức tạp.

Ví dụ: nhiều quy trình kinh doanh trong tổ chức yêu cầu chức năng xác thực người dùng. Thay vì phải viết lại đoạn mã xác thực cho tất cả quy trình kinh doanh, bạn có thể tạo và tái sử dụng duy nhất một dịch vụ xác thực cho mọi ứng dụng. Tương tự, hầu hết mọi hệ thống trong tổ chức chăm sóc sức khỏe, chẳng hạn như hệ thống quản lý bệnh nhân và hệ thống hồ sơ y tế điện tử (EHR), cần đăng ký bệnh nhân. Những hệ thống này có thể gọi một dịch vụ chung để thực hiện tác vụ đăng ký bệnh nhân.

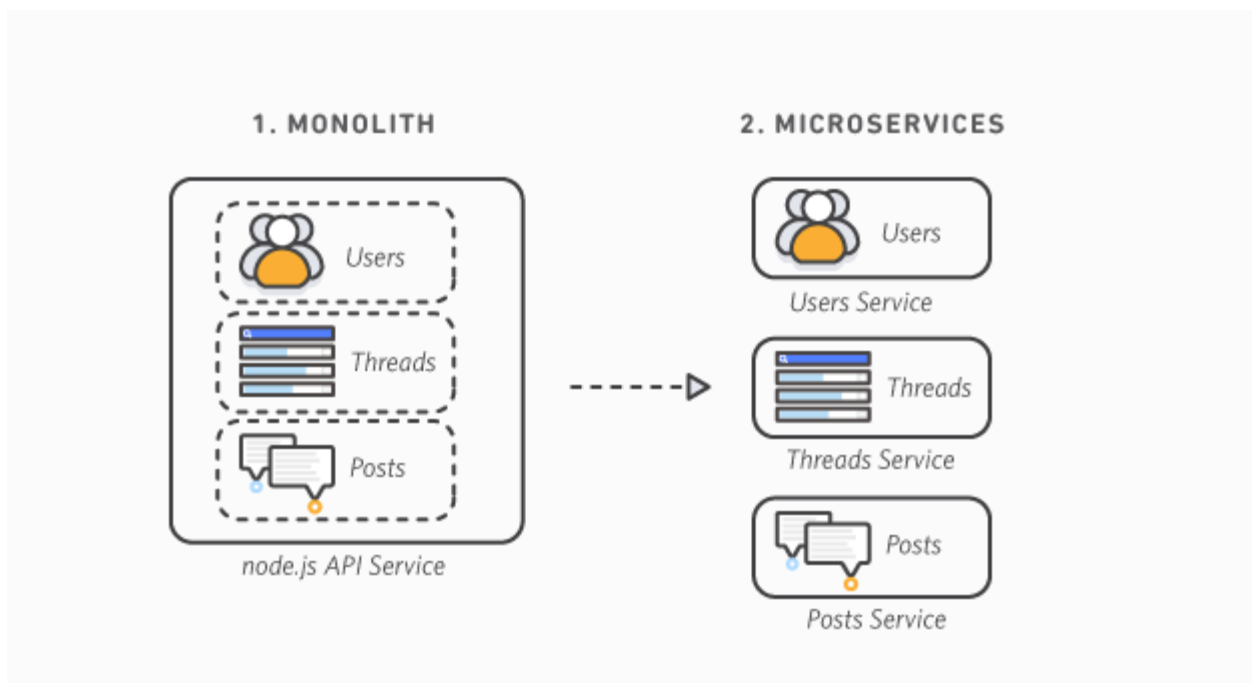
Kiến trúc hướng dịch vụ mang lại những lợi ích gì?

Kiến trúc hướng dịch vụ (SOA) có một vài lợi ích so với các kiến trúc đơn khối truyền thống, trong đó mọi quy trình hoạt động như một đơn vị duy nhất. Một số lợi ích đáng kể của SOA bao gồm:

Rút ngắn thời gian đưa ra thị trường: Nhà phát triển tái sử dụng các dịch vụ trên những quy trình kinh doanh khác nhau để tiết kiệm thời gian và chi phí. Họ có thể hợp dịch các ứng dụng nhanh hơn bằng SOA so với việc lập trình và thực hiện tích hợp từ đầu.

Bảo trì hiệu quả: Các dịch vụ nhỏ dễ tạo dựng, cập nhật và khắc phục lỗi hơn những đoạn mã lớn trong ứng dụng đơn khối. Việc sửa đổi bất kỳ dịch vụ nào trong SOA cũng không làm ảnh hưởng đến chức năng tổng thể của quy trình kinh doanh.

Khả năng thích ứng cao hơn: SOA dễ thích ứng hơn với những cải tiến về công nghệ. Bạn có thể hiện đại hóa các ứng dụng của mình một cách hiệu quả và tiết kiệm. Ví dụ: các tổ chức chăm sóc sức khỏe có thể sử dụng chức năng của các hệ thống hồ sơ y tế điện tử cũ hơn vào những ứng dụng mới trên nền tảng đám mây.



Những nguyên tắc cơ bản của kiến trúc hướng dịch vụ

Không có hướng dẫn tiêu chuẩn được xác định rõ ràng cho việc triển khai kiến trúc hướng dịch vụ (SOA). Tuy nhiên, có một vài nguyên tắc cơ bản phổ biến trong các quy trình triển khai SOA.

Khả năng tương tác: Mỗi dịch vụ trong SOA bao gồm các tài liệu mô tả chi rõ chức năng của dịch vụ cùng các điều khoản và điều kiện liên quan. Mọi hệ thống máy khách đều có thể vận hành dịch vụ, dù dùng nền tảng cơ sở hay ngôn ngữ lập trình gì. Ví dụ: các quy trình kinh doanh có thể sử dụng dịch vụ được lập trình bằng C# và Python. Do không có sự tương tác trực tiếp, các thay đổi ở một dịch vụ không ảnh hưởng đến các thành phần khác sử dụng dịch vụ này.

Liên kết ít phụ thuộc: Các dịch vụ trong SOA cần được liên kết ít phụ thuộc, có càng ít sự phụ thuộc vào các tài nguyên bên ngoài như mô hình dữ liệu hay hệ thống thông tin thì càng tốt. Các dịch vụ này cũng nên ở tình trạng vô trạng thái mà không giữ lại bất kỳ thông tin nào từ các phiên làm việc hay giao dịch trước. Bằng cách này, khi bạn sửa đổi một dịch vụ, các ứng dụng máy khách và dịch vụ khác đang sử dụng dịch vụ này sẽ không bị ảnh hưởng đáng kể.

Tính trừu tượng: Khách hàng hoặc người sử dụng dịch vụ trong SOA không cần biết logic lập trình hay chi tiết triển khai của dịch vụ. Đối với họ, dịch vụ nên như một chiếc hộp đen. Khách hàng nhận được thông tin cần thiết về tính năng và cách sử dụng dịch vụ thông qua hợp đồng dịch vụ và các tài liệu mô tả dịch vụ khác.

Độ chi tiết: Các dịch vụ trong SOA nên có kích thước và phạm vi phù hợp, lý tưởng là đóng gói một chức năng kinh doanh riêng biệt mỗi dịch vụ. Sau đó, nhà phát triển có thể sử dụng nhiều dịch vụ để tạo ra một dịch vụ tổng hợp phục vụ việc thực hiện những thao tác phức tạp.

Những thành phần của Kiến trúc hướng dịch vụ

Có bốn thành phần chính trong kiến trúc hướng dịch vụ (SOA).

Dịch vụ

Dịch vụ là những khối dựng cơ bản của SOA. Chúng có thể là dịch vụ tư nhân – chỉ dành cho người dùng nội bộ của một tổ chức – hoặc công cộng – tất cả mọi người đều có thể truy cập dịch vụ đó qua Internet. Cụ thể, mỗi dịch vụ có ba đặc điểm chính.

Triển khai dịch vụ

Triển khai dịch vụ là phần mã xây dựng logic để thực hiện chức năng dịch vụ cụ thể, ví dụ như xác thực người dùng hoặc tính toán hóa đơn.

Hợp đồng dịch vụ

Hợp đồng dịch vụ xác định bản chất của dịch vụ cùng các điều khoản và điều kiện liên quan, chẳng hạn như những điều kiện tiên quyết để sử dụng dịch vụ, chi phí dịch vụ, và chất lượng của dịch vụ được cung cấp.

Giao diện dịch vụ: Trong SOA, các dịch vụ hoặc hệ thống khác giao tiếp với một dịch vụ thông qua giao diện của dịch vụ đó. Giao diện xác định cách bạn có thể gọi dịch

vụ để thực hiện các hoạt động hoặc trao đổi dữ liệu. Giao diện giúp làm giảm sự phụ thuộc giữa dịch vụ và trình yêu cầu dịch vụ. Ví dụ: ngay cả những người dùng có ít hoặc không có kiến thức về logic lập trình cơ sở cũng có thể sử dụng dịch vụ thông qua giao diện.

Nhà cung cấp dịch vụ: Nhà cung cấp dịch vụ tạo dựng, duy trì và cung cấp một hoặc nhiều dịch vụ mà người khác có thể sử dụng. Các tổ chức có thể tự tạo dịch vụ của riêng họ hoặc mua từ một nhà cung cấp dịch vụ bên thứ ba.

Người sử dụng dịch vụ: Người sử dụng dịch vụ yêu cầu nhà cung cấp dịch vụ vận hành một dịch vụ cụ thể. Đó có thể là cả một hệ thống, một ứng dụng, hay một dịch vụ khác. Hợp đồng dịch vụ nêu rõ các quy tắc nhà cung cấp dịch vụ và người sử dụng dịch vụ phải tuân theo khi tương tác với nhau. Nhà cung cấp dịch vụ và người sử dụng dịch vụ có thể đến từ các bộ phận, tổ chức và thậm chí là các ngành khác nhau.

Sổ đăng ký dịch vụ: Sổ đăng ký dịch vụ, hay kho dịch vụ, là một danh mục các dịch vụ có sẵn, có thể truy cập qua mạng. Sổ đăng ký dịch vụ này lưu trữ các tài liệu mô tả dịch vụ từ các nhà cung cấp dịch vụ. Tài liệu mô tả dịch vụ chứa thông tin và cách giao tiếp với dịch vụ. Người sử dụng dịch vụ có thể dễ dàng tìm được những dịch vụ họ cần bằng cách dùng sổ đăng ký dịch vụ.

Kiến trúc hướng dịch vụ hoạt động như thế nào?

Trong kiến trúc hướng dịch vụ (SOA), các dịch vụ hoạt động độc lập và cung cấp chức năng hoặc trao đổi dữ liệu cho người sử dụng. Người sử dụng yêu cầu thông tin và gửi dữ liệu đầu vào cho dịch vụ. Dịch vụ xử lý dữ liệu, thực hiện tác vụ và gửi lại phản hồi.

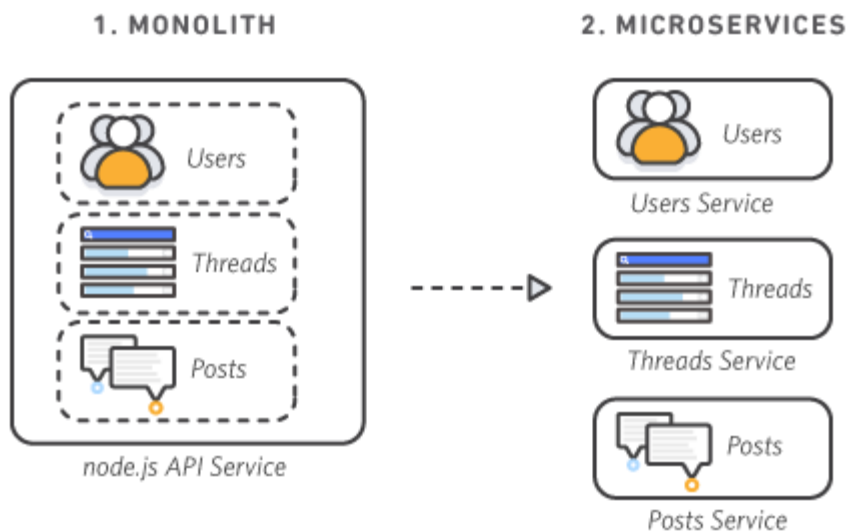
Ví dụ: nếu một ứng dụng sử dụng dịch vụ ủy quyền, ứng dụng đó sẽ cung cấp tên tài khoản và mật khẩu cho dịch vụ. Dịch vụ xác minh tên tài khoản và mật khẩu rồi gửi lại phản hồi thích hợp.

Giao thức giao tiếp: Các dịch vụ giao tiếp bằng những quy tắc được thiết lập sẵn, quyết định việc trao đổi thông tin qua một mạng lưới. Những quy tắc này được gọi là giao thức giao tiếp. Sau đây là một vài giao thức tiêu chuẩn để triển khai SOA:

- Giao thức truy cập đối tượng đơn giản (SOAP)
- HTTP RESTful
- Apache Thrift

- Apache ActiveMQ
- Dịch vụ thông báo Java (JMS)

Bạn thậm chí có thể sử dụng nhiều hơn một giao thức trong khi triển khai SOA.



ESB trong kiến trúc hướng dịch vụ là gì?

Bus dịch vụ doanh nghiệp (ESB) là phần mềm bạn có thể sử dụng khi giao tiếp với một hệ thống có nhiều dịch vụ. ESB thiết lập giao tiếp giữa dịch vụ và người sử dụng dịch vụ, dù dùng công nghệ gì.

Lợi ích của ESB: ESB cung cấp khả năng giao tiếp và chuyển đổi thông qua một giao diện dịch vụ có thể tái sử dụng. Bạn có thể coi ESB là một dịch vụ tập trung định tuyến các yêu cầu dịch vụ đến dịch vụ thích hợp. ESB cũng chuyển đổi yêu cầu thành một dạng thức mà nền tảng cơ sở và ngôn ngữ lập trình của ứng dụng có thể chấp nhận.

Triển khai kiến trúc hướng dịch vụ có những hạn chế:

- Khả năng điều chỉnh quy mô hạn chế: Khả năng điều chỉnh quy mô của hệ thống bị ảnh hưởng đáng kể khi các dịch vụ chia sẻ nhiều tài nguyên và cần phối hợp để thực hiện chức năng của mình.
- Gia tăng sự phụ thuộc lẫn nhau: Các hệ thống kiến trúc hướng dịch vụ (SOA) có thể trở nên phức tạp hơn theo thời gian và gây ra sự phụ thuộc lẫn nhau giữa các dịch vụ.

SOA có thể khó sửa đổi và khắc phục lỗi nếu nhiều dịch vụ gọi nhau trong một vòng lặp. Tài nguyên chung, ví dụ như cơ sở dữ liệu tập trung, cũng có thể làm chậm hệ thống.

- Điểm lỗi chí mạng đơn lẻ: Đối với việc triển khai SOA với ESB, ESB tạo ra một điểm lỗi chí mạng đơn lẻ. Đó là một dịch vụ tập trung, đi ngược lại ý tưởng phi tập trung mà SOA ủng hộ. Máy khách và dịch vụ không thể giao tiếp với nhau nếu ESB gặp sự cố.

Vi dịch vụ là gì?

Kiến trúc vi dịch vụ được cấu tạo từ các thành phần rất nhỏ và hoàn toàn độc lập của phần mềm, được gọi là vi dịch vụ, chỉ chuyên và tập trung vào một tác vụ duy nhất. Vi dịch vụ giao tiếp thông qua API, những quy tắc nhà phát triển tạo ra để các hệ thống phần mềm khác giao tiếp với vi dịch vụ của họ.

Phong cách kiến trúc vi dịch vụ phù hợp nhất với các môi trường điện toán đám mây hiện đại. Chúng thường hoạt động trong bộ chứa – những đơn vị phần mềm độc lập đóng gói mã cùng các thành phần phụ thuộc của mã.

Lợi ích của vi dịch vụ

Vi dịch vụ có thể độc lập điều chỉnh phạm vi, nhanh chóng, di động và không phụ thuộc vào nền tảng – những đặc điểm tương tự đám mây. Chúng cũng được tách riêng, tức là phụ thuộc rất ít hoặc không phụ thuộc vào các vi dịch vụ khác. Để đạt được điều này, vi dịch vụ phải có quyền truy cập cục bộ tới tất cả dữ liệu cần thiết thay vì truy cập từ xa tới dữ liệu tập trung mà các hệ thống khác cũng truy cập và sử dụng. Điều này tạo ra sự trùng lặp dữ liệu, được vi dịch vụ bù đắp bằng hiệu suất và tính linh hoạt.

So sánh SOA với vi dịch vụ

Kiến trúc vi dịch vụ là sự tiến hóa của phong cách kiến trúc SOA. Vi dịch vụ giải quyết những thiếu sót của SOA để khiến phần mềm trở nên tương thích hơn với những môi trường doanh nghiệp trên nền tảng đám mây hiện đại. Chúng có mức độ chi tiết cao và ưu tiên trùng lặp dữ liệu thay vì chia sẻ dữ liệu. Điều này khiến vi dịch vụ hoàn toàn độc lập với những giao thức giao tiếp riêng, có thể nhìn thấy qua các API gọn nhẹ. Về cơ bản, việc của người sử dụng là dùng vi dịch vụ thông qua API của nó, từ đó loại bỏ nhu cầu cần có ESB tập trung.

AWS giúp bạn triển khai vi dịch vụ như thế nào?

AWS là nơi tuyệt vời để xây dựng những ứng dụng hiện đại bằng các mẫu kiến trúc mô-đun, mô hình vận hành phi máy chủ và quy trình phát triển linh hoạt. AWS mang lại nền tảng hoàn thiện nhất để xây dựng vi dịch vụ có tính sẵn sàng cao nhằm hỗ trợ cho những ứng dụng hiện đại ở mọi phạm vi và quy mô. Ví dụ: bạn có thể thực hiện những việc sau:

- Xây dựng, cô lập và chạy các vi dịch vụ bảo mật trong các bộ chứa được quản lý để đơn giản hóa thao tác và giảm chi phí quản lý gián tiếp.
- Sử dụng AWS Lambda để chạy các vi dịch vụ mà không cần cung cấp và quản lý các máy chủ.
- Chọn trong số 15 cơ sở dữ liệu AWS quan hệ và phi quan hệ được xây dựng cho mục đích nhất định để hỗ trợ kiến trúc vi dịch vụ.
- Dễ dàng giám sát và kiểm soát các vi dịch vụ đang chạy trên AWS với AWS App Mesh.
- Giám sát và khắc phục sự cố của các tương tác vi dịch vụ phức tạp với AWS X-Ray.

Vi dịch vụ trên AWS giúp bạn đổi mới nhanh hơn, giảm thiểu rủi ro, tăng tốc thời gian đưa ra thị trường và làm giảm tổng chi phí sở hữu của bạn. Bắt đầu sử dụng SOA và vi dịch vụ trên AWS bằng cách tạo tài khoản AWS ngay hôm nay.

Câu hỏi và bài tập cuối chương 2:

Câu hỏi trắc nghiệm:

1. Kiến trúc nào sau đây phân chia hệ thống thành các lớp riêng biệt, mỗi lớp có một trách nhiệm cụ thể?
 - a) Kiến trúc microservices
 - b) Kiến trúc hướng sự kiện
 - c) Kiến trúc lớp (Layered Architecture)
 - d) Kiến trúc máy chủ-khách hàng
2. Kiến trúc nào sau đây bao gồm một tập hợp các dịch vụ nhỏ, độc lập và có thể triển khai riêng lẻ?
 - a) Kiến trúc lớp
 - b) Kiến trúc máy chủ-khách hàng
 - c) Kiến trúc microservices
 - d) Kiến trúc hướng sự kiện
3. Kiến trúc nào sau đây dựa trên việc truyền tải các sự kiện giữa các thành phần?
 - a) Kiến trúc lớp
 - b) Kiến trúc máy chủ-khách hàng
 - c) Kiến trúc microservices
 - d) Kiến trúc hướng sự kiện
4. Kiến trúc nào sau đây bao gồm một máy chủ trung tâm và nhiều máy khách kết nối đến máy chủ đó?
 - a) Kiến trúc lớp
 - b) Kiến trúc máy chủ-khách hàng
 - c) Kiến trúc microservices
 - d) Kiến trúc hướng sự kiện
5. Kiến trúc nào sau đây phù hợp cho các ứng dụng web phức tạp với nhiều tính năng?
 - a) Kiến trúc lớp
 - b) Kiến trúc máy chủ-khách hàng

- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

6. Kiến trúc nào sau đây phù hợp cho các ứng dụng thời gian thực, chẳng hạn như hệ thống IoT?

- a) Kiến trúc lớp
- b) Kiến trúc máy chủ-khách hàng
- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

7. Kiến trúc nào sau đây giúp tăng cường khả năng mở rộng và khả năng phục hồi của hệ thống?

- a) Kiến trúc lớp
- b) Kiến trúc máy chủ-khách hàng
- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

8. Kiến trúc nào sau đây có thể gặp khó khăn trong việc quản lý sự phức tạp của hệ thống?

- a) Kiến trúc lớp
- b) Kiến trúc máy chủ-khách hàng
- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

9. Kiến trúc nào sau đây có thể gặp khó khăn trong việc đảm bảo tính nhất quán của dữ liệu?

- a) Kiến trúc lớp
- b) Kiến trúc máy chủ-khách hàng
- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

10. Kiến trúc nào sau đây có thể gặp khó khăn trong việc gỡ lỗi và kiểm thử?

- a) Kiến trúc lớp
- b) Kiến trúc máy chủ-khách hàng

- c) Kiến trúc microservices
- d) Kiến trúc hướng sự kiện

Câu hỏi lý thuyết:

1. Mô tả kiến trúc ống và lọc. Hãy giải thích các thành phần chính và cách chúng tương tác với nhau.
2. So sánh kiến trúc ống và lọc với các kiến trúc phần mềm khác (ví dụ: kiến trúc lớp, kiến trúc microservices).
3. Nêu ưu điểm và nhược điểm của kiến trúc ống và lọc.
4. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc ống và lọc?
Hãy trình bày các bước cơ bản.
5. Các yếu tố nào cần cân nhắc khi thiết kế các ống lọc trong hệ thống?
6. Làm thế nào để đảm bảo hiệu suất của một hệ thống sử dụng kiến trúc ống và lọc?
7. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc ống và lọc.
(ví dụ: xử lý hình ảnh, xử lý ngôn ngữ tự nhiên, hệ thống log)
8. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc ống và lọc được áp dụng.
9. Các vấn đề thường gặp khi sử dụng kiến trúc ống và lọc là gì? (ví dụ: tắc nghẽn, lỗi trong một ống lọc). Làm thế nào để giải quyết các vấn đề này?
10. Mô tả kiến trúc hướng sự kiện. Hãy giải thích các thành phần chính (như nhà sản xuất sự kiện, nhà tiêu thụ sự kiện, kênh sự kiện) và cách chúng tương tác với nhau.
11. So sánh kiến trúc hướng sự kiện với kiến trúc hướng dịch vụ (SOA).
12. Nêu ưu điểm và nhược điểm của kiến trúc hướng sự kiện.
13. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc hướng sự kiện? Hãy trình bày các bước cơ bản.
14. Các yếu tố nào cần cân nhắc khi thiết kế các sự kiện trong hệ thống?
15. Làm thế nào để đảm bảo tính nhất quán và độ tin cậy của dữ liệu trong một hệ thống sử dụng kiến trúc hướng sự kiện?

16. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc hướng sự kiện. (ví dụ: hệ thống thương mại điện tử, hệ thống IoT)
17. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc hướng sự kiện được áp dụng.
18. Các vấn đề thường gặp khi sử dụng kiến trúc hướng sự kiện là gì? (ví dụ: xử lý sự kiện trùng lặp, đảm bảo thứ tự sự kiện). Làm thế nào để giải quyết các vấn đề này?
19. Mô tả kiến trúc phân lớp. Hãy giải thích các lớp điển hình (ví dụ: lớp giao diện người dùng, lớp nghiệp vụ, lớp truy cập dữ liệu) và cách chúng tương tác với nhau.
20. So sánh kiến trúc phân lớp với các kiến trúc khác (ví dụ: kiến trúc hướng dịch vụ, kiến trúc vi dịch vụ)?
21. Nêu ưu điểm và nhược điểm của kiến trúc phân lớp?
22. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc phân lớp? Hãy trình bày các bước cơ bản?
23. Các yếu tố nào cần cân nhắc khi thiết kế các lớp trong hệ thống?
24. Làm thế nào để đảm bảo tính kết hợp chặt chẽ giữa các lớp?
25. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc phân lớp. (ví dụ: ứng dụng web, hệ thống quản lý dữ liệu)?
26. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc phân lớp được áp dụng.
27. Các vấn đề thường gặp khi sử dụng kiến trúc phân lớp là gì? (ví dụ: sự phụ thuộc quá chặt chẽ giữa các lớp). Làm thế nào để giải quyết các vấn đề này?
28. Mô tả kiến trúc client-server. Hãy giải thích vai trò của client và server, và cách chúng tương tác với nhau?
29. So sánh kiến trúc client-server với các kiến trúc khác (ví dụ: kiến trúc phân tán, kiến trúc peer-to-peer)?
30. Nêu ưu điểm và nhược điểm của kiến trúc client-server?

31. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc client-server? Hãy trình bày các bước cơ bản?
32. Các yếu tố nào cần cân nhắc khi thiết kế giao thức truyền thông giữa client và server?
33. Làm thế nào để đảm bảo tính bảo mật và độ tin cậy của một hệ thống client-server?
34. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc client-server. (ví dụ: web application, email, trò chơi trực tuyến)?
35. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc client-server được áp dụng.
36. Các vấn đề thường gặp khi sử dụng kiến trúc client-server là gì? (ví dụ: hiệu suất, bảo mật, khả năng mở rộng). Làm thế nào để giải quyết các vấn đề này?
37. Mô tả kiến trúc hướng đối tượng. Hãy giải thích các khái niệm chính như lớp, đối tượng, tính kế thừa, tính đa hình và đóng gói?
38. So sánh kiến trúc hướng đối tượng với các kiến trúc khác (ví dụ: kiến trúc thủ tục).
39. Nêu ưu điểm và nhược điểm của kiến trúc hướng đối tượng?
40. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc hướng đối tượng? Hãy trình bày các bước cơ bản?
41. Các yếu tố nào cần cân nhắc khi thiết kế các lớp trong hệ thống?
42. Làm thế nào để đảm bảo tính kết hợp chặt chẽ giữa các lớp?
43. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc hướng đối tượng. (ví dụ: ứng dụng quản lý nhân sự, game)?
44. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc hướng đối tượng được áp dụng?
45. Các vấn đề thường gặp khi sử dụng kiến trúc hướng đối tượng là gì? (ví dụ: thiết kế lớp quá phức tạp, thừa kế nhiều cấp). Làm thế nào để giải quyết các vấn đề này?

46. Mô tả kiến trúc hướng dịch vụ. Hãy giải thích các thành phần chính (dịch vụ, giao diện dịch vụ, nhà cung cấp dịch vụ, nhà tiêu dùng dịch vụ) và cách chúng tương tác với nhau?
47. So sánh kiến trúc hướng dịch vụ với các kiến trúc khác (ví dụ: kiến trúc phân lớp, kiến trúc vi dịch vụ)?
48. Nêu ưu điểm và nhược điểm của kiến trúc hướng dịch vụ?
49. Làm thế nào để thiết kế một hệ thống sử dụng kiến trúc hướng dịch vụ? Hãy trình bày các bước cơ bản?
50. Các yếu tố nào cần cân nhắc khi thiết kế các dịch vụ trong hệ thống?
51. Làm thế nào để đảm bảo tính kết hợp chặt chẽ giữa các dịch vụ?
52. Đưa ra các ví dụ thực tế về các hệ thống sử dụng kiến trúc hướng dịch vụ. (ví dụ: hệ thống thương mại điện tử, hệ thống quản lý khách hàng)?
53. Phân tích một ví dụ cụ thể và giải thích cách kiến trúc hướng dịch vụ được áp dụng.
54. Các vấn đề thường gặp khi sử dụng kiến trúc hướng dịch vụ là gì? (ví dụ: quản lý dịch vụ, đảm bảo tính nhất quán). Làm thế nào để giải quyết các vấn đề này?

Bài tập vận dụng:

1. Thiết kế một hệ thống lọc spam cho email sử dụng kiến trúc ống và lọc. Xác định các ống lọc cần thiết và chức năng của chúng.
2. Tạo một hệ thống xử lý hình ảnh đơn giản sử dụng kiến trúc ống và lọc. Các ống lọc có thể bao gồm: chuyển đổi định dạng, thay đổi kích thước, làm mờ, tăng cường cạnh.
3. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc ống và lọc, và một hệ thống sử dụng kiến trúc đơn khối.
4. Thiết kế một hệ thống đặt hàng online sử dụng kiến trúc hướng sự kiện. Xác định các sự kiện chính (ví dụ: đặt hàng, thanh toán, giao hàng) và các nhà sản xuất/tiêu thụ sự kiện tương ứng.

5. Tạo một hệ thống IoT đơn giản sử dụng kiến trúc hướng sự kiện. Các sự kiện có thể bao gồm: cảm biến đo nhiệt độ, độ ẩm, chuyển động.
6. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc hướng sự kiện, và một hệ thống sử dụng kiến trúc truyền thống.
7. Thiết kế một ứng dụng quản lý thư viện đơn giản sử dụng kiến trúc phân lớp. Xác định các lớp cần thiết và cách chúng tương tác với nhau.
8. Tạo một hệ thống đăng nhập người dùng sử dụng kiến trúc phân lớp. Các lớp có thể bao gồm: lớp giao diện người dùng, lớp xác thực, lớp truy cập cơ sở dữ liệu.
9. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc phân lớp, và một hệ thống không sử dụng kiến trúc này.
10. Thiết kế một ứng dụng chat đơn giản sử dụng kiến trúc client-server. Xác định các chức năng của client và server, và giao thức truyền thông giữa chúng.
11. Tạo một hệ thống quản lý tài liệu trực tuyến sử dụng kiến trúc client-server. Các client có thể tải lên, tải xuống và chia sẻ tài liệu.
12. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc client-server, và một hệ thống không sử dụng kiến trúc này.
13. Thiết kế một lớp "Sinh viên" với các thuộc tính như tên, tuổi, mã sinh viên và các phương thức như nhập thông tin, tính điểm trung bình.
14. Tạo một hệ thống quản lý thư viện đơn giản sử dụng kiến trúc hướng đối tượng. Xác định các lớp cần thiết như sách, độc giả, mượn trả.
15. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc hướng đối tượng, và một hệ thống không sử dụng kiến trúc này.
16. Thiết kế một hệ thống đặt vé máy bay đơn giản sử dụng kiến trúc hướng dịch vụ. Xác định các dịch vụ cần thiết (ví dụ: dịch vụ tìm kiếm chuyến bay, dịch vụ đặt vé, dịch vụ thanh toán) và cách chúng tương tác với nhau.
17. Tạo một hệ thống quản lý kho hàng sử dụng kiến trúc hướng dịch vụ. Các dịch vụ có thể bao gồm: dịch vụ quản lý sản phẩm, dịch vụ quản lý đơn hàng, dịch vụ quản lý kho.

18. So sánh hiệu suất của hai hệ thống xử lý dữ liệu: một hệ thống sử dụng kiến trúc hướng dịch vụ, và một hệ thống không sử dụng kiến trúc này.

CHƯƠNG 3: THIẾT KẾ KIẾN TRÚC PHẦN MỀM

3.1. Tổng quan về thiết kế kiến trúc phần mềm

3.1.1 Khái niệm về thiết kế kiến trúc phần mềm

Trong thế giới công nghệ số ngày nay, phần mềm đã trở thành một phần không thể thiếu trong cuộc sống của chúng ta. Để xây dựng nên những phần mềm chất lượng cao, đáp ứng được nhu cầu ngày càng đa dạng của người dùng, các nhà phát triển phần mềm cần phải có một bản thiết kế chi tiết, đó chính là **thiết kế kiến trúc phần mềm**. Thiết kế kiến trúc phần mềm giống như một bản đồ chỉ đường, giúp định hướng quá trình phát triển phần mềm, đảm bảo rằng sản phẩm cuối cùng đáp ứng được các yêu cầu về chất lượng, hiệu suất và khả năng mở rộng.

Hãy tưởng tượng bạn đang xây dựng một ngôi nhà. Trước khi bắt tay vào xây dựng, bạn sẽ cần một bản thiết kế chi tiết, mô tả cấu trúc, vật liệu và cách bố trí các phòng. Tương tự như vậy, khi phát triển một phần mềm, chúng ta cũng cần một bản thiết kế chi tiết, đó chính là **thiết kế kiến trúc phần mềm**.

Thiết kế kiến trúc phần mềm là quá trình tạo ra một bản thiết kế chi tiết về cấu trúc và tổ chức của một hệ thống phần mềm. Nó bao gồm việc xác định các thành phần của hệ thống, mối quan hệ giữa các thành phần và cách thức chúng tương tác với nhau để đạt được mục tiêu chung.

Thiết kế kiến trúc phần mềm đóng vai trò quan trọng trong việc đảm bảo chất lượng, hiệu suất và khả năng mở rộng của phần mềm.

3.1.2 Tiến trình thiết kế kiến trúc

Phát triển phần mềm dù theo phương pháp luận nào cũng thường bao gồm các công đoạn xác định yêu cầu, phân tích, thiết kế, cài đặt-tích hợp và kiểm chứng. Thiết kế thường lại được chia thành hai giai đoạn là thiết kế kiến trúc và thiết kế chi tiết. Các phương pháp luận ngày nay như UP (Unified Process) và các phương pháp tiên hóa của nó đều dựa trên *phát triển lặp và tăng dần*. Vì vậy, giống như các công đoạn khác, thiết kế kiến trúc cần được tiến hành ngay từ khi khởi đầu dự án. Các hoạt động thiết kế kiến trúc xem như một quá trình lặp của một dãy các bước và mở rộng thêm khi có thông tin mới được phát hiện

trong các pha trước đó. Tiến trình thiết kế kiến trúc thông thường gồm các bước cơ bản sau đây ([1][3]):

1. *Hiểu được vấn đề cần giải quyết*
2. *Xác định các phần tử của thiết kế và quan hệ của chúng*
3. *Đánh giá thiết kế kiến trúc*
4. *Biến đổi thiết kế kiến trúc*

Bước đầu tiên được cho là then chốt vì nếu không hiểu rõ bài toán đặt ra sẽ không thể đưa ra giải pháp hữu hiệu. Bước thứ hai nhằm chỉ ra các phần tử thiết kế và sự phụ thuộc của chúng. Bước thứ ba liên quan đánh giá kiến trúc theo yêu cầu thuộc tính của chất lượng kiến trúc.

Rõ ràng, chức năng của một ứng dụng không thể kiểm chứng được từ quá trình phân rã cấu trúc. Tuy nhiên, nhiều thuộc tính chất lượng khác có thể đánh giá được như xem xét kiểu thiết kế hay cài đặt bản mẫu cho tương tác giữa các thành phần quan trọng. Bước thứ tư liên quan đến áp dụng các thao tác thiết kế để biến đổi thiết kế kiến trúc thành một thiết kế mới với những yêu cầu thuộc tính chất lượng tốt hơn thiết kế trước. Bước này có thể lặp lại nhiều lần. Tiến trình thiết kế kiến trúc được xem là sự mở rộng của tiến trình thiết kế kỹ thuật thông thường.

Mục đích của thiết kế kiến trúc là tập trung vào phân rã hệ thống thành các thành phần và tương tác giữa các thành phần sao cho thỏa mãn các yêu cầu chức năng và phi chức năng được đặt ra. Một hệ phần mềm có thể xem như một phân cấp của các quyết định thiết kế trong đó mỗi mức phân cấp có một tập luật thiết kế để ràng buộc hay kết nối các thành phần ở mức đó.

Ví dụ trong kiến trúc kiểu Client/server, các quy luật thiết kế có thể liên quan đến đặc tả giao diện giữa client và server. Mức phân cấp kế tiếp sẽ mô tả mỗi client hay mỗi server như là một tập các thành phần tương tác. Mỗi nhánh phân cấp có những luật thiết kế riêng mà các nhánh khác không nhìn thấy được. Như vậy phân rã của client là ẩn dấu đối với server và ngược lại. Đầu ra của tiến trình thiết kế kiến trúc là mô tả cấu trúc.

1 Hiểu được vấn đề cần giải quyết

Nhiều dự án hay sản phẩm phần mềm được cho là không thành công vì nó không giải quyết được vấn đề thực sự trong nghiệp vụ khách hàng hoặc không thỏa mãn lợi tức đầu tư mà phần mềm đem lại. Câu hỏi đặt ra là tại sao phải đầu tư một giải pháp mới mà chẳng đem lại lợi ích gì so với giải pháp cũ. Đa số kỹ sư phần mềm đều tập trung vào các các giải pháp kỹ thuật hơn là giải quyết vấn đề cơ bản mà nhà đầu tư yêu cầu mà thường được gọi là *bẫy cài đặt*. Để tránh cái bẫy như vậy, chúng ta nên tự đặt vấn đề lặp đi lặp lại là các quyết định thiết kế đưa ra nhằm giải quyết những vấn đề gì.

2 Xác định các phần tử thiết kế và quan hệ của chúng

Trong bước này, chúng ta xác lập một phân rã ban đầu của hệ thống dựa trên các yêu cầu chức năng bao gồm các phần tử thiết kế và các phụ thuộc giữa chúng. Kiến trúc của một hệ phần mềm thường được biểu diễn như một tập các module hay hệ thống con mà ta gọi là thành phần.

Theo cách phân cấp, các chương trình trong C hay Java cũng có thể xem như cấu trúc gồm những thành phần mức thấp hơn. Các ngôn ngữ lập trình xác định một tập các thành phần theo các từ khóa và cú pháp của ngôn ngữ đó. Khai báo số nguyên và gán giá trị cho nó cũng có thể xem như thành phần gồm một tập các thành phần mức thấp hơn là những lệnh và mẫu dữ liệu. Điều này tương tự như chip trong máy tính gồm những thành phần là cổng và cổng lại gồm những thành phần mức thấp hơn là các mạch. Như vậy, kiến trúc có thể xem như cấu trúc phân cấp của phần mềm.

Việc chỉ ra các phần tử thiết kế và quan hệ của chúng có thể phân chia hơn nữa thành các bước sau đây:

- Xác định ngữ cảnh hệ thống.
- Xác định các module
- Mô tả các thành phần và kết nối

Hai bước đầu sử dụng kỹ thuật thiết kế trừu tượng cho phép chúng ta tập trung vào mức chi tiết cụ thể mà không phải tập trung vào mọi khía cạnh của hệ thống. Bước thứ ba liên quan đến việc mô tả các thể hiện của các kiểu module trong cấu hình thời gian chạy.

3 Đánh giá thiết kế kiến trúc

Thiết kế kiến trúc có thể được đánh giá theo những cách khác nhau. Để đánh giá một thiết kế, chúng ta phải hình dung rõ ràng những yêu cầu thuộc tính chất lượng ảnh hưởng bởi thiết kế. Những phán đoán kiểu như “thiết kế phải chạy nhanh” hay “hệ thống phải dễ dàng sửa đổi hay hiệu chỉnh cho nhu cầu mới” chỉ là những yêu cầu ban đầu mà không đủ để đánh giá thiết kế. Đánh giá kiến trúc không phải là đánh giá chính sản phẩm phần mềm mà là đánh giá mô tả về mặt thiết kế kiến trúc của hệ thống. Đánh giá kiến trúc là nhằm đo chất lượng thiết kế sau khi nó được tạo ra. Không phải bao giờ cũng có thể đánh giá thiết kế để hiểu thuộc tính chất lượng như khả năng chỉnh sửa và hiệu năng.

Thông thường để tạo một hệ thống mà sau này có thể chỉnh sửa được là phân rã hệ thống thành các tầng. Tuy nhiên, các tầng có thể dẫn đến khó hiểu về mặt tính toán. Cho đến nay việc đánh giá kiến trúc phần mềm là dựa vào kiểu không hình thức trong đó các b n liên quan dự án gặp nhau và đưa ra ý kiến của mình.

4 Biến đổi thiết kế

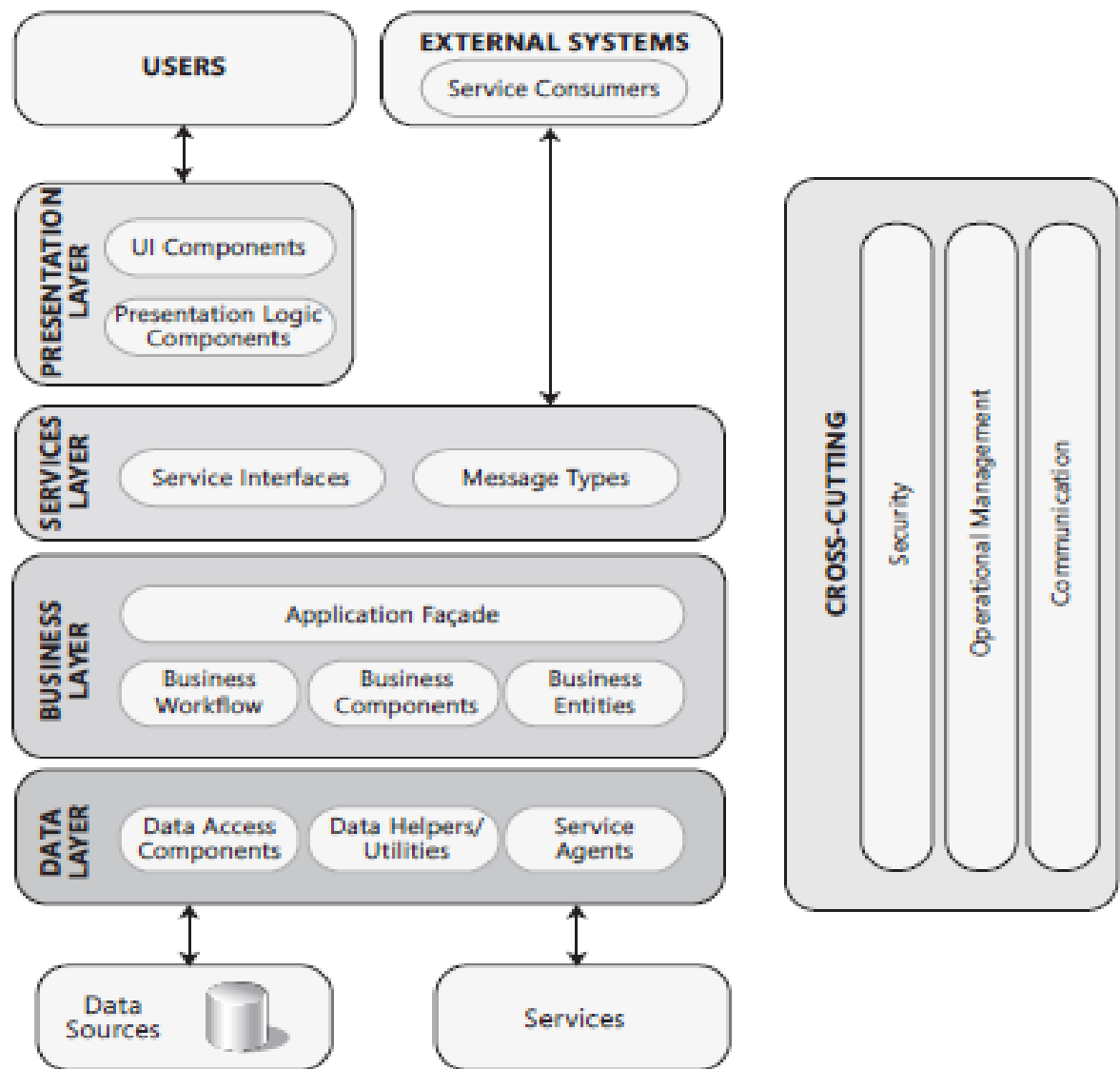
Bước này được tiến hành sau đánh giá thiết kế. Nếu thiết kế kiến trúc không thỏa mãn đầy đủ yêu cầu thuộc tính chất lượng, thì nó phải được thay đổi cho đến khi thỏa mãn yêu cầu. Như vậy, bắt đầu từ thiết kế hiện thời chúng ta thực hiện phép biến đổi bằng cách sử dụng các toán tử thiết kế, kiểu và mẫu thiết kế. Các toán tử thiết kế cơ bản bao gồm: phân rã hệ thống thành các thành phần; nén một số thành phần thành một thực thể để cải tiến hiệu năng; trừu tượng hóa thành phần để cải tiến khả năng chỉnh sửa và thích nghi; chia sẻ một thành phần với các thành phần khác để cải tiến khả năng tích hợp, chỉnh sửa.

3.1.3 Các nguyên lý thiết kế kiến trúc

Những nghiên cứu về kiến trúc cho rằng thiết kế phải tiến hóa theo thời gian vì chúng ta không thể biết được mọi điều trong khi tiến hành kiến trúc hệ thống. Do đó, thiết kế cần phải được tiến hóa suốt trong giai đoạn cài đặt ứng dụng vì khi đó chúng ta biết nhiều hơn qua kiểm chứng các yêu cầu thế giới thực. Xây dựng kiến trúc với sự tiến hóa sẽ làm cho nó có thể thích nghi được với yêu cầu chưa được biết đầy đủ trong giai đoạn đầu của quá trình thiết kế.

1 Kiến trúc phần mềm tổng quát

Một kiến trúc phần mềm thường được mô tả như một cấu trúc hay tổ chức của một hệ thống. Thuật ngữ hệ thống được hiểu như là một tập các thành phần xác lập một chức năng hay một tập các chức năng nào đó. Như vậy, kiến trúc phần mềm tập trung vào tổ chức các thành phần hay nhóm các thành phần thành những cụm có những mục tiêu khác nhau. Kiến trúc tổng quát (Hình 2.1) với các thành phần được nhóm thành những tầng có mục tiêu khác nhau [8]:



Hình 2.1: Kiến trúc hệ phần mềm tổng quát [8]

- Tầng trình diễn (presentation layer): Bao gồm các thành phần giao diện người dùng (UI: User Interface) và các thành phần logic trình diễn (Presentation logic)
- Tầng dịch vụ (service layer): Bao gồm các giao diện dịch vụ và các kiểu dịch vụ.
- Tầng nghiệp vụ (business layer): Bao gồm các thành phần, thực thể nghiệp vụ và luồng hoạt động nghiệp vụ.
- Tầng dữ liệu (data layer): Bao gồm các thành phần truy nhập dữ liệu, các tiện ích dữ liệu và access tác nhân dịch vụ.
- Tầng kết nối (cross cutting): Bao gồm an toàn, truyền thông và quản lý điều hành

Chương này sẽ xem xét những thao tác thiết kế tổng quát của thiết kế kiến trúc phần mềm và cách sử dụng chúng để phân rã hệ thống thành các thành phần và kết nối để đạt được thuộc tính chất lượng mong muốn. Các thao tác thiết kế thông thường bao gồm: phân rã, gộp, nén, trừu tượng hóa và chia sẻ tài nguyên. Các thao tác này lại được xem xét từ mức độ kiến trúc của thiết kế.

2.2.2 Nguyên lý thiết kế cơ bản

Khi bắt đầu thiết kế, cần phải chú ý đến những nguyên lý cơ bản nhằm giúp tạo ra một kiến trúc có thể tối thiểu hóa được chi phí cũng như yêu cầu bảo trì và tăng cường được khả năng mở rộng. Sau đây là một số nguyên lý cơ bản:

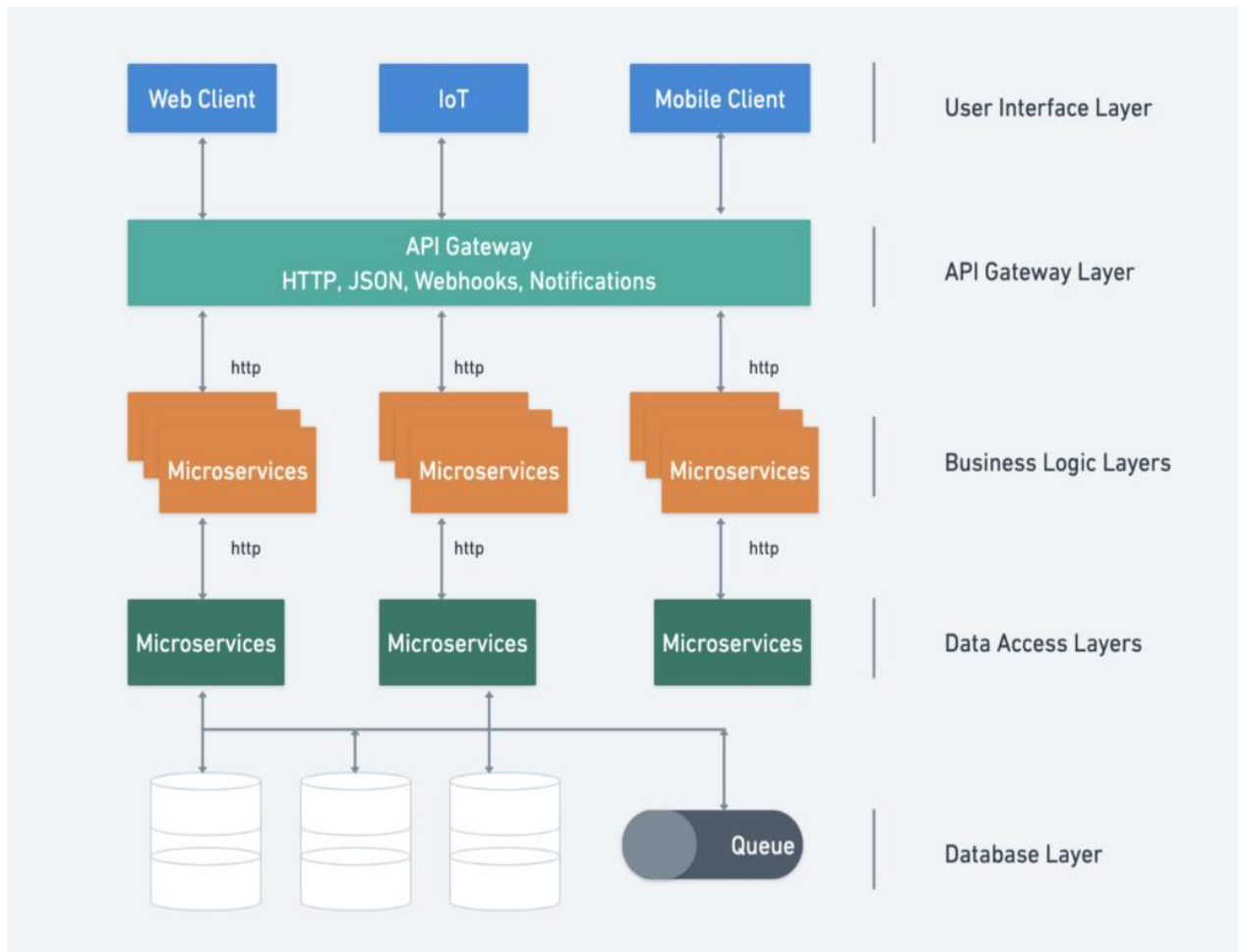
- Nguyên lý phân rã các chức năng: Phân chia ứng dụng thành những nhóm sao cho tối thiểu hóa được sự chồng lấp các chức năng. Yếu tố quan trọng là giảm thiểu những điểm tương tác để đạt được kết hợp cao (high cohesion) và kết dính thấp (low coupling). Tuy nhiên, việc phân rã các chức năng không hợp lý có thể dẫn đến sự kết dính cao giữa các chức năng dù các chức năng liên quan không thực sự chồng lấp.
- Nguyên lý đơn nhiệm: Mỗi thành phần hay môđun nên chịu trách nhiệm cho chỉ một chức năng hay một hợp chức năng có liên kết nhau.
- Nguyên lý hiểu biết tối thiểu: Một thành phần hay một đối tượng không nên biết về chi tiết bên trong của các thành phần hay các đối tượng khác.
- Nguyên lý không tự lặp lại: Chỉ nên đặc tả ý định tại một vị trí. Ví dụ, theo thiết kế ứng dụng, một chức năng cụ thể nên được cài đặt chỉ trong một thành phần. Nghĩa là chức năng đó không nên cài trong những thành phần nào khác.

- Nguyên lý cực tiểu hóa thiết kế: Chỉ thiết kế những gì cần thiết. Nguyên lý này đôi khi còn gọi là YAGNI (you ain't gonna need it). Nếu yêu cầu ứng dụng không rõ ràng hay nếu thiết kế cần phải tiến hóa theo thời gian thì tránh đưa ra thiết kế lớn.

Khi thiết kế một hệ thống hay một ứng dụng, kiến trúc sư phần mềm phải cực tiểu hóa độ phức tạp bằng cách tách thiết kế thành những quan tâm khác nhau. Ví dụ, các phần giao diện người dùng, xử lý nghiệp vụ và truy nhập dữ liệu là những thể hiện quan tâm khác nhau. Điều này được thể hiện trong mẫu thiết kế MVC (Model, View, Control) thường gặp.

Trong mỗi lĩnh vực ứng dụng, các thành phần nên tập trung vào một khía cạnh cụ thể và không nên trộn mã nguồn từ những quan tâm khác nhau. Ví dụ, các thành phần xử lý giao diện không nên chứa mã truy nhập trực tiếp nguồn dữ liệu mà nên sử dụng thành phần nghiệp vụ hay thành phần truy nhập dữ liệu để truy xuất dữ liệu.

3.2. Vai trò của thiết kế kiến trúc phần mềm



Khi thiết kế một hệ thống hay một ứng dụng, kiến trúc sư phần mềm phải cực tiểu hóa độ phức tạp bằng cách tách thiết kế thành những quan tâm khác nhau. Ví dụ, các phần giao diện người dùng, xử lý nghiệp vụ và truy nhập dữ liệu là những thể hiện quan tâm khác nhau. Điều này được thể hiện trong mẫu thiết kế MVC (Model, View, Control) thường gặp. Có thể nói, đây là bước đầu tiên, có vai trò nền tảng trong việc phát triển bất cứ một chương trình nào.

- Cung cấp thông tin cần thiết, định hướng cho các giai đoạn khác trong quá trình phát triển phần mềm: Dựa trên thiết kế đề ra, các bộ phận có thể tiến hành lập trình, xây dựng phần mềm theo đúng yêu cầu và hạn chế thấp nhất sai sót không mong muốn.
- Là cơ sở để kiểm tra, phân tích tính chính xác, mức độ phù hợp, khả năng đáp ứng yêu cầu được đặt ra của một sản phẩm sau khi hoàn thành: Mỗi phần mềm ra đời đều

có những sai số nhất định so với dự tính ban đầu. Lúc này, việc đối chiếu với thiết kế sẽ giúp tìm ra và khắc phục vấn đề nhanh chóng.

- Là nguồn tư liệu để tham khảo cho những dự án phát triển phần mềm sau này: Đôi khi, bạn sẽ gặp một số dự án có cấu trúc tương tự nhau. Lúc này, việc tham khảo thông tin sẵn có sẽ giúp tiết kiệm được nguồn lực, rút ngắn tối đa thời gian.

- Cung cấp cái nhìn tổng quan về hệ thống: Kiến trúc phần mềm giúp các nhà phát triển, quản lý dự án và các bên liên quan có một cái nhìn tổng thể về hệ thống, hiểu rõ cách các thành phần kết nối và hoạt động.

- Đảm bảo chất lượng và hiệu suất: Một kiến trúc tốt giúp đảm bảo phần mềm được xây dựng một cách có hệ thống, dễ bảo trì và mở rộng. Nó giúp tối ưu hóa hiệu suất, giảm thiểu lỗi và tăng cường khả năng đáp ứng các yêu cầu của người dùng.

- Giảm thiểu rủi ro: Bằng cách xác định rõ các thành phần và mối quan hệ giữa chúng từ sớm, thiết kế kiến trúc giúp giảm thiểu rủi ro phát sinh trong quá trình phát triển, như thay đổi yêu cầu, lỗi kỹ thuật, hoặc chậm tiến độ.

- Hỗ trợ quá trình ra quyết định: Kiến trúc phần mềm cung cấp cơ sở để đưa ra các quyết định quan trọng trong quá trình phát triển, như lựa chọn công nghệ, phân công công việc, và quản lý tài nguyên.

- Tăng cường khả năng tái sử dụng: Một kiến trúc tốt cho phép tái sử dụng các thành phần phần mềm trong các dự án khác, giảm thiểu chi phí và thời gian phát triển.

- Cải thiện khả năng hợp tác: Kiến trúc phần mềm tạo ra một ngôn ngữ chung cho các thành viên trong nhóm phát triển, giúp họ hiểu rõ vai trò và trách nhiệm của mình, đồng thời tăng cường sự hợp tác.

Tóm lại, thiết kế kiến trúc phần mềm là nền tảng vững chắc cho sự thành công của một dự án phần mềm. Nó không chỉ giúp đảm bảo chất lượng và hiệu suất của phần mềm mà còn góp phần vào việc giảm thiểu rủi ro, tăng cường khả năng hợp tác và tiết kiệm chi phí.

3.3. Phương pháp thiết kế kiến trúc ADD

3.3.1 Tổng quát về phương pháp ADD

Nguồn gốc của **Thiết kế hướng thuộc tính (ADD)** có thể bắt nguồn từ những năm 1990, khi nó được phát triển bởi Viện Kỹ thuật phần mềm (SEI) tại Đại học Carnegie Mellon. Nó được tạo ra như một phương pháp tiếp cận thiết kế phần mềm tập trung vào việc xác định và mô hình hóa các thuộc tính (đặc điểm) của một hệ thống quan trọng đối với người dùng và các bên liên quan, với mục tiêu tạo ra các hệ thống có khả năng thích ứng cao, dễ bảo trì và có thể mở rộng quy mô, và đáp ứng nhu cầu của người dùng và các bên liên quan.

Thiết kế theo thuộc tính (ADD) ban đầu được phát triển cho các ứng dụng phần mềm quan trọng như hệ thống kiểm soát không lưu và hệ thống quản lý năng lượng, nhưng đã được sử dụng trong nhiều loại thiết kế phần mềm khác. kể từ đó. Nó đã được sử dụng và nghiên cứu rộng rãi trong hàng không vũ trụ, quân sự và quốc phòng, nhưng cũng có ứng dụng trong các lĩnh vực khác bao gồm giao thông vận tải, chăm sóc sức khỏe, tài chính và công nghệ thông tin.

ADD lấy đầu vào là một tập hợp các kịch bản thuộc tính chất lượng và sử dụng kiến thức về mối quan hệ giữa việc đạt được thuộc tính chất lượng và kiến trúc để thiết kế kiến trúc. Phương pháp ADD có thể được xem như một phần mở rộng của hầu hết các phương pháp phát triển khác, chẳng hạn như Quy trình hợp nhất Rational. Quy trình hợp nhất Rational có một số bước dẫn đến thiết kế cấp cao của một kiến trúc nhưng sau đó tiến hành thiết kế và triển khai chi tiết. Việc kết hợp ADD vào đó bao gồm việc sửa đổi các bước liên quan đến thiết kế cấp cao của kiến trúc và sau đó làm theo quy trình như được Rational mô tả.

Giới thiệu về phương pháp ADD:

- ADD (Attribute-Driven Design - Thiết kế theo hướng thuộc tính) là một phương pháp thiết kế kiến trúc phần mềm.
- Nó tập trung vào việc xác định kiến trúc phần mềm dựa trên các thuộc tính chất lượng mà phần mềm phải đáp ứng.

- Đây là một quá trình phân rã đệ quy, trong đó, ở mỗi giai đoạn, các chiến thuật và mẫu kiến trúc được chọn để đáp ứng một tập hợp các kích bản chất lượng và sau đó chức năng được phân bổ để khởi tạo các loại mô-đun do mẫu cung cấp.

- ADD được định vị trong vòng đời sau khi phân tích yêu cầu và có thể bắt đầu khi các trình điều khiển kiến trúc được biết đến với một số sự tự tin.

- Đầu ra của ADD là một số cấp độ đầu tiên của chế độ xem phân tích mô-đun của kiến trúc và các chế độ xem khác khi thích hợp.

Vai trò của ADD:

- Đảm bảo chất lượng: ADD giúp đảm bảo rằng kiến trúc phần mềm đáp ứng các yêu cầu về chất lượng quan trọng, chẳng hạn như hiệu suất, khả năng mở rộng, bảo mật và khả năng bảo trì.

- Hướng dẫn thiết kế cấp cao: ADD cung cấp một khuôn khổ và các bước cụ thể để thiết kế kiến trúc ở cấp độ cao, trước khi đi vào chi tiết triển khai.

- Tạo ra kiến trúc linh hoạt: ADD giúp tạo ra các hệ thống có kiến trúc linh hoạt, dễ thích ứng và dễ dàng sửa đổi khi yêu cầu thay đổi.

- Kết hợp với các phương pháp phát triển khác: ADD có thể được tích hợp vào các phương pháp phát triển phần mềm khác (ví dụ: Quy trình hợp nhất Rational) để cải thiện quá trình thiết kế kiến trúc.

- Giảm rủi ro: Bằng cách tập trung vào các thuộc tính chất lượng quan trọng từ giai đoạn đầu của thiết kế, ADD giúp giảm thiểu rủi ro và các vấn đề tiềm ẩn trong quá trình phát triển.

- Cải thiện giao tiếp: ADD giúp tạo ra sự giao tiếp hiệu quả giữa các nhà phát triển, người dùng và các bên liên quan, giúp dễ dàng thống nhất kỳ vọng và đảm bảo rằng hệ thống đáp ứng được nhu cầu của tất cả mọi người liên quan.

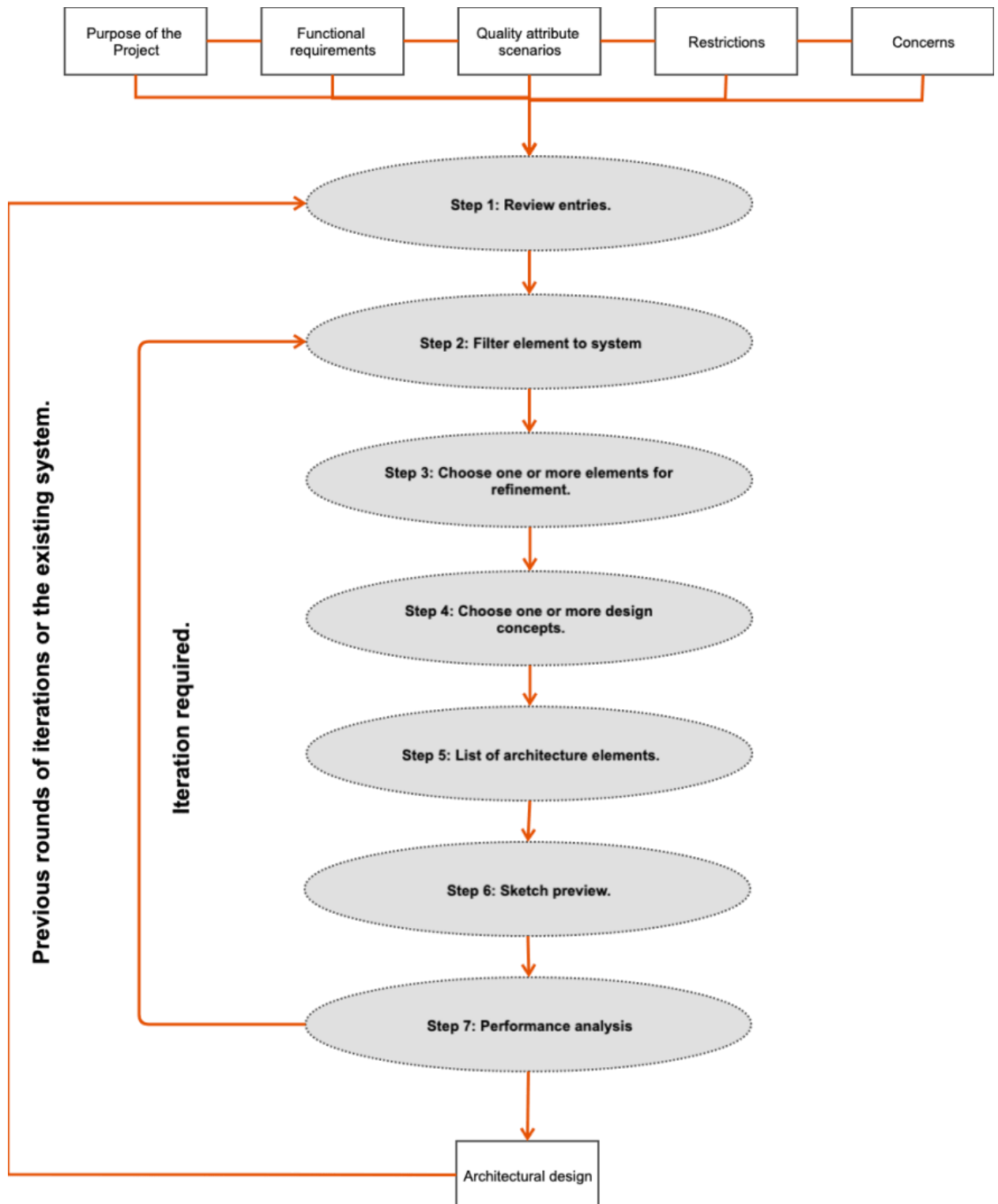
3.3.2 Các bước của phương pháp ADD:

ADD là một cách tiếp cận để xác định kiến trúc phần mềm dựa trên các thuộc tính chất lượng mà phần mềm phải đáp ứng. Đây là một quá trình phân rã đệ quy, trong đó, ở mỗi giai đoạn, các chiến thuật và mẫu kiến trúc được chọn để đáp ứng một tập hợp các kích bản chất lượng và sau đó chức năng được phân bổ để khởi tạo các loại mô-đun do mẫu

cung cấp. ADD được định vị trong vòng đời sau khi phân tích yêu cầu và, như chúng tôi đã nói, có thể bắt đầu khi các trình điều khiển kiến trúc được biết đến với một số sự tự tin.

Đầu ra của ADD là một số cấp độ đầu tiên của chế độ xem phân tích mô-đun của kiến trúc và các chế độ xem khác khi thích hợp. Không phải tất cả các chi tiết của chế độ xem đều xuất phát từ ứng dụng ADD; hệ thống được mô tả như một tập hợp các thùng chứa cho chức năng và các tương tác giữa chúng. Đây là sự khớp nối đầu tiên của kiến trúc trong quá trình thiết kế và do đó cần phải có chi tiết thô. Tuy nhiên, nó rất quan trọng để đạt được các phẩm chất mong muốn và nó cung cấp một khuôn khổ để đạt được chức năng. Sự khác biệt giữa kiến trúc xuất phát từ ADD và kiến trúc sẵn sàng để triển khai nằm ở các quyết định thiết kế chi tiết hơn cần được đưa ra. Ví dụ, chúng có thể là quyết định sử dụng các mẫu thiết kế hướng đối tượng cụ thể hoặc một phần mềm trung gian cụ thể đi kèm với nhiều ràng buộc về kiến trúc. Kiến trúc do ADD thiết kế có thể đã cố tình hoãn quyết định này để linh hoạt hơn.

Dưới đây là các bước của **Thiết kế theo thuộc tính (ADD)** và cách chúng được lặp lại.



Bước 1: Xem lại các mục nhập

Trước khi bắt đầu triển khai bước đầu tiên, các bước mục đích thiết kế, yêu cầu chức năng, kịch bản thuộc tính chất lượng, ràng buộc và các mối quan tâm trước đó cần được phân tích để đảm bảo khả năng lặp lại của các bước từ 2 đến 7.

Bước 2: Lọc phần tử vào hệ thống

Ở bước này, bạn chọn phần tử nào của hệ thống sẽ là trọng tâm của dự án để lặp lại các bước tiếp theo.

Bước 3: Chọn một hoặc nhiều phần tử để tinh chỉnh.

Trong bước này, chúng tôi phân tích phần tử đã chọn để phân tích và liệt kê các mức độ ưu tiên của các yêu cầu của bên liên quan ảnh hưởng đến phần tử. Họ có thể đặt mức độ ưu tiên Cao, Trung bình hoặc Thấp của mỗi yêu cầu được liệt kê.

Bước 4: Chọn một hoặc nhiều khái niệm thiết kế.

Ở bước này, chúng ta cần chọn khái niệm thiết kế, nghĩa là chọn các loại thành phần chính và mối quan hệ của chúng.

Bạn có thể xác định những mối lo ngại liên quan đến thiết kế và các thành phần kiến trúc có xu hướng giải quyết vấn đề.

Bước 5: Liệt kê các thành phần kiến trúc.

Trong bước này, bạn có một số loại phần tử phần mềm được chọn ở bước trước. Các phần tử này được giao trách nhiệm theo loại của chúng. Trách nhiệm của các phần tử được liệt kê được bắt nguồn từ mỗi yêu cầu chức năng liên quan đến nó.

Bước 6: Xem trước bản phác thảo.

Trong bước này, các dịch vụ và thuộc tính cần thiết được cung cấp cho từng phần tử được gọi là giao diện chứ không chỉ là danh sách các thành phần hoạt động. Giao diện có thể bao gồm bất kỳ mục nào sau đây:

- cú pháp của các hoạt động (ví dụ chữ ký)
- ngữ nghĩa của các hoạt động (ví dụ mô tả, điều kiện trước và sau, ràng buộc)
- thông tin được trao đổi (ví dụ sự kiện được đánh dấu, dữ liệu toàn cầu)
- yêu cầu về thuộc tính chất lượng của từng yếu tố hoặc hoạt động
- Xử lý lỗi.

Bước 7: Phân tích hiệu suất

Trong bước này, công việc đã thực hiện ở các bước trước sẽ được phân tích để tránh phải làm lại. Trong bước này, kiến trúc sư có thể quay lại bước 2 và bắt đầu lại từ đầu nếu có điều gì đó không đúng ngữ cảnh. Nếu mọi thứ diễn ra như mong đợi, bước tiếp theo là cung cấp một phần tử mới cho một lần lặp lại mới để bước 1 bắt đầu.

3.3.3 Ưu điểm và hạn chế của phương pháp ADD:

Mô hình **Thiết kế theo thuộc tính (ADD)** có một số ưu điểm sau:

- Tập trung vào người dùng: ADD tập trung vào việc xác định và mô hình hóa các thuộc tính quan trọng đối với người dùng và các bên liên quan, đảm bảo rằng hệ thống đáp ứng được nhu cầu và mong đợi của họ.
- Kiến trúc linh hoạt: Bằng cách mô hình hóa các thuộc tính của hệ thống, ADD cho phép tạo ra một kiến trúc linh hoạt và dễ thích ứng, có thể dễ dàng sửa đổi và mở rộng khi nhu cầu của hệ thống thay đổi.
- Dễ bảo trì: Vì ADD tập trung vào các thuộc tính quan trọng và có liên quan nên các hệ thống được thiết kế theo cách tiếp cận này thường dễ bảo trì và mở rộng hơn.
- Giao tiếp hiệu quả: ADD giúp tạo ra sự giao tiếp hiệu quả giữa các nhà phát triển, người dùng và các bên liên quan, giúp dễ dàng thống nhất kỳ vọng và đảm bảo rằng hệ thống đáp ứng được nhu cầu của tất cả mọi người liên quan.
- Giảm chi phí: Bằng cách thiết kế các hệ thống có kiến trúc linh hoạt, dễ bảo trì và mở rộng quy mô, ADD có thể giúp giảm chi phí bảo trì và phát triển trong dài hạn.

Mô hình **Thiết kế theo thuộc tính (ADD)** có một số hạn chế sau:

- Độ phức tạp: ADD có thể là một phương pháp phức tạp, đặc biệt là đối với các hệ thống lớn và phức tạp.
- Yêu cầu chuyên môn: ADD yêu cầu các nhà thiết kế kiến trúc có kiến thức sâu rộng về các thuộc tính chất lượng và các mẫu kiến trúc.
- Tốn thời gian: Việc phân tích các kịch bản chất lượng và lựa chọn các chiến thuật kiến trúc phù hợp có thể tốn nhiều thời gian.
- Khó khăn trong việc xác định các kịch bản chất lượng: Việc xác định đầy đủ và chính xác các kịch bản chất lượng có thể gặp nhiều khó khăn, đặc biệt là trong các dự án mới hoặc các lĩnh vực chưa có nhiều kinh nghiệm.

- Yêu cầu sự tham gia của các bên liên quan: Để đảm bảo hiệu quả của ADD, cần có sự tham gia tích cực của các bên liên quan, bao gồm cả người dùng cuối, trong việc xác định và đánh giá các thuộc tính chất lượng.

Tóm lại, ADD là một phương pháp mạnh mẽ để thiết kế kiến trúc phần mềm chất lượng cao, nhưng nó cũng có một số hạn chế cần được xem xét.

3.3.4 Ứng dụng của phương pháp ADD:

Thiết kế theo thuộc tính (ADD) có thể được sử dụng trong nhiều dự án phần mềm, bao gồm:

- Hệ thống quan trọng đối với nhiệm vụ: ADD ban đầu được phát triển cho các ứng dụng phần mềm quan trọng đối với nhiệm vụ như hệ thống kiểm soát không lưu và hệ thống quản lý năng lượng.

- Hệ thống nhúng: ADD là một phương pháp phổ biến để thiết kế các hệ thống nhúng, chẳng hạn như thiết bị di động, hệ thống tự động hóa công nghiệp và xe tự hành.

- Hệ thống quản lý dữ liệu: ADD có thể được sử dụng để thiết kế các hệ thống quản lý dữ liệu như cơ sở dữ liệu phân tán, hệ thống quản lý lưu trữ và hệ thống quản lý dữ liệu lớn.

- Hệ thống an ninh: ADD có thể được sử dụng để thiết kế các hệ thống an ninh như hệ thống phát hiện xâm nhập, hệ thống quản lý truy cập và hệ thống bảo vệ dữ liệu.

- Hệ thống thời gian thực: ADD là một phương pháp phổ biến để thiết kế các hệ thống thời gian thực, chẳng hạn như hệ thống kiểm soát quy trình, hệ thống quản lý giao thông và hệ thống quản lý hoạt động.

- Hệ thống phân tán: ADD có thể được sử dụng để thiết kế các hệ thống phân tán như hệ thống quản lý đám mây, hệ thống quản lý biên và hệ thống Internet vạn vật (IoT).

- Ngoài ra, ADD có thể được áp dụng cho các loại dự án phần mềm khác, tùy thuộc vào nhu cầu.

3.3.5 So sánh ADD với các phương pháp thiết kế kiến trúc khác:

Thiết kế hướng chức năng (Function-Driven Design):

- Phương pháp này tập trung vào việc đáp ứng các yêu cầu chức năng của hệ thống.

- ADD khác biệt ở chỗ nó đặt trọng tâm vào các thuộc tính chất lượng, coi chúng là yếu tố then chốt trong quá trình thiết kế.

- Trong khi thiết kế hướng chức năng có thể bỏ qua hoặc xem nhẹ các yêu cầu phi chức năng, ADD đảm bảo rằng chúng được xem xét một cách toàn diện.

Thiết kế hướng đối tượng (Object-Oriented Design):

- Phương pháp này tập trung vào việc tổ chức hệ thống thành các đối tượng tương tác.

- ADD có thể kết hợp với thiết kế hướng đối tượng, nhưng nó cung cấp một khuôn khổ bổ sung để xem xét các thuộc tính chất lượng ở cấp độ kiến trúc.

- ADD giúp đảm bảo rằng các quyết định thiết kế đối tượng phù hợp với các mục tiêu chất lượng của hệ thống.

Rational Unified Process (RUP):

- RUP là một quy trình phát triển phần mềm lặp đi lặp lại.

- ADD có thể được xem như một phần mở rộng của RUP, đặc biệt là trong giai đoạn thiết kế kiến trúc.

- ADD bổ sung các bước cụ thể để xem xét các thuộc tính chất lượng, giúp RUP tạo ra các kiến trúc mạnh mẽ và linh hoạt hơn.

Các điểm khác biệt chính:

- Tập trung vào thuộc tính chất lượng: ADD đặt các thuộc tính chất lượng làm yếu tố chính trong quá trình thiết kế, trong khi các phương pháp khác có thể tập trung vào chức năng hoặc cấu trúc.

- Phân rã đệ quy: ADD sử dụng một quá trình phân rã đệ quy để phân tích và thiết kế kiến trúc, cho phép xem xét các thuộc tính chất lượng ở nhiều cấp độ khác nhau.

- Sử dụng kịch bản chất lượng: ADD sử dụng các kịch bản chất lượng cụ thể để hướng dẫn quá trình thiết kế, giúp đảm bảo rằng các yêu cầu phi chức năng được đáp ứng.

3.4 Bài tập áp dụng

Chúng tôi trình bày phương pháp ADD bằng cách sử dụng nó để thiết kế kiến trúc dòng sản phẩm cho bộ mở cửa nhà để xe trong hệ thống thông tin gia đình. Bộ mở có trách

nhiệm nâng và hạ cửa thông qua công tắc, điều khiển từ xa hoặc hệ thống thông tin gia đình. Cũng có thể chẩn đoán sự cố với bộ mở từ bên trong hệ thống thông tin gia đình.

Đầu vào mẫu

Đầu vào của ADD là một tập hợp các yêu cầu. ADD coi các yêu cầu chức năng (thường được thể hiện dưới dạng các trường hợp sử dụng) và các ràng buộc là đầu vào, giống như các phương pháp thiết kế khác. Tuy nhiên, trong ADD, chúng tôi khác với các phương pháp đó trong cách xử lý các yêu cầu chất lượng. ADD yêu cầu các yêu cầu chất lượng phải được thể hiện dưới dạng một tập hợp các kịch bản chất lượng cụ thể của hệ thống. Các kịch bản chung được thảo luận trong Chương 4 đóng vai trò là đầu vào cho quy trình yêu cầu và cung cấp danh sách kiểm tra để sử dụng trong quá trình phát triển các kịch bản cụ thể của hệ thống. Các kịch bản cụ thể của hệ thống phải được định nghĩa chi tiết cần thiết cho ứng dụng. Trong các ví dụ của chúng tôi, chúng tôi bỏ qua một số phần của một kịch bản đầy đủ vì các phần này không đóng góp vào quy trình thiết kế.

Đối với ví dụ về cửa gara của chúng tôi, các tình huống chất lượng bao gồm:

- Thiết bị và điều khiển để mở và đóng cửa khác nhau đối với các sản phẩm khác nhau trong dòng sản phẩm, như đã đề cập. Chúng có thể bao gồm các điều khiển từ bên trong hệ thống thông tin gia đình. Kiến trúc sản phẩm cho một bộ điều khiển cụ thể phải được suy ra trực tiếp từ kiến trúc dòng sản phẩm.
- Bộ xử lý được sử dụng trong các sản phẩm khác nhau sẽ khác nhau. Kiến trúc sản phẩm cho từng bộ xử lý cụ thể phải được suy ra trực tiếp từ kiến trúc dòng sản phẩm.
- Nếu cửa gara phát hiện chướng ngại vật (người hoặc vật) trong quá trình hạ xuống, cửa phải dừng lại (hoặc mở lại) trong vòng 0,1 giây.
- Bộ mở cửa nhà để xe phải có thể được truy cập để chẩn đoán và quản lý từ bên trong hệ thống thông tin gia đình bằng cách sử dụng giao thức chẩn đoán dành riêng cho sản phẩm. Có thể trực tiếp tạo ra một kiến trúc phản ánh giao thức này.

Bài tập áp dụng:

Bài tập 1: Hệ thống quản lý thư viện trực tuyến

- Yêu cầu: Thiết kế kiến trúc cho một hệ thống quản lý thư viện trực tuyến, cho phép người dùng tìm kiếm, mượn và trả sách. Hệ thống cần đáp ứng các thuộc tính chất lượng sau:

- Hiệu suất: Thời gian phản hồi nhanh cho các yêu cầu tìm kiếm và mượn sách.
- Khả năng mở rộng: Hệ thống có thể xử lý số lượng lớn người dùng và sách.
- Bảo mật: Bảo vệ thông tin người dùng và dữ liệu sách.

Bài tập 2: Hệ thống thương mại điện tử

• **Yêu cầu:** Thiết kế kiến trúc cho một hệ thống thương mại điện tử, cho phép người dùng mua sắm trực tuyến. Hệ thống cần đáp ứng các thuộc tính chất lượng sau:

- Khả năng sẵn sàng: Hệ thống luôn hoạt động và sẵn sàng phục vụ người dùng.
- Khả năng bảo trì: Hệ thống dễ dàng được bảo trì và cập nhật.
- Khả năng kiểm thử: Hệ thống dễ dàng được kiểm thử để đảm bảo chất lượng.

Bài tập 3: Hệ thống quản lý bệnh viện

• **Yêu cầu:** Thiết kế kiến trúc cho một hệ thống quản lý bệnh viện, cho phép quản lý thông tin bệnh nhân, lịch hẹn, hồ sơ bệnh án, v.v. Hệ thống cần đáp ứng các thuộc tính chất lượng sau:

- Bảo mật: Bảo vệ thông tin nhạy cảm của bệnh nhân.
- Độ tin cậy: Hệ thống hoạt động chính xác và đáng tin cậy.
- Hiệu suất: Hệ thống xử lý các yêu cầu nhanh chóng và hiệu quả.

Bài tập 4: Hệ thống mạng xã hội

• **Yêu cầu:** Thiết kế kiến trúc cho một hệ thống mạng xã hội, cho phép người dùng kết nối, chia sẻ thông tin và tương tác với nhau. Hệ thống cần đáp ứng các thuộc tính chất lượng sau:

- Khả năng mở rộng: Hệ thống có thể xử lý số lượng lớn người dùng và dữ liệu.
- Hiệu suất: Hệ thống phản hồi nhanh chóng và mượt mà.
- Khả năng sẵn sàng: Hệ thống luôn hoạt động và sẵn sàng phục vụ người dùng.

Bài tập 5: Hệ thống quản lý giao thông thông minh

• **Yêu cầu:** Thiết kế kiến trúc cho một hệ thống quản lý giao thông thông minh, cho phép giám sát và điều khiển giao thông trong một thành phố. Hệ thống cần đáp ứng các thuộc tính chất lượng sau:

- **Hiệu suất thực thi:** Hệ thống phải phản hồi nhanh chóng với các thay đổi của lưu lượng giao thông.
- **Độ tin cậy:** Hệ thống phải hoạt động liên tục và chính xác để tránh gây ra ùn tắc giao thông.
- **Khả năng mở rộng:** Hệ thống phải có khả năng mở rộng để quản lý giao thông ở các khu vực rộng lớn hơn.
- **Bảo mật:** Hệ thống phải bảo vệ dữ liệu giao thông và ngăn chặn các truy cập trái phép.

Câu hỏi và bài tập

Câu hỏi lý thuyết:

1. Định nghĩa phương pháp ADD và giải thích tại sao nó lại tập trung vào các thuộc tính chất lượng?
2. Trình bày các bước chính trong quy trình thiết kế kiến trúc ADD và giải thích mục đích của từng bước.
3. So sánh phương pháp ADD với các phương pháp thiết kế kiến trúc khác như thiết kế hướng chức năng và thiết kế hướng đối tượng. Nêu rõ những điểm khác biệt chính.
4. Giải thích vai trò của các kịch bản thuộc tính chất lượng trong phương pháp ADD và cho ví dụ minh họa.
5. Nêu những ưu điểm và hạn chế của phương pháp ADD trong thiết kế kiến trúc phần mềm.

Bài tập vận dụng:

1. **Tình huống:** Bạn được yêu cầu thiết kế kiến trúc cho một hệ thống ngân hàng trực tuyến. Hệ thống này cần đáp ứng các yêu cầu về bảo mật, hiệu suất và khả năng mở rộng.

- **Yêu cầu:** Áp dụng phương pháp ADD để xác định các thành phần kiến trúc chính và mối quan hệ giữa chúng.
- **Gợi ý:** Xác định các kịch bản thuộc tính chất lượng (ví dụ: bảo mật thông tin khách hàng, thời gian phản hồi nhanh), chọn các chiến thuật và mẫu kiến trúc phù hợp.

2. **Tình huống:** Một công ty phần mềm muốn phát triển một hệ thống quản lý kho hàng. Hệ thống này cần có khả năng tích hợp với các hệ thống khác và dễ dàng bảo trì.

- **Yêu cầu:** Sử dụng phương pháp ADD để thiết kế kiến trúc cho hệ thống này.
- **Gợi ý:** Tập trung vào các thuộc tính chất lượng như khả năng tương tác và khả năng bảo trì.

3. **Tình huống:** Thiết kế kiến trúc cho một hệ thống điều khiển giao thông thông minh. Hệ thống này cần có khả năng xử lý dữ liệu thời gian thực và đảm bảo an toàn giao thông.

- **Yêu cầu:** Áp dụng phương pháp ADD để xác định các thành phần và tương tác cần thiết.

- **Gợi ý:** Chú ý đến các yêu cầu về hiệu suất và độ tin cậy.

4. **Tình huống:** Một công ty thương mại điện tử muốn xây dựng một hệ thống đề xuất sản phẩm cho khách hàng. Hệ thống này cần có khả năng cá nhân hóa đề xuất và đáp ứng nhanh chóng.

- **Yêu cầu:** Sử dụng phương pháp ADD để thiết kế kiến trúc cho hệ thống này.

- **Gợi ý:** Tập trung vào các thuộc tính chất lượng như khả năng mở rộng và hiệu suất.

5. **Tình huống:** Thiết kế kiến trúc cho một hệ thống quản lý hồ sơ bệnh án điện tử. Hệ thống này cần đảm bảo tính bảo mật và khả năng truy cập nhanh chóng.

- **Yêu cầu:** Áp dụng phương pháp ADD để xác định các thành phần và tương tác cần thiết.

- **Gợi ý:** Chú ý đến các yêu cầu về bảo mật và hiệu suất

Câu hỏi trắc nghiệm:

1. Phong cách kiến trúc nào sau đây phân chia hệ thống thành các lớp riêng biệt, mỗi lớp có một trách nhiệm cụ thể?

- a) Kiến trúc microservices.
- b) Kiến trúc hướng sự kiện.
- c) Kiến trúc lớp.
- d) Kiến trúc máy chủ-khách hàng.

2. Nguyên tắc thiết kế kiến trúc nào sau đây khuyến khích việc chia nhỏ hệ thống thành các thành phần độc lập và có thể tái sử dụng?

- a) Nguyên tắc tách biệt mối quan tâm.

- b) Nguyên tắc trừu tượng hóa.
- c) Nguyên tắc tái sử dụng.
- d) Tất cả các đáp án trên.

3. Mô hình xem nào sau đây mô tả cách các thành phần phần mềm được triển khai trên phần cứng?

- a) Mô hình xem logic.
- b) Mô hình xem triển khai.
- c) Mô hình xem xử lý.
- d) Mô hình xem phát triển.

4. Trong phương pháp ADD, bước đầu tiên là gì?

- a) Chọn phong cách kiến trúc.
- b) Xác định các thuộc tính chất lượng.
- c) Thiết kế các thành phần phần mềm.
- d) Triển khai hệ thống.

5. Chiến thuật kiến trúc được sử dụng để làm gì trong phương pháp ADD?

- a) Xác định các yêu cầu chức năng.
- b) Đáp ứng các thuộc tính chất lượng.
- c) Quản lý dự án phần mềm.
- d) Kiểm thử phần mềm.

6. Thuộc tính chất lượng nào sau đây KHÔNG phải là một yếu tố quan trọng trong phương pháp ADD?

- a) Hiệu suất.
- b) Khả năng mở rộng.
- c) Tính thẩm mỹ của giao diện người dùng.
- d) Bảo mật.

7. Phương pháp ADD tập trung vào điều gì trong quá trình thiết kế kiến trúc?

- a) Các yêu cầu chức năng.
- b) Các yêu cầu phi chức năng.
- c) Công nghệ lập trình.

d) Quản lý dự án.

8. Một công ty phát triển phần mềm đang xây dựng một hệ thống quản lý tài chính. Họ ưu tiên thuộc tính chất lượng về "bảo mật" lên hàng đầu. Theo phương pháp ADD, bước tiếp theo họ nên làm gì?

- a) Chọn phong cách kiến trúc phù hợp.
- b) Xác định các chiến thuật kiến trúc liên quan đến bảo mật.
- c) Phân tích các yêu cầu chức năng của hệ thống.
- d) Vẽ sơ đồ kiến trúc tổng quan.

9. Một hệ thống thương mại điện tử cần đáp ứng thuộc tính chất lượng "khả năng mở rộng" để xử lý lượng truy cập tăng đột biến trong các dịp lễ. Chiến thuật kiến trúc nào sau đây phù hợp nhất?

- a) Sử dụng cơ sở dữ liệu tập trung.
- b) Triển khai hệ thống theo kiến trúc microservices.
- c) Giảm thiểu việc sử dụng bộ nhớ đệm.
- d) Tăng cường bảo mật bằng cách hạn chế truy cập.

10. Một hệ thống quản lý bệnh viện cần đảm bảo "độ tin cậy" cao để tránh sai sót trong việc quản lý hồ sơ bệnh nhân. Bước nào sau đây KHÔNG thuộc quy trình ADD để đạt được mục tiêu này?

- a) Xác định các kịch bản lỗi tiềm ẩn.
- b) Lựa chọn công nghệ lập trình mới nhất.
- c) Áp dụng chiến thuật dự phòng (redundancy).
- d) Thiết kế cơ chế khôi phục dữ liệu.

CHƯƠNG 4: THIẾT KẾ MẪU

4.1. Giới thiệu về mẫu thiết kế

Trong công nghệ phần mềm, một mẫu thiết kế (tiếng Anh: design pattern) là một giải pháp tổng thể cho các vấn đề chung trong thiết kế phần mềm. Một mẫu thiết kế không phải là một thiết kế hoàn thiện để mà có thể được chuyển đổi trực tiếp thành mã; nó chỉ là một mô tả hay là sườn (template) mô tả cách giải quyết một vấn đề mà có thể được dùng trong nhiều tình huống khác nhau. Các mẫu thiết kế hướng đối tượng thường cho thấy mối quan hệ và sự tương tác giữa các lớp hay các đối tượng, mà không cần chỉ rõ các lớp hay đối tượng của từng ứng dụng cụ thể. Các giải thuật không được xem là các mẫu thiết kế, vì chúng giải quyết các vấn đề về tính toán hơn là các vấn đề về thiết kế.

Lịch sử:

Các mẫu xuất phát từ một ý niệm kiến trúc đưa ra bởi Christopher Alexander. Vào năm 1987, Kent Beck và Ward Cunningham bắt đầu thử nghiệm ý tưởng áp dụng các mẫu vào lập trình và đưa ra các kết quả của chúng tại hội thảo OOPSLA vào năm đó. Vào các năm tiếp theo, Beck, Cunningham và những người khác vẫn tiếp tục với công việc này.

Các mẫu thiết kế đã trở nên phổ biến trong khoa học máy tính sau khi cuốn sách Design Patterns: Elements of Reusable Object-Oriented Software được ấn hành vào năm 1994 bởi các tác giả được biết đến với tên "Gang of Four"(Gamma et al) gọi tắt là "GoF". Vào cùng năm đó, cuộc hội thảo đầu tiên về Các ngôn ngữ mẫu cho các chương trình đã được tổ chức và vào năm sau, Kho dự trữ các mẫu Portland (Portland Pattern Repository) đã được thiết lập để lưu trữ văn bản về các mẫu thiết kế.

Cách sử dụng:

Các mẫu thiết kế có thể giúp tăng tốc quá trình phát triển phần mềm bằng cách cung cấp các mẫu hình (paradigms) phát triển đã được chứng thực và kiểm chứng. Để thiết kế phần mềm hiệu quả đòi hỏi phải xem xét các yếu tố mà chỉ trở nên rõ ràng sau khi hiện thực. Xác định được chúng, thông qua các mẫu thiết kế, chúng ta sẽ thoát khỏi chúng vì chúng có thể dẫn đến những rắc rối lớn và cải tiến khả năng dễ đọc của mã cho người viết mã và các nhà kiến trúc sẽ cảm thấy quen thuộc với các mẫu.

Thông thường, mọi người chỉ biết cách áp dụng một số kỹ thuật thiết kế phần mềm nào đó vào một vài vấn đề cụ thể nào đó. Những kỹ thuật này khó áp dụng mở rộng cho các vấn đề khác. Các mẫu thiết kế cung cấp các giải pháp chung, được viết tài liệu dưới một định dạng mà không gắn liền với một vấn đề cụ thể nào cả.

Các mẫu cho phép các nhà phát triển giao tiếp với nhau dùng các tên dễ hiểu, được dùng rộng rãi để đặt cho các tương tác của phần mềm. Các mẫu thiết kế chung có thể được cải tiến qua thời gian, để trở nên ổn định hơn là thiết kế tùy biến (ad-hoc designs).

Mẫu thiết kế và Framework

Mẫu thiết kế không tồn tại dưới dạng thành phần phần mềm mà phải được cài đặt tại thời điểm sử dụng, không nên nhầm lẫn giữa framework với mẫu thiết kế. Framework là một dạng phần mềm gồm các thành phần phối hợp hoạt động với nhau để cung cấp một kiến trúc cho một miền ứng dụng nên nó có thể cài đặt nhiều mẫu thiết kế. **Bảng 7.1** sau đây nêu lên một số đặc điểm thể hiện sự khác biệt này [10]:

Bảng 7-1: Mẫu thiết kế và Framework

Mẫu thiết kế	Framework
Mẫu thiết kế là giải pháp lặp đi lặp lại đối với vấn đề nảy sinh trong vòng đời của ứng dụng phần mềm	Framework là nhóm thành phần cộng hợp với nhau để cung cấp một kiến trúc có thể sử dụng lại cho những ứng dụng
Mục đích là: ☐ Giúp cải tiến chất lượng phần mềm (sử dụng lại, bảo trì, mở rộng...) ☐ Giảm thời gian phát triển	Mục đích là: ☐ Giúp cải tiến chất lượng phần mềm (sử dụng lại, bảo trì, mở rộng...) ☐ Giảm thời gian phát triển
Mẫu là một cấu trúc logic	Framework là một dạng phần mềm
Mô tả mẫu thường độc lập với ngôn ngữ lập trình và chi tiết cài đặt	Framework đặc trưng cho cài đặt
Mẫu là tổng quát hơn và có thể sử dụng trong nhiều ứng dụng	Framework cung cấp chức năng cho miền cụ thể

Mẫu thiết kế không tồn tại dưới dạng thành phần phần mềm. Nó cần được cài đặt mỗi khi sử dụng	Framework không phải là một ứng dụng đầy đủ, các ứng dụng có thể xây dựng bằng cách kế thừa từ các thành phần có sẵn
Mẫu thiết kế là cách thiết kế tốt và được sử dụng để thiết kế các framework	Mẫu thiết kế có thể sử dụng trong thiết kế và cài đặt các framework nghĩa là framework có thể chứa nhiều mẫu thiết kế

Định dạng mẫu thiết kế:

Vì các mẫu thiết kế được sử dụng bởi nhiều người nên chúng cần được thể hiện theo một định dạng được chấp nhận rộng rãi. Nghĩa là, kiểu định dạng phải dễ hiểu và tổng quát để có thể cài đặt với nhiều ngôn ngữ lập trình khác nhau. Hơn nữa vấn đề quan trọng là cần phải thể hiện được cách sử dụng các mẫu này trong thực tế ngữ cảnh tương ứng. Sau đây là một số đặc trưng của các mẫu thiết kế:

- Tên mẫu: Tên viết ngắn gọn của mẫu thường là một hoặc hai từ. Ví dụ, tên mẫu phải chọn làm sao thể hiện được mục đích của mô hình và được sử dụng như một tên duy nhất đặc trưng cho mẫu đó.
- Phân loại: Các mẫu được phân chia thành nhiều loại khác nhau. Nhóm mẫu tạo dựng (creational) là các mẫu tập trung vào cách tạo các đối tượng; nhóm mẫu cấu trúc (structural) tập trung vào việc sắp xếp các đối tượng với nhau thành một cấu trúc tổng thể lớn hơn; nhóm mẫu hành vi (behavioral) liên quan đến hành vi, cộng tác giữa các đối tượng để thực hiện một mục tiêu nào đó.
- Mục đích: Một mô tả chung ngắn gọn (một hoặc hai câu) về mô hình. Ví dụ, mẫu Observer mô tả phụ thuộc 1-nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái thì tất cả phụ thuộc của nó có thể nhận biết và cập nhật tự động.
- Định danh: Các mẫu đều có định danh của mẫu đó.
- Động lực: Xác định vấn đề mà mẫu thiết kế này có thể được sử dụng để giải quyết.
- Khả năng áp dụng: Lĩnh vực mà mẫu thiết kế này có thể được sử dụng.
- Cấu trúc: Những biểu đồ lớp và thành phần với UML nhằm thể hiện cách hoạt động mẫu thiết kế này.

- Các bên tham gia: Xác định các đối tượng liên quan hoặc những công việc mà đối tượng đó được yêu cầu thực hiện.
- Cộng tác: Xác định sự cộng tác của các bên tham gia.
- Kết quả: Lợi ích và hạn chế của mẫu thiết kế này (thêm những gợi ý để giải quyết các vấn đề về hạn chế).
- Cài đặt: Những gợi ý, chỉ dẫn về cách sử dụng mẫu này bao gồm cả những thủ thuật, các cách ứng dụng hiệu quả và những điều nên tránh.
- Mã nguồn mẫu: Có ví dụ triển khai đầy đủ cho mẫu này được viết dưới dạng ngôn ngữ C++, java, C#...
- Các áp dụng: Ví dụ về các lĩnh vực mà mẫu này đã được áp dụng trong thực tế.
- Các mẫu liên quan: Các mẫu thiết kế tương tự như mẫu này hoặc cũng mang lại hiệu quả tương tự khi sử dụng như mẫu hiện thời.

Mẫu thiết kế hỗ trợ developers như thế nào

- Tăng tốc độ phát triển phần mềm: Trong quá trình phát triển ứng dụng, việc sử dụng design patterns cho phép các developers có một công cụ để giải quyết các vấn đề thông dụng trong thiết kế phần mềm. Kể cả khi không gặp phải những vấn đề đó, việc nắm vững design patterns cũng rất hữu ích khi nó giúp developers thấy được cách giải quyết vấn đề thông qua việc ứng dụng các nguyên tắc thiết kế hướng đối tượng.
- Code tường minh, dễ dàng team work: Mẫu thiết kế định nghĩa một ngôn ngữ chung mà developers có thể dùng để giao tiếp hiệu quả hơn. Chẳng hạn, chỉ cần nêu tên một pattern, tất cả những đồng đội trong nhóm đều sẽ hình dung được cấu trúc, ý tưởng đằng sau đó và cách ứng dụng nó. Tối ưu thời gian phát triển ý tưởng hơn vì hạn chế được thời gian giải thích.
- Tái sử dụng code: Về phía dự án phần mềm, design patterns giúp developers có thể dễ dàng tái sử dụng và mở rộng code với các giải pháp tối ưu đã được kiểm chứng để giải quyết những vấn đề thông thường trong phát triển phần mềm. Do đó, khi gặp vấn đề trong xây dựng phần mềm, developers có thể coi design patterns như là kim chỉ nam để giúp mình giải quyết những vấn đề thay vì tự đi tìm kiếm giải pháp (mà có thể chưa có sự kiểm chứng kỹ càng).

- Hạn chế lỗi tiềm ẩn, dễ dàng nâng cấp: việc sử dụng lại các design patterns còn giúp developers tránh các vấn đề tiềm ẩn có thể sẽ gây ra những lỗi lớn trong tương lai, cùng với đó, điều đó cũng giúp dự án dễ nâng cấp và bảo trì trong tương lai hơn.

4.2. Một số mẫu thiết kế cơ bản

Từ khi các mẫu thiết kế được đưa ra bởi nhóm GoF, đã có nhiều mẫu được đề xuất thêm. Hệ thống các mẫu này có thể nói là khá đầy đủ cho những người mới làm quen với mẫu thiết kế để giải quyết các vấn đề của thiết kế và phát triển phần mềm hiện nay. Hệ thống 23 mẫu thiết kế của GoF được chia thành ba nhóm: Nhóm tạo dựng, nhóm cấu trúc và nhóm hành vi.

4.2.1 Nhóm mẫu tạo dựng:

"Mẫu thiết kế tạo dựng" (creational design pattern) là một nhóm các mẫu thiết kế phần mềm liên quan đến việc tạo ra các đối tượng. Chúng giúp cho việc tạo đối tượng trở nên linh hoạt và dễ quản lý hơn, đặc biệt trong các hệ thống phức tạp.

- Mục đích của mẫu nhóm này nhằm giải quyết công việc thường xuyên là tạo ra các đối tượng.

- Các mẫu sẽ tạo ra một cơ chế đơn giản, thống nhất khi tạo các thể hiện của đối tượng.

- Cho phép đóng gói các chi tiết về các lớp nào được khởi tạo và cách các thể hiện này được tạo ra

- Nó khuyến khích sử dụng interface nhằm giảm độ liên kết.

- Nhóm này gồm các mẫu thiết kế sau đây: Abstract Factory, Factory Method, Builder, Prototype, và Singleton:

1. Singleton: Đảm bảo rằng một lớp chỉ có một thể hiện duy nhất và cung cấp một điểm truy cập toàn cục đến thể hiện đó. Ví dụ: Quản lý kết nối cơ sở dữ liệu, cấu hình ứng dụng.
2. Factory Method: Định nghĩa một giao diện để tạo ra các đối tượng, nhưng để cho các lớp con quyết định lớp nào sẽ được khởi tạo. Ví dụ: Tạo các loại hình khác nhau (hình tròn, hình vuông, hình tam giác) từ một giao diện chung.

3. Abstract Factory: Cung cấp một giao diện để tạo ra các họ đối tượng liên quan hoặc phụ thuộc lẫn nhau mà không chỉ định các lớp cụ thể của chúng. Ví dụ: Tạo giao diện người dùng cho các nền tảng khác nhau (Windows, macOS, Linux).
4. Builder: Tách biệt việc xây dựng một đối tượng phức tạp khỏi biểu diễn của nó, cho phép tạo ra các biểu diễn khác nhau bằng cách sử dụng cùng một quy trình xây dựng. Ví dụ: Xây dựng một chiếc xe hơi với các tùy chọn khác nhau (động cơ, màu sắc, nội thất).
5. Prototype: Tạo ra các đối tượng mới bằng cách sao chép một đối tượng hiện có (prototype). Ví dụ: Tạo các bản sao của một tài liệu mà không cần phải tạo lại từ đầu.

Ví dụ: Giả sử bạn đang phát triển một trò chơi điện tử. Bạn có thể sử dụng mẫu thiết kế Factory Method để tạo ra các loại kẻ thù khác nhau (ví dụ: quái vật, boss) từ một lớp cha chung. Hoặc, bạn có thể sử dụng mẫu thiết kế Builder để tạo ra các loại vũ khí khác nhau với các thuộc tính khác nhau (ví dụ: sức mạnh, tốc độ, tầm bắn).

Lợi ích của việc sử dụng mẫu thiết kế tạo dựng

- Tính linh hoạt: Dễ dàng thay đổi cách thức tạo đối tượng mà không cần sửa đổi nhiều code.
- Tính tái sử dụng: Có thể sử dụng lại các mẫu thiết kế cho nhiều tình huống khác nhau.
- Tính bảo trì: Code dễ hiểu và dễ bảo trì hơn.

4.2.2 Nhóm mẫu cấu trúc

"Mẫu thiết kế cấu trúc" (structural design pattern) là một nhóm các mẫu thiết kế phần mềm tập trung vào việc tổ chức và cấu trúc các lớp và đối tượng để tạo thành các cấu trúc lớn hơn, đồng thời đơn giản hóa thiết kế.

- Nhóm này chủ yếu giải quyết vấn đề một đối tượng ủy thác trách nhiệm cho những đối tượng khác. Điều này dẫn đến kiến trúc phân tầng của các thành phần với độ kết nối thấp.

- Tạo điều kiện giao tiếp giữa các đối tượng khi một đối tượng không thể truy nhập đối tượng khác theo cách thông thường hay khi một đối tượng không thể sử dụng được do giao diện không tương thích.

- Cung cấp các cách để cấu trúc một đối tượng với quan hệ hợp thành được tạo ra đầy đủ. Nhóm này liên quan tới các quan hệ cấu trúc giữa các thể hiện, bằng cách sử dụng các quan hệ kế thừa, liên kết, phụ thuộc. Để xem thông tin về mẫu này phải dựa vào biểu đồ lớp của mẫu.

- Nhóm này gồm các mẫu sau đây: Adapter, Bridge, Composite, Decorator, Façade, Proxy và Flyweight.

6. Adapter: Cho phép các lớp có giao diện không tương thích làm việc cùng nhau bằng cách chuyển đổi giao diện của một lớp thành giao diện mà lớp khác mong đợi. Ví dụ: Sử dụng một thư viện bên ngoài với giao diện khác với giao diện của ứng dụng của bạn.
7. Bridge: Tách biệt phần trừu tượng của một lớp khỏi phần triển khai của nó, cho phép cả hai thay đổi độc lập với nhau. Ví dụ: Xây dựng một ứng dụng có thể chạy trên nhiều nền tảng khác nhau (Windows, macOS, Linux).
8. Composite: Cho phép bạn xử lý các đối tượng riêng lẻ và các nhóm đối tượng một cách thống nhất. Ví dụ: Xây dựng một cấu trúc cây thư mục, nơi mỗi thư mục có thể chứa các tệp tin và các thư mục con khác.
9. Decorator: Cho phép bạn thêm các trách nhiệm mới vào một đối tượng một cách động mà không thay đổi cấu trúc của nó. Ví dụ: Thêm các tính năng bổ sung vào một đối tượng mà không cần phải tạo ra một lớp con mới.
10. Facade: Cung cấp một giao diện đơn giản cho một hệ thống phức tạp. Ví dụ: Một lớp facade có thể che giấu sự phức tạp của nhiều lớp bên trong và cung cấp một giao diện dễ sử dụng hơn cho người dùng.
11. Flyweight: Sử dụng chung các đối tượng nhỏ để giảm thiểu việc sử dụng bộ nhớ. Ví dụ: Lưu trữ các ký tự trong một văn bản, nơi mỗi ký tự có thể được sử dụng lại nhiều lần.

12. Proxy: Cung cấp một đối tượng đại diện cho một đối tượng khác để kiểm soát truy cập vào đối tượng đó. Ví dụ: Một proxy có thể được sử dụng để trì hoãn việc tải một hình ảnh lớn cho đến khi người dùng thực sự muốn xem nó.

Ví dụ: Giả sử bạn đang phát triển một ứng dụng chỉnh sửa ảnh. Bạn có thể sử dụng mẫu thiết kế Decorator để thêm các hiệu ứng khác nhau vào một bức ảnh (ví dụ: làm mờ, tăng độ tương phản) mà không cần phải thay đổi lớp ảnh gốc. Hoặc, bạn có thể sử dụng mẫu thiết kế Composite để tạo ra một cấu trúc phức tạp của các lớp layer, nơi mỗi layer có thể chứa nhiều đối tượng hình ảnh khác nhau.

Lợi ích của việc sử dụng mẫu thiết kế cấu trúc

- Tính linh hoạt: Dễ dàng thay đổi cấu trúc của hệ thống mà không cần sửa đổi nhiều code.
- Tính tái sử dụng: Có thể sử dụng lại các mẫu thiết kế cho nhiều tình huống khác nhau.
- Tính bảo trì: Code dễ hiểu và dễ bảo trì hơn.

4.2.3 Nhóm mẫu hành vi

"Mẫu thiết kế hành vi" (behavioral design pattern) là một nhóm các mẫu thiết kế phần mềm tập trung vào việc xác định và quản lý cách các đối tượng tương tác và giao tiếp với nhau trong một hệ thống. Chúng giúp cho việc thiết kế các hệ thống linh hoạt, dễ thay đổi và mở rộng hơn.

- Nhóm mẫu thiết kế này liên quan đến các quan hệ gán trách nhiệm để cung cấp các chức năng giữa các đối tượng trong hệ thống.
- Mô tả cơ chế giao tiếp giữa các đối tượng và xác định cơ chế chọn các thuật toán khác nhau bởi các đối tượng khác nhau tại thời gian chạy.
- Đối với các mẫu thuộc nhóm này ta có thể dựa vào biểu đồ giao tiếp và biểu đồ tuần tự để giải thích cách chuyển giao của các chức năng.
- Nhóm này gồm có một số mẫu sau đây: Interpreter, Template Method, Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy và Visitor.

Ngoài ra, hai mẫu interface và abstract được xem là thuộc nhóm mẫu cơ bản và đã được sử dụng rộng rãi trong thiết kế các ngôn ngữ lập trình như Java. Các mẫu cơ bản này sẽ sinh viên đã quen thuộc khi học lập trình Java nên không đề cập trong tài liệu này và dành như bài tập cho bạn đọc.

1. Chain of Responsibility: Cho phép bạn chuyển yêu cầu dọc theo một chuỗi các đối tượng xử lý nó. Mỗi đối tượng sẽ quyết định xem nó có xử lý yêu cầu hay chuyển nó cho đối tượng tiếp theo trong chuỗi. Ví dụ: Xử lý các sự kiện trong một ứng dụng GUI, nơi mỗi đối tượng có thể xử lý một loại sự kiện cụ thể.
2. Command: Biến một yêu cầu thành một đối tượng, cho phép bạn tham số hóa các yêu cầu, xếp hàng hoặc ghi lại chúng, và hỗ trợ các hoạt động hủy bỏ. Ví dụ: Thực hiện các thao tác chỉnh sửa văn bản (sao chép, dán, cắt) trong một ứng dụng soạn thảo văn bản.
3. Interpreter: Định nghĩa một biểu diễn ngữ pháp cho một ngôn ngữ đơn giản và cung cấp một bộ thông dịch để diễn giải các câu trong ngôn ngữ đó. Ví dụ: Xây dựng một bộ thông dịch cho một ngôn ngữ truy vấn cơ sở dữ liệu đơn giản.
4. Iterator: Cung cấp một cách để truy cập các phần tử của một đối tượng tập hợp một cách tuần tự mà không cần tiết lộ biểu diễn bên trong của nó. Ví dụ: Duyệt qua các phần tử của một danh sách hoặc một cây.
5. Mediator: Định nghĩa một đối tượng trung gian để điều phối sự tương tác giữa các đối tượng khác. Mediator giúp giảm sự phụ thuộc lẫn nhau giữa các đối tượng và làm cho chúng dễ dàng thay đổi hơn. Ví dụ: Quản lý giao tiếp giữa các thành phần trong một ứng dụng chat.
6. Memento: Lưu trữ trạng thái bên trong của một đối tượng tại một thời điểm nào đó và cho phép bạn khôi phục đối tượng về trạng thái đó sau này. Ví dụ: Thực hiện chức năng "undo" trong một ứng dụng chỉnh sửa ảnh.
7. Observer: Định nghĩa một sự phụ thuộc một-nhiều giữa các đối tượng, sao cho khi trạng thái của một đối tượng thay đổi, tất cả các đối tượng phụ thuộc vào nó sẽ được thông báo và cập nhật tự động. Ví dụ: Cập nhật giao diện người dùng khi dữ liệu của ứng dụng thay đổi.

8. State: Cho phép một đối tượng thay đổi hành vi của nó dựa trên trạng thái bên trong của nó. Ví dụ: Một đối tượng có thể có các trạng thái khác nhau (ví dụ: "đang hoạt động", "tạm dừng", "đã dừng") và hành vi của nó sẽ khác nhau tùy thuộc vào trạng thái hiện tại.
9. Strategy: Định nghĩa một họ các thuật toán và cho phép bạn chọn một thuật toán cụ thể tại thời điểm chạy. Ví dụ: Sắp xếp một danh sách các đối tượng bằng các thuật toán khác nhau (ví dụ: sắp xếp theo tên, sắp xếp theo ngày).
10. Template Method: Định nghĩa bộ khung của một thuật toán trong một phương thức, nhưng cho phép các lớp con ghi đè các bước cụ thể của thuật toán đó. Ví dụ: Xử lý một đơn đặt hàng, nơi các bước như "xác nhận đơn hàng", "thanh toán", "giao hàng" có thể được ghi đè bởi các lớp con để xử lý các loại đơn hàng khác nhau.
11. Visitor: Cho phép bạn thêm các thao tác mới vào một tập hợp các đối tượng mà không cần thay đổi cấu trúc của chúng. Ví dụ: Tính toán tổng giá trị của một giỏ hàng, nơi các "visitor" khác nhau có thể thực hiện các loại tính toán khác nhau (ví dụ: tính thuế, tính phí vận chuyển).

Ví dụ

Giả sử bạn đang phát triển một trò chơi điện tử. Bạn có thể sử dụng mẫu thiết kế State để quản lý trạng thái của nhân vật (ví dụ: "đang di chuyển", "đang tấn công", "đang nhảy"). Hoặc, bạn có thể sử dụng mẫu thiết kế Observer để thông báo cho các đối tượng khác trong trò chơi (ví dụ: kẻ thù, vật phẩm) khi nhân vật thay đổi trạng thái.

Lợi ích của việc sử dụng mẫu thiết kế hành vi

- Tính linh hoạt: Dễ dàng thay đổi hành vi của hệ thống mà không cần sửa đổi nhiều code.
- Tính tái sử dụng: Có thể sử dụng lại các mẫu thiết kế cho nhiều tình huống khác nhau.
- Tính bảo trì: Code dễ hiểu và dễ bảo trì hơn.

Khi nào sử dụng mẫu thiết kế?

Mẫu thiết kế là mô tả tổng quát của một giải pháp đối với một vấn đề được “lặp đi lặp lại” trong các dự án phần mềm. Như vậy, một mẫu thiết kế có thể được xem như là một

“khuôn mẫu” có sẵn được áp dụng để giải quyết một vấn đề trong các tình huống khác nhau.

Mẫu thiết kế được hiểu theo nghĩa tái sử dụng ý tưởng hơn là tái sử dụng mã lệnh. Mẫu thiết kế cho phép các nhà thiết kế ngồi lại cùng nhau để giải quyết một vấn đề nào đó mà không phải mất nhiều thời gian tranh cãi. Nhiều dự án phần mềm thất bại là do những người phát triển không có được sự hiểu biết chung về các vấn đề trong kiến trúc phần mềm. Do đó, mẫu thiết kế có thể sử dụng như một nguồn cung cấp những thuật ngữ và khái niệm để nhanh chóng hình dung ra “bức tranh” của giải pháp trong quá trình thiết kế. Hơn nữa, nếu mẫu thiết kế được sử dụng một cách hiệu quả thì dễ dàng đưa ra được kiến trúc hệ thống ban đầu và việc bảo trì phần mềm sau này cũng được tiến hành thuận lợi hơn.

Mẫu thiết kế hỗ trợ tái sử dụng kiến trúc và mô hình thiết kế phần mềm quy mô lớn. Cần phân biệt mẫu thiết kế với framework. Framework hỗ trợ tái sử dụng mô hình thiết kế và mã nguồn ở mức chi tiết hơn. Trong khi đó, mẫu thiết kế được vận dụng ở mức tổng quát hơn, giúp các nhà phát triển hình dung và nhận biết các cấu trúc tĩnh và động cũng như quan hệ tương tác giữa các giải pháp trong quá trình thiết kế ứng dụng đối với một lĩnh vực riêng biệt.

Mẫu thiết kế là đa tương thích nghĩa là nó không phụ thuộc vào ngôn ngữ lập trình, công nghệ hoặc các nền tảng lớn như J2EE của Sun hay Microsoft .NET. Tiềm năng ứng dụng của mẫu thiết kế là rất lớn. Các kiến trúc dựa trên mẫu thiết kế đã được sử dụng khá nhiều trong các phần mềm mã nguồn mở, trong nền tảng J2EE hoặc .NET... Trong các dạng ứng dụng này, có thể dễ dàng nhận ra một số tên lớp chứa các mẫu thiết kế như Factory, Proxy, Adapter...

Sử dụng mẫu thiết kế như thế nào?

Các mẫu thiết kế cần phải thay đổi để phù hợp với tình huống sử dụng cụ thể. Sau đây là một số lý do vì sao cần sự điều chỉnh như vậy:

- Sự khác biệt về ngôn ngữ biên dịch: Các ngôn ngữ lập trình kể cả các ngôn ngữ hướng đối tượng đều có những khác biệt nhau như cú pháp, biến dịch...Do đó, các mẫu thiết kế cần phải hiệu chỉnh sao cho phù hợp với ngôn ngữ đang sử dụng cho dự án.

- Sự khác biệt về quan điểm: Thông thường, có thể có nhiều giải pháp đối với mỗi vấn đề đặt ra trong khi thiết kế. Do đó, một mẫu thiết kế được đưa ra bởi một chuyên gia hoặc một nhóm các chuyên gia không có nghĩa là nó đã là hoàn hảo.

- Sự khác biệt về kiểu: Thiết kế và kiểu mã hóa có thể nảy sinh từ nhiều nguồn như kinh nghiệm, nguyên tắc viết mã của doanh nghiệp, các yêu cầu của một thư viện hoặc framework. Nếu một mô hình không phù hợp với kiểu mà dự án sử dụng thì chúng ta phải điều chỉnh nó.

- Vấn đề khác nhau: Mỗi một mẫu thiết kế phải được thể hiện hết sức tổng quát nên một số mô hình lớp trình bày trong mẫu có thể sẽ không được sử dụng. Ví dụ, trong mẫu thiết kế Observer (xem Chương 10), các lớp ConcreteObserver và ConcreteSubject có thể không được sử dụng vì không phù hợp với tình huống thiết kế của chúng ta.

- Thành phần, kế thừa và đa kế thừa: Các ngôn ngữ hướng đối tượng và các nhà phát triển thường có sự khác biệt trong cách sử dụng kế thừa và thành phần. Ví dụ, trong C++ có đa kế thừa nhưng Java thì lại không có.

- Đơn giản hóa: Một mẫu thiết kế thường là tổng quát hóa cho nhiều tính năng và do đó có thể sẽ phức tạp hơn thực tế thiết kế của yêu cầu dự án đề ra. Vì vậy, chúng ta có thể làm đơn giản mẫu đó nhằm mang lại hiệu quả cao hơn.

4.3. Bài tập thiết kế phần mềm trong bài toán thực

4.3.1 Các bước thiết kế phần mềm trong bài toán thực tế

Để thiết kế một phần mềm hoàn chỉnh, chúng ta cần trải qua nhiều giai đoạn khác nhau. Dưới đây là các bước cơ bản trong quy trình thiết kế phần mềm:

1. Xác định yêu cầu bài toán:

- Bước đầu tiên và quan trọng nhất là phải hiểu rõ bài toán mà chúng ta muốn giải quyết.

- Chúng ta cần xác định rõ mục tiêu của phần mềm, đối tượng sử dụng, các chức năng cần thiết và các ràng buộc (nếu có).

- Để làm được điều này, chúng ta cần thu thập thông tin từ nhiều nguồn khác nhau, như người dùng, khách hàng, hoặc các chuyên gia trong lĩnh vực liên quan.

2. Phân tích yêu cầu:

- Sau khi đã xác định được yêu cầu bài toán, chúng ta cần phân tích chúng một cách chi tiết để hiểu rõ hơn về chúng.
- Chúng ta cần xác định các chức năng chính của phần mềm, các dữ liệu cần thiết, và các mối quan hệ giữa chúng.
- Bước này giúp chúng ta có cái nhìn tổng quan về phần mềm và xác định được các thành phần cần thiết.

3. Thiết kế kiến trúc phần mềm:

- Dựa trên các yêu cầu đã phân tích, chúng ta tiến hành thiết kế kiến trúc tổng thể của phần mềm.
- Chúng ta cần quyết định lựa chọn mô hình kiến trúc phù hợp (ví dụ: client-server, peer-to-peer), các công nghệ và nền tảng phát triển, và các thành phần chính của phần mềm.
- Bản thiết kế kiến trúc sẽ là cơ sở cho các giai đoạn tiếp theo.

4. Thiết kế chi tiết:

- Sau khi đã có kiến trúc tổng thể, chúng ta tiến hành thiết kế chi tiết từng thành phần của phần mềm.
- Chúng ta cần xác định cấu trúc dữ liệu, các thuật toán, giao diện người dùng, và các module chức năng.
- Bước này đòi hỏi sự hiểu biết sâu sắc về các nguyên tắc thiết kế phần mềm và kỹ năng lập trình.

5. Lập trình và kiểm thử:

- Dựa trên bản thiết kế chi tiết, chúng ta tiến hành lập trình các thành phần của phần mềm.
- Trong quá trình lập trình, chúng ta cần tuân thủ các nguyên tắc và tiêu chuẩn lập trình để đảm bảo chất lượng của phần mềm.
- Sau khi hoàn thành lập trình, chúng ta tiến hành kiểm thử để phát hiện và sửa lỗi.

6. Triển khai và bảo trì:

- Sau khi phần mềm đã được kiểm thử và đảm bảo chất lượng, chúng ta tiến hành triển khai cho người dùng.
- Chúng ta cần cài đặt phần mềm, cấu hình hệ thống, và đào tạo người dùng.
- Trong quá trình sử dụng, chúng ta cần bảo trì và cập nhật phần mềm để đảm bảo hoạt động ổn định và đáp ứng các yêu cầu mới.

Ví dụ về bài tập thiết kế phần mềm

Để giúp bạn hiểu rõ hơn về quy trình thiết kế phần mềm, chúng ta sẽ cùng nhau xem xét một số ví dụ cụ thể:

4.3.2 Bài toán 1: Thiết kế hệ thống phần mềm quản lý thư viện trực tuyến dùng phương pháp sử dụng nhóm mẫu thiết kế tạo dựng (Factory Method).

1. Xác định yêu cầu

Trước khi bắt tay vào thiết kế, chúng ta cần xác định rõ các yêu cầu của hệ thống. Điều này sẽ giúp chúng ta đi đúng hướng và đảm bảo hệ thống đáp ứng được nhu cầu sử dụng.

- **Mục tiêu:***
 - Xây dựng một hệ thống quản lý thư viện trực tuyến hiện đại, tiện lợi và dễ sử dụng.
 - Giúp người dùng dễ dàng tìm kiếm, mượn, trả sách và quản lý thông tin cá nhân.
 - Hỗ trợ nhân viên thư viện quản lý sách, người dùng và các hoạt động của thư viện một cách hiệu quả.
- **Yêu cầu:***
 - **Chức năng:***
 - **Người dùng:***
 - Tìm kiếm sách theo tên, tác giả, chủ đề, ISBN,...
 - Xem thông tin chi tiết về sách (mô tả, hình ảnh, đánh giá,...).
 - Đăng ký tài khoản và đăng nhập.

- Mượn và trả sách trực tuyến.
- Quản lý thông tin cá nhân (hồ sơ, lịch sử mượn trả,...).
- Đánh giá và nhận xét về sách.
- Nhận thông báo về sách sắp đến hạn trả.
- **Nhân viên thư viện:***
 - Quản lý danh mục sách (thêm, sửa, xóa sách).
 - Quản lý thông tin người dùng.
 - Theo dõi tình trạng mượn trả sách.
 - Thống kê báo cáo về sách, người dùng, hoạt động của thư viện.
 - Quản lý các khoản phí (nếu có).
- **Giao diện:***
 - Thân thiện, dễ sử dụng, trực quan.
 - Thiết kế đẹp mắt, chuyên nghiệp.
 - Tối ưu hóa cho các thiết bị di động.
- **Công nghệ:***
 - Sử dụng nền tảng web (ví dụ: HTML, CSS, JavaScript, PHP, Python,...).
 - Cơ sở dữ liệu: MySQL, PostgreSQL,...
 - Bảo mật: Đảm bảo an toàn thông tin người dùng và dữ liệu của thư viện.

2. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu là nền tảng của hệ thống, nơi lưu trữ tất cả thông tin về sách, người dùng và các hoạt động của thư viện. Chúng ta cần thiết kế cơ sở dữ liệu một cách cẩn thận để đảm bảo tính toàn vẹn và hiệu quả của dữ liệu.

- **Bảng dữ liệu:***
 - **Sách:***
 - id (khóa chính)

- ten_sach
- tac_gia
- nha_xuat_ban
- nam_xuat_ban
- isbn
- mo_ta
- hinh_anh
- the_loai (khóa ngoại)
- so_luong
- **Người dùng:***
 - id (khóa chính)
 - ho_ten
 - email
 - mat_khau
 - dia_chi
 - so_dien_thoai
 - vai_tro (ví dụ: người dùng, nhân viên)
- **Thẻ loại:***
 - id (khóa chính)
 - ten_the_loai
- **Mượn trả:***
 - id (khóa chính)
 - nguoi_dung_id (khóa ngoại)
 - sach_id (khóa ngoại)
 - ngay_muon
 - ngay_tra
 - trang_thai (ví dụ: đang mượn, đã trả)

3. Thiết kế hệ thống

Chúng ta sẽ sử dụng mẫu thiết kế Factory Method để tạo ra các loại sản phẩm khác nhau (ví dụ: sách, tạp chí, báo cáo khoa học,...).

- **Lớp trừu tượng Product:***
 - Định nghĩa giao diện chung cho tất cả các loại sản phẩm.
 - Bao gồm các phương thức trừu tượng như `displayInfo()`, `getPrice()`,...
- **Các lớp sản phẩm cụ thể:***
 - Kế thừa từ lớp Product.
 - Hiện thực các phương thức trừu tượng của lớp Product.
 - Ví dụ: Book, Magazine, ScientificReport,...
- **Lớp ProductFactory:***
 - Chứa phương thức `createProduct()` để tạo ra các đối tượng sản phẩm dựa trên kiểu dữ liệu đầu vào.
 - Ví dụ: Nếu kiểu dữ liệu là "book", phương thức `createProduct()` sẽ tạo ra một đối tượng Book.

4. Ví dụ code (C++)

1. Lớp trừu tượng Product

C++

```
#include <iostream>
```

```
#include <string>
```

```
class Product {
```

```
public:
```

```
    // Phương thức thuần ảo để hiển thị thông tin sản phẩm
```

```
    virtual void displayInfo() = 0;
```

```
    // Phương thức ảo để lấy tên sản phẩm (có thể ghi đè nếu cần)
```

```
    virtual std::string getName() {
```

```
        return name;
```



```
}
```

protected:

```
std::string name; // Tên sản phẩm
```

```
double price; // Giá sản phẩm
```

```
Product(std::string name, double price) : name(name), price(price) {}
```

```
};
```

2. Các lớp sản phẩm cụ thể

C++

```
class Book : public Product {
```

```
public:
```

```
Book(std::string name, double price, std::string author)
```

```
: Product(name, price), author(author) {}
```

```
void displayInfo() override {
```

```
std::cout << "Sách: " << name << " - Tác giả: " << author
```

```
<< " - Giá: " << price << std::endl;
```

```
}
```

```
private:
```

```
std::string author; // Tác giả
```

```
};
```

```
class Magazine : public Product {
```

```
public:
```

```
Magazine(std::string name, double price, std::string publisher)
```

```
: Product(name, price), publisher(publisher) {}
```

```

void displayInfo() override {
    std::cout << "Tập chí: " << name << " - Nhà xuất bản: " << publisher
        << " - Giá: " << price << std::endl;
}

private:
    std::string publisher; // Nhà xuất bản
};

class ScientificReport : public Product {
public:
    ScientificReport(std::string name, double price, std::string author)
        : Product(name, price), author(author) {}

    void displayInfo() override {
        std::cout << "Báo cáo khoa học: " << name << " - Tác giả: " << author
            << " - Giá: " << price << std::endl;
    }

private:
    std::string author; // Tác giả
};

```

3. Lớp ProductFactory

```

C++
#include <vector>
#include <stdexcept> // Để xử lý ngoại lệ

class ProductFactory {
public:

```

```

static Product* createProduct(std::string type, std::vector<std::string> args) {
    if (type == "book") {
        // Kiểm tra số lượng tham số
        if (args.size() != 3) {
            throw std::invalid_argument("Invalid arguments for Book creation.");
        }
        return new Book(args[0], std::stod(args[1]), args[2]);
    } else if (type == "magazine") {
        if (args.size() != 3) {
            throw std::invalid_argument("Invalid arguments for Magazine
creation.");
        }
        return new Magazine(args[0], std::stod(args[1]), args[2]);
    } else if (type == "scientific_report") {
        if (args.size() != 3) {
            throw std::invalid_argument("Invalid arguments for ScientificReport
creation.");
        }
        return new ScientificReport(args[0], std::stod(args[1]), args[2]);
    } else {
        return nullptr;
    }
}
};

```

4. Sử dụng

C++

```

int main() {
    try {

```

```

        std::vector<std::string> bookArgs = {"Harry Potter", "250000", "J.K.
Rowling"};
        Product* book = ProductFactory::createProduct("book", bookArgs);
        if (book) {
            book->displayInfo();
            delete book; // Nhớ giải phóng bộ nhớ
        }

        std::vector<std::string> magazineArgs = {"Forbes", "150000", "Forbes
Media"};
        Product* magazine = ProductFactory::createProduct("magazine",
magazineArgs);
        if (magazine) {
            magazine->displayInfo();
            delete magazine;
        }

        std::vector<std::string> reportArgs = {"The Theory of Relativity", "500000",
"Albert Einstein"};
        Product* report = ProductFactory::createProduct("scientific_report",
reportArgs);
        if (report) {
            report->displayInfo();
            delete report;
        }
    } catch (const std::invalid_argument& e) {
        std::cerr << "Lỗi: " << e.what() << std::endl;
    }
}

```

```
    return 0;
}
```

Giải thích

- Lớp Product là lớp trừu tượng, định nghĩa giao diện chung cho các loại sản phẩm.
- Các lớp Book, Magazine, ScientificReport kế thừa từ lớp Product và hiện thực phương thức displayInfo().
- Lớp ProductFactory sử dụng mẫu Factory Method để tạo ra các đối tượng sản phẩm dựa trên kiểu dữ liệu đầu vào.
- Trong hàm main(), chúng ta tạo ra các đối tượng sản phẩm khác nhau thông qua ProductFactory và gọi phương thức displayInfo() để hiển thị thông tin chi tiết.
- Chúng ta sử dụng try-catch để xử lý các ngoại lệ có thể xảy ra trong quá trình tạo sản phẩm.

5. Phát triển và kiểm thử

Sau khi đã có thiết kế chi tiết, chúng ta tiến hành phát triển và kiểm thử hệ thống.

Quá trình này bao gồm các giai đoạn:

- **Lập trình:** Xây dựng các chức năng của hệ thống dựa trên thiết kế.
- **Kiểm thử:** Kiểm tra và sửa lỗi để đảm bảo hệ thống hoạt động ổn định và đúng chức năng.

6. Triển khai và bảo trì

Sau khi hệ thống đã được kiểm thử và hoàn thiện, chúng ta tiến hành triển khai và bảo trì.

- **Triển khai:** Đưa hệ thống lên môi trường trực tuyến để người dùng có thể truy cập và sử dụng.
- **Bảo trì:** Sửa lỗi, cập nhật và nâng cấp hệ thống để đáp ứng các yêu cầu mới và đảm bảo hoạt động ổn định.

4.3.3 Bài toán 2: Thiết kế hệ thống phần mềm đặt xe trực tuyến dùng phương pháp sử dụng nhóm mẫu thiết kế cấu trúc

1. Xác định yêu cầu

Trước khi bắt tay vào thiết kế, chúng ta cần xác định rõ các yêu cầu của hệ thống. Điều này sẽ giúp chúng ta đi đúng hướng và đảm bảo hệ thống đáp ứng được nhu cầu sử dụng.

- **Mục tiêu:***
 - Xây dựng một hệ thống đặt xe trực tuyến hiện đại, tiện lợi và dễ sử dụng.
 - Giúp người dùng dễ dàng đặt xe, theo dõi chuyến đi và thanh toán.
 - Hỗ trợ tài xế quản lý thông tin cá nhân, nhận và thực hiện các chuyến đi.
 - Cung cấp cho quản trị viên các công cụ để quản lý người dùng, tài xế và các hoạt động của hệ thống.
- **Yêu cầu:***
 - **Chức năng:***
 - **Người dùng:***
 - Đăng ký tài khoản và đăng nhập.
 - Tìm kiếm xe dựa trên vị trí, loại xe, thời gian,...
 - Đặt xe và theo dõi chuyến đi.
 - Thanh toán trực tuyến hoặc tiền mặt.
 - Đánh giá và nhận xét về tài xế.
 - **Tài xế:***
 - Đăng ký tài khoản và đăng nhập.
 - Quản lý thông tin cá nhân và xe.
 - Nhận và chấp nhận các yêu cầu đặt xe.
 - Cập nhật trạng thái chuyến đi.
 - Xem lịch sử chuyến đi và thu nhập.

- **Quản trị viên:***
 - Quản lý người dùng và tài xế.
 - Quản lý các loại xe và giá cước.
 - Xem báo cáo và thống kê về hoạt động của hệ thống.
- **Giao diện:***
 - Thân thiện, dễ sử dụng, trực quan.
 - Thiết kế đẹp mắt, chuyên nghiệp.
 - Tối ưu hóa cho các thiết bị di động.
- **Công nghệ:***
 - Sử dụng nền tảng web và ứng dụng di động (ví dụ: HTML, CSS, JavaScript, React Native,...).
 - Cơ sở dữ liệu: MySQL, PostgreSQL,...
 - Bảo mật: Đảm bảo an toàn thông tin người dùng và dữ liệu của hệ thống.

2. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu là nền tảng của hệ thống, nơi lưu trữ tất cả thông tin về người dùng, tài xế, xe và các hoạt động của hệ thống. Chúng ta cần thiết kế cơ sở dữ liệu một cách cẩn thận để đảm bảo tính toàn vẹn và hiệu quả của dữ liệu.

- **Bảng dữ liệu:***
 - **Người dùng:***
 - id (khóa chính)
 - ho_ten
 - email
 - mat_khau
 - dien_thoai
 - dia_chi
 - **Tài xế:***
 - id (khóa chính)

- ho_ten
- email
- mat_khau
- dien_thoai
- dia_chi
- bien_so_xe
- loai_xe
- **Xe:***
 - id (khóa chính)
 - bien_so_xe
 - loai_xe
 - suc_chua
- **Chuyến đi:***
 - id (khóa chính)
 - nguoi_dung_id (khóa ngoại)
 - tai_xe_id (khóa ngoại)
 - dia_diem_don
 - dia_diem_tra
 - thoi_gian_don
 - thoi_gian_tra
 - trang_thai (ví dụ: đang chờ, đang đi, đã hoàn thành)
 - gia_cuoc

3. Thiết kế hệ thống

Chúng ta sẽ sử dụng một số mẫu thiết kế cấu trúc để xây dựng hệ thống đặt xe trực tuyến.

- **Facade:***
 - Cung cấp một giao diện đơn giản cho các chức năng phức tạp của hệ thống, giúp người dùng và tài xế dễ dàng tương tác với hệ thống hơn.

- Ví dụ: Lớp BookingFacade sẽ cung cấp các phương thức như findDrivers(), bookRide(), cancelRide(),...
- **Adapter:***
 - Cho phép các lớp có giao diện không tương thích làm việc cùng nhau.
 - Ví dụ: Sử dụng một cổng thanh toán bên thứ ba với giao diện khác với giao diện của hệ thống.
- **Composite:***
 - Cho phép xử lý các đối tượng riêng lẻ và các nhóm đối tượng một cách thống nhất.
 - Ví dụ: Một chuyến đi có thể bao gồm nhiều điểm dừng chân.
- **Decorator:***
 - Cho phép thêm các tính năng mới vào một đối tượng một cách động mà không thay đổi cấu trúc của nó.
 - Ví dụ: Thêm các tùy chọn cho chuyến đi như "xe có điều hòa", "xe 7 chỗ",...

4. Ví dụ code (C++)

C++

```
#include <iostream>
```

```
#include <string>
```

```
class Vehicle {
```

```
public:
```

```
    // Phương thức thuần ảo để lấy sức chứa của xe
```

```
    virtual int getCapacity() = 0;
```

```
    // Phương thức ảo để lấy biển số xe (có thể ghi đè nếu cần)
```

```
    virtual std::string getLicensePlate() {
```

```
        return licensePlate;
```

```
}
```

protected:

```
std::string licensePlate; // Biển số xe
```

```
Vehicle(std::string licensePlate) : licensePlate(licensePlate) {}  
};
```

2. Các lớp xe cụ thể

C++

```
class Car : public Vehicle {
```

public:

```
Car(std::string licensePlate) : Vehicle(licensePlate) {}
```

```
int getCapacity() override {
```

```
    return 4;
```

```
}
```

```
};
```

```
class SUV : public Vehicle {
```

public:

```
SUV(std::string licensePlate) : Vehicle(licensePlate) {}
```

```
int getCapacity() override {
```

```
    return 7;
```

```
}
```

```
};
```

3. Lớp Facade cho việc đặt xe

C++

```
#include <vector>
```

```

#include <stdexcept>

class BookingFacade {
public:
    // Tìm kiếm tài xế dựa trên vị trí và loại xe
    std::vector<std::string> findDrivers(std::string location, std::string vehicleType)
{
    // (logic tìm kiếm tài xế sẽ được hiện thực sau)
    // Ví dụ: trả về danh sách các tài xế có loại xe phù hợp
    if (vehicleType == "car") {
        return {"driver1", "driver2"};
    } else if (vehicleType == "suv") {
        return {"driver3"};
    } else {
        return {};
    }
}

    // Đặt xe và tạo chuyến đi
    void bookRide(std::string user, std::string driver, std::string pickupLocation,
std::string dropoffLocation) {
        // (logic đặt xe và tạo chuyến đi sẽ được hiện thực sau)
        std::cout << user << " đã đặt xe " << driver << " từ " << pickupLocation << "
đến " << dropoffLocation << std::endl;
    }

    // Hủy chuyến đi
    void cancelRide(int rideId) {
        // (logic hủy chuyến đi sẽ được hiện thực sau)

```

```

        std::cout << "Đã hủy chuyến đi " << rideId << std::endl;
    }
};

4. Ví dụ sử dụng

C++

int main() {
    BookingFacade facade;

    std::vector<std::string> carDrivers = facade.findDrivers("Hanoi", "car");
    for (const auto& driver : carDrivers) {
        std::cout << "Tài xế: " << driver << std::endl;
    }

    facade.bookRide("user1", "driver1", "Hanoi", "Noi Bai");
    facade.cancelRide(123);

    return 0;
}

```

Giải thích

- Lớp Vehicle là lớp trừu tượng, định nghĩa giao diện chung cho các loại xe.
- Các lớp Car, SUV kế thừa từ lớp Vehicle và hiện thực phương thức getCapacity().
- Lớp BookingFacade cung cấp một giao diện đơn giản cho các chức năng đặt xe, tìm kiếm tài xế và hủy chuyến đi.
- Trong hàm main(), chúng ta sử dụng BookingFacade để thực hiện các thao tác đặt xe.

5. Phát triển và kiểm thử

Sau khi đã có thiết kế chi tiết, chúng ta tiến hành phát triển và kiểm thử hệ thống. Quá trình này bao gồm các giai đoạn:

- **Lập trình:** Xây dựng các chức năng của hệ thống dựa trên thiết kế.
- **Kiểm thử:** Kiểm tra và sửa lỗi để đảm bảo hệ thống hoạt động ổn định và đúng chức năng.

6. Triển khai và bảo trì

Sau khi hệ thống đã được kiểm thử và hoàn thiện, chúng ta tiến hành triển khai và bảo trì.

- **Triển khai:** Đưa hệ thống lên môi trường trực tuyến để người dùng có thể truy cập và sử dụng.
- **Bảo trì:** Sửa lỗi, cập nhật và nâng cấp hệ thống để đáp ứng các yêu cầu mới và đảm bảo hoạt động ổn định.

4.3.4 Bài toán 3: Thiết kế hệ thống phần mềm game đua xe dùng phương pháp sử dụng nhóm mẫu thiết kế hành vi.

1. Xác định yêu cầu

Trước khi bắt tay vào thiết kế, chúng ta cần xác định rõ các yêu cầu của hệ thống. Điều này sẽ giúp chúng ta đi đúng hướng và đảm bảo hệ thống đáp ứng được nhu cầu sử dụng.

- **Mục tiêu:***
 - Xây dựng một game đua xe hấp dẫn, có tính tương tác cao và dễ dàng mở rộng.
 - Tạo ra một môi trường đua xe đa dạng với nhiều loại xe, đường đua và chế độ chơi khác nhau.
 - Đảm bảo hiệu suất và tính ổn định của game.
- **Yêu cầu:***
 - **Chức năng:***
 - **Người chơi:***
 - Điều khiển xe (tăng tốc, phanh, rẽ trái/phải).

- Chọn xe và đường đua.
- Tham gia các chế độ chơi khác nhau (đua đơn, đua nhiều người chơi, đua tính thời gian,...).
- Xem thông tin về xe, đường đua và kết quả cuộc đua.
- **Hệ thống:***
 - Quản lý các loại xe và đường đua.
 - Xử lý va chạm giữa các xe.
 - Tính toán điểm số và thứ hạng.
 - Hiển thị đồ họa và âm thanh.
- **Giao diện:***
 - Trực quan, dễ sử dụng.
 - Hiển thị đầy đủ thông tin về cuộc đua.
- **Công nghệ:***
 - Sử dụng ngôn ngữ lập trình phù hợp (ví dụ: C++, Unity, Unreal Engine,...).
 - Tối ưu hóa hiệu suất để đảm bảo game chạy mượt mà.

2. Thiết kế hệ thống

Chúng ta sẽ sử dụng một số mẫu thiết kế hành vi để xây dựng hệ thống game đua xe.

- **State:***
 - Quản lý trạng thái của xe (ví dụ: đang chạy, đang phanh, đang va chạm,...).
 - Mỗi trạng thái sẽ có các hành động và sự kiện tương ứng.
- **Strategy:***
 - Cho phép người chơi lựa chọn các chiến lược đua khác nhau (ví dụ: tấn công, phòng thủ, cân bằng).
 - Mỗi chiến lược sẽ ảnh hưởng đến cách xe di chuyển và tương tác với các xe khác.

- **Observer:***
 - Thông báo cho các đối tượng khác khi có sự kiện xảy ra (ví dụ: xe va chạm, xe về đích,...).
 - Các đối tượng quan sát có thể là giao diện người dùng, hệ thống tính điểm,...
- **Command:***
 - Đóng gói các hành động của người chơi thành các đối tượng (ví dụ: tăng tốc, phanh, rẽ trái/phải).
 - Điều này giúp chúng ta dễ dàng quản lý và thực hiện các hành động, cũng như hỗ trợ các tính năng như replay.

3. Ví dụ code (C++)

C++

```
#include <iostream>
```

```
#include <vector>
```

```
// Lớp trừu tượng cho trạng thái của xe
```

```
class CarState {
```

```
public:
```

```
    virtual void handleInput(Car* car, Input input) = 0;
```

```
    virtual void update(Car* car) = 0;
```

```
};
```

```
// Các trạng thái cụ thể của xe
```

```
class RacingState : public CarState {
```

```
public:
```

```
    void handleInput(Car* car, Input input) override {
```

```
        if (input == ACCELERATE) {
```

```
            car->accelerate();
```

```
        } else if (input == BRAKE) {
```

```

        car->brake();
    } else if (input == TURN_LEFT) {
        car->turnLeft();
    } else if (input == TURN_RIGHT) {
        car->turnRight();
    }
}

void update(Car* car) override {
    // Cập nhật vị trí và vận tốc của xe khi đang đua
}

};

// Lớp xe
class Car {
public:
    void setState(CarState* state) {
        this->state = state;
    }

    void handleInput(Input input) {
        state->handleInput(this, input);
    }

    void update() {
        state->update(this);
    }

    void accelerate() {

```



```

        // Thực hiện hành động tăng tốc
    }

    void brake() {
        // Thực hiện hành động phanh
    }

    void turnLeft() {
        // Thực hiện hành động rẽ trái
    }

    void turnRight() {
        // Thực hiện hành động rẽ phải
    }

private:
    CarState* state;
    // Các thuộc tính khác của xe (vị trí, vận tốc,...)
};

int main() {
    Car car;
    CarState* racingState = new RacingState();
    car.setState(racingState);

    // Xử lý input của người chơi
    Input input = ACCELERATE;
    car.handleInput(input);
}

```

```

// Cập nhật trạng thái của xe
car.update();

delete racingState;
return 0;
}

```

4. Phát triển và kiểm thử

Sau khi đã có thiết kế chi tiết, chúng ta tiến hành phát triển và kiểm thử hệ thống. Quá trình này bao gồm các giai đoạn:

- **Lập trình:** Xây dựng các chức năng của hệ thống dựa trên thiết kế.
- **Kiểm thử:** Kiểm tra và sửa lỗi để đảm bảo hệ thống hoạt động ổn định và đúng chức năng.

5. Triển khai và bảo trì

Sau khi hệ thống đã được kiểm thử và hoàn thiện, chúng ta tiến hành triển khai và bảo trì.

- **Triển khai:** Đưa hệ thống lên nền tảng phù hợp (ví dụ: PC, mobile,...).
- **Bảo trì:** Sửa lỗi, cập nhật và nâng cấp hệ thống để đáp ứng các yêu cầu mới và đảm bảo hoạt động ổn định.

Bài tập áp dụng

1. Hệ thống quản lý thư viện trực tuyến

- **Mô tả:** Thiết kế một hệ thống quản lý thư viện trực tuyến cho phép người dùng tìm kiếm, mượn, trả sách và quản lý thông tin cá nhân.
- **Yêu cầu:***
 - Người dùng có thể tìm kiếm sách theo tên, tác giả, chủ đề, ISBN,...
 - Người dùng có thể xem thông tin chi tiết về sách (mô tả, hình ảnh, đánh giá,...).
 - Người dùng có thể đăng ký tài khoản và đăng nhập.
 - Người dùng có thể mượn và trả sách trực tuyến.
 - Người dùng có thể quản lý thông tin cá nhân (hồ sơ, lịch sử mượn trả,...).
 - Nhân viên thư viện có thể quản lý danh mục sách, thông tin người dùng và các hoạt động của thư viện.

2. Ứng dụng đặt đồ ăn trực tuyến

- **Mô tả:** Thiết kế một ứng dụng đặt đồ ăn trực tuyến cho phép người dùng tìm kiếm nhà hàng, xem thực đơn, đặt món và thanh toán.
- **Yêu cầu:***
 - Người dùng có thể tìm kiếm nhà hàng theo vị trí, loại món ăn, giá cả,...
 - Người dùng có thể xem thực đơn và thông tin chi tiết về món ăn.
 - Người dùng có thể thêm món ăn vào giỏ hàng và đặt hàng.
 - Người dùng có thể chọn nhiều hình thức thanh toán (ví dụ: thẻ tín dụng, tiền mặt).
 - Người dùng có thể theo dõi trạng thái đơn hàng.
 - Nhà hàng có thể quản lý thực đơn và nhận đơn hàng.

- Ứng dụng cần có tính năng đánh giá và nhận xét về nhà hàng và món ăn.

3. Hệ thống quản lý bán hàng

- **Mô tả:** Thiết kế một hệ thống quản lý bán hàng cho phép người dùng mua sắm trực tuyến, quản lý đơn hàng và thanh toán.
- **Yêu cầu:***
 - Người dùng có thể xem danh sách sản phẩm theo danh mục.
 - Người dùng có thể tìm kiếm sản phẩm theo tên, mã sản phẩm,...
 - Người dùng có thể thêm sản phẩm vào giỏ hàng và đặt hàng.
 - Người dùng có thể chọn nhiều hình thức thanh toán và giao hàng.
 - Người dùng có thể theo dõi trạng thái đơn hàng.
 - Nhân viên bán hàng có thể quản lý sản phẩm, đơn hàng và khách hàng.
 - Hệ thống cần có tính năng báo cáo doanh thu và thống kê bán hàng.

4. Ứng dụng mạng xã hội

- **Mô tả:** Thiết kế một ứng dụng mạng xã hội cho phép người dùng kết nối, chia sẻ thông tin và tương tác với nhau.
- **Yêu cầu:***
 - Người dùng có thể tạo tài khoản và đăng nhập.
 - Người dùng có thể kết bạn và theo dõi người dùng khác.
 - Người dùng có thể đăng bài viết, hình ảnh, video và chia sẻ với bạn bè.
 - Người dùng có thể bình luận và thích bài viết của người khác.
 - Người dùng có thể nhắn tin riêng với người dùng khác.
 - Ứng dụng cần có tính năng thông báo về các hoạt động của bạn bè.

5. Hệ thống quản lý bệnh viện

- **Mô tả:** Thiết kế một hệ thống quản lý bệnh viện cho phép quản lý thông tin bệnh nhân, lịch hẹn, hồ sơ bệnh án và các hoạt động của bệnh viện.
- **Yêu cầu:***
 - Bệnh nhân có thể đăng ký khám bệnh trực tuyến.
 - Bệnh nhân có thể xem lịch sử khám bệnh và kết quả xét nghiệm.
 - Bác sĩ có thể quản lý lịch hẹn và hồ sơ bệnh án của bệnh nhân.
 - Nhân viên bệnh viện có thể quản lý thông tin bệnh nhân và các hoạt động của bệnh viện.
 - Hệ thống cần có tính năng bảo mật thông tin bệnh nhân.

6. Ứng dụng học tập trực tuyến

- **Mô tả:** Thiết kế một ứng dụng học tập trực tuyến cho phép người dùng học tập các khóa học, làm bài tập và kiểm tra kiến thức.
- **Yêu cầu:***
 - Người dùng có thể đăng ký tài khoản và đăng nhập.
 - Người dùng có thể xem danh sách các khóa học.
 - Người dùng có thể học các bài giảng trực tuyến.
 - Người dùng có thể làm bài tập và kiểm tra kiến thức.
 - Giáo viên có thể tạo và quản lý các khóa học.
 - Ứng dụng cần có tính năng theo dõi tiến độ học tập của người dùng.

7. Hệ thống quản lý dự án

- **Mô tả:** Thiết kế một hệ thống quản lý dự án cho phép người dùng tạo dự án, quản lý công việc và theo dõi tiến độ.
- **Yêu cầu:***
 - Người dùng có thể tạo dự án và mời thành viên tham gia.
 - Người dùng có thể tạo công việc và giao cho thành viên.
 - Người dùng có thể theo dõi tiến độ công việc.
 - Người dùng có thể trao đổi và thảo luận về công việc.

- Hệ thống cần có tính năng báo cáo tiến độ dự án.

8. Ứng dụng quản lý tài chính cá nhân

- **Mô tả:** Thiết kế một ứng dụng quản lý tài chính cá nhân cho phép người dùng theo dõi thu nhập, chi tiêu và lập kế hoạch tài chính.
- **Yêu cầu:***
 - Người dùng có thể nhập thông tin thu nhập và chi tiêu.
 - Người dùng có thể xem báo cáo về thu nhập và chi tiêu.
 - Người dùng có thể lập kế hoạch tài chính.
 - Ứng dụng cần có tính năng nhắc nhở về các khoản chi tiêu.

9. Hệ thống quản lý kho hàng

- **Mô tả:** Thiết kế một hệ thống quản lý kho hàng cho phép quản lý thông tin hàng hóa, nhập kho, xuất kho và tồn kho.
- **Yêu cầu:***
 - Nhân viên kho có thể nhập thông tin hàng hóa.
 - Nhân viên kho có thể thực hiện nhập kho và xuất kho.
 - Hệ thống cần có tính năng thống kê tồn kho.
 - Hệ thống cần có tính năng cảnh báo khi hàng tồn kho thấp.

10. Ứng dụng dự báo thời tiết

- **Mô tả:** Thiết kế một ứng dụng dự báo thời tiết cho phép người dùng xem thông tin thời tiết hiện tại và dự báo thời tiết cho các ngày tiếp theo.
- **Yêu cầu:***
 - Người dùng có thể xem thông tin thời tiết hiện tại (nhiệt độ, độ ẩm, gió,...).
 - Người dùng có thể xem dự báo thời tiết cho các ngày tiếp theo.
 - Ứng dụng cần có tính năng cảnh báo thời tiết xấu.

11. Hệ thống quản lý khách sạn

- **Mô tả:** Thiết kế một hệ thống quản lý khách sạn cho phép quản lý thông tin khách hàng, đặt phòng, quản lý phòng, dịch vụ và các hoạt động của khách sạn.

- **Yêu cầu:***
 - Khách hàng có thể tìm kiếm phòng theo loại phòng, giá cả, tiện nghi,...
 - Khách hàng có thể đặt phòng trực tuyến.
 - Nhân viên lễ tân có thể quản lý thông tin khách hàng, đặt phòng, nhận phòng, trả phòng,...
 - Nhân viên quản lý phòng có thể quản lý tình trạng phòng.
 - Hệ thống cần có tính năng quản lý dịch vụ (ví dụ: ăn uống, spa, gym,...).
 - Hệ thống cần có tính năng báo cáo doanh thu và thống kê hoạt động.

12. Ứng dụng quản lý công việc cá nhân

- **Mô tả:** Thiết kế một ứng dụng quản lý công việc cá nhân cho phép người dùng tạo công việc, lên kế hoạch, đặt lịch hẹn và theo dõi tiến độ.
- **Yêu cầu:***
 - Người dùng có thể tạo công việc với các thông tin như tiêu đề, mô tả, thời gian bắt đầu và kết thúc, độ ưu tiên,...
 - Người dùng có thể phân loại công việc theo danh mục (ví dụ: công việc nhà, công việc ở cơ quan,...).
 - Người dùng có thể đặt lịch hẹn và nhận thông báo nhắc nhở.
 - Người dùng có thể theo dõi tiến độ công việc.
 - Ứng dụng cần có tính năng đồng bộ dữ liệu giữa các thiết bị.

13. Hệ thống quản lý bán vé máy bay

- **Mô tả:** Thiết kế một hệ thống quản lý bán vé máy bay cho phép người dùng tìm kiếm chuyến bay, đặt vé và thanh toán.
- **Yêu cầu:***
 - Người dùng có thể tìm kiếm chuyến bay theo điểm đến, thời gian, hãng hàng không,...

- Người dùng có thể xem thông tin chi tiết về chuyến bay (ví dụ: giá vé, thời gian bay, số ghế trống,...).
- Người dùng có thể chọn chỗ ngồi và đặt vé.
- Người dùng có thể thanh toán trực tuyến.
- Hệ thống cần có tính năng quản lý thông tin khách hàng và chuyến bay.

14. Ứng dụng quản lý sức khỏe

- **Mô tả:** Thiết kế một ứng dụng quản lý sức khỏe cho phép người dùng theo dõi sức khỏe, lịch sử bệnh án, lịch sử khám bệnh và nhận lời khuyên từ bác sĩ.

- **Yêu cầu:***
 - Người dùng có thể nhập thông tin về sức khỏe (ví dụ: cân nặng, chiều cao, huyết áp,...).
 - Người dùng có thể theo dõi lịch sử bệnh án và lịch sử khám bệnh.
 - Người dùng có thể đặt lịch hẹn khám bệnh trực tuyến.
 - Người dùng có thể nhận lời khuyên từ bác sĩ.
 - Ứng dụng cần có tính năng bảo mật thông tin người dùng.

15. Hệ thống quản lý bất động sản

- **Mô tả:** Thiết kế một hệ thống quản lý bất động sản cho phép quản lý thông tin bất động sản, khách hàng, giao dịch và các hoạt động của công ty bất động sản.

- **Yêu cầu:***
 - Nhân viên có thể nhập thông tin về bất động sản (ví dụ: diện tích, giá cả, vị trí,...).
 - Nhân viên có thể quản lý thông tin khách hàng.
 - Nhân viên có thể theo dõi các giao dịch mua bán bất động sản.
 - Hệ thống cần có tính năng tìm kiếm và lọc bất động sản.

- Hệ thống cần có tính năng báo cáo doanh thu và thống kê hoạt động.

16. Ứng dụng quản lý nhà hàng

- **Mô tả:** Thiết kế một ứng dụng quản lý nhà hàng cho phép quản lý thông tin bàn ăn, thực đơn, đơn đặt hàng và các hoạt động của nhà hàng.
- **Yêu cầu:***
 - Nhân viên có thể quản lý thông tin bàn ăn và đặt bàn cho khách hàng.
 - Nhân viên có thể quản lý thực đơn và cập nhật giá cả.
 - Nhân viên có thể nhận đơn đặt hàng từ khách hàng.
 - Hệ thống cần có tính năng in hóa đơn.
 - Hệ thống cần có tính năng báo cáo doanh thu và thống kê hoạt động.

17. Hệ thống quản lý đào tạo

- **Mô tả:** Thiết kế một hệ thống quản lý đào tạo cho phép quản lý thông tin học viên, khóa học, giảng viên và các hoạt động của trung tâm đào tạo.
- **Yêu cầu:***
 - Học viên có thể đăng ký khóa học trực tuyến.
 - Học viên có thể xem lịch học và kết quả học tập.
 - Giảng viên có thể quản lý thông tin khóa học và học viên.
 - Nhân viên quản lý có thể quản lý thông tin học viên, khóa học và giảng viên.
 - Hệ thống cần có tính năng báo cáo thống kê về hoạt động đào tạo.

18. Ứng dụng quản lý du lịch

- **Mô tả:** Thiết kế một ứng dụng quản lý du lịch cho phép người dùng tìm kiếm địa điểm du lịch, đặt tour, đặt phòng khách sạn và vé máy bay.
- **Yêu cầu:***
 - Người dùng có thể tìm kiếm địa điểm du lịch theo sở thích.

- Người dùng có thể xem thông tin chi tiết về địa điểm du lịch.
- Người dùng có thể đặt tour du lịch và phòng khách sạn.
- Người dùng có thể đặt vé máy bay.
- Ứng dụng cần có tính năng đánh giá và nhận xét về địa điểm du lịch.

19. Hệ thống quản lý thư viện nhạc trực tuyến

- **Mô tả:** Thiết kế một hệ thống quản lý thư viện nhạc trực tuyến cho phép người dùng nghe nhạc, tạo playlist và chia sẻ với bạn bè.
- **Yêu cầu:***
 - Người dùng có thể tìm kiếm bài hát theo tên, ca sĩ, album,...
 - Người dùng có thể nghe nhạc trực tuyến.
 - Người dùng có thể tạo playlist và chia sẻ với bạn bè.
 - Người dùng có thể đánh giá và nhận xét về bài hát.
 - Hệ thống cần có tính năng gợi ý nhạc theo sở thích của người dùng.

20. Ứng dụng quản lý video

- **Mô tả:** Thiết kế một ứng dụng quản lý video cho phép người dùng tải lên, xem, chia sẻ và quản lý video cá nhân.
- **Yêu cầu:***
 - Người dùng có thể tải lên video từ thiết bị của mình.
 - Người dùng có thể xem video trực tuyến.
 - Người dùng có thể chia sẻ video với bạn bè.
 - Người dùng có thể tạo kênh video cá nhân.
 - Ứng dụng cần có tính năng tìm kiếm và phân loại video.

Kiểm tra định kỳ số 3

Câu hỏi và bài tập cuối chương 3:

Câu hỏi trắc nghiệm:

1. Mục đích chính của các mẫu tạo dựng là gì?
 - a) Quản lý giao tiếp giữa các đối tượng.
 - b) Đơn giản hóa việc tạo ra các đối tượng.
 - c) Định nghĩa các thuật toán.
 - d) Quản lý trạng thái của các đối tượng.
2. Mẫu Singleton đảm bảo điều gì?
 - a) Luôn có nhiều hơn một thể hiện của một lớp.
 - b) Chỉ có một thể hiện của một lớp được tạo ra.
 - c) Các đối tượng được tạo ra theo một thứ tự cụ thể.
 - d) Các đối tượng có thể thay đổi trạng thái của nhau.
3. Mẫu Factory Method được sử dụng khi nào?
 - a) Khi cần tạo ra các đối tượng phức tạp.
 - b) Khi cần trì hoãn việc tạo ra các đối tượng con.
 - c) Khi cần đóng gói một tập hợp các đối tượng.
 - d) Khi cần tạo ra một họ các đối tượng liên quan.
4. Mẫu Builder giúp giải quyết vấn đề gì?
 - a) Vấn đề về hiệu suất.
 - b) Vấn đề về khả năng mở rộng.
 - c) Vấn đề về việc tạo ra các đối tượng với nhiều cấu hình khác nhau.
 - d) Vấn đề về việc quản lý trạng thái.
5. Mẫu Prototype được sử dụng để làm gì?
 - a) Tạo ra các đối tượng mới bằng cách sao chép các đối tượng hiện có.
 - b) Tạo ra các đối tượng theo một mẫu cụ thể.
 - c) Tạo ra các đối tượng với các thuộc tính ngẫu nhiên.
 - d) Tạo ra các đối tượng dựa trên dữ liệu từ một nguồn bên ngoài.

6. Một ứng dụng cần tích hợp một thư viện bên thứ ba với giao diện không tương thích. Mẫu thiết kế nào sẽ phù hợp nhất?

- a) Composite
- b) Decorator
- c) Adapter
- d) Facade

7. Một hệ thống đồ họa cần quản lý các đối tượng hình học phức tạp, bao gồm cả các hình dạng đơn giản và các nhóm hình dạng. Mẫu thiết kế nào sẽ phù hợp nhất?

- a) Bridge
- b) Composite
- c) Proxy
- d) Flyweight

8. Một hệ thống cần hỗ trợ nhiều thuật toán khác nhau cho cùng một tác vụ, và người dùng có thể lựa chọn thuật toán nào sẽ được sử dụng. Mẫu thiết kế nào sẽ phù hợp nhất?

- a) Observer
- b) Strategy
- c) Command
- d) Template Method

9. Một ứng dụng cần hỗ trợ cơ chế undo/redo cho các hành động của người dùng. Mẫu thiết kế nào sẽ phù hợp nhất?

- a) Observer
- b) State
- c) Command
- d) Iterator

10. Một ứng dụng cần thông báo cho nhiều đối tượng khi trạng thái của một đối tượng thay đổi. Mẫu thiết kế nào sẽ phù hợp nhất?

- a) Observer
- b) Mediator
- c) Visitor

d) Memento

Câu hỏi lý thuyết:

1. Mẫu tạo dựng là gì và tại sao chúng lại quan trọng trong kiến trúc phần mềm?
2. Hãy so sánh và đối chiếu các mẫu tạo dựng phổ biến như Singleton, Factory Method, Abstract Factory, Builder và Prototype. Trong những tình huống nào thì mỗi mẫu này được sử dụng phù hợp?
3. Mẫu Singleton được triển khai như thế nào và những cạm bẫy tiềm ẩn nào cần tránh khi sử dụng nó?
4. Khi nào thì việc sử dụng mẫu Builder sẽ có lợi hơn so với việc sử dụng các hàm tạo (constructor) với nhiều tham số?
5. Mẫu Abstract Factory giúp giải quyết vấn đề gì trong thiết kế phần mềm và nó khác với mẫu Factory Method như thế nào?
6. Mẫu cấu trúc là gì và chúng giải quyết những vấn đề nào trong thiết kế phần mềm?
7. So sánh và đối chiếu các mẫu cấu trúc phổ biến như Adapter, Bridge, Composite, Decorator và Facade. Trong những tình huống nào thì mỗi mẫu này được áp dụng phù hợp?
8. Mẫu Adapter giúp giải quyết vấn đề tương thích giữa các giao diện như thế nào và nó khác với mẫu Bridge ở điểm nào?
9. Khi nào thì việc sử dụng mẫu Decorator sẽ có lợi hơn so với việc sử dụng kế thừa để mở rộng chức năng của một đối tượng?
10. Mẫu Facade giúp đơn giản hóa việc sử dụng một hệ thống phức tạp như thế nào và những hạn chế nào cần lưu ý khi sử dụng nó?
11. Mẫu hành vi là gì và chúng giải quyết những vấn đề nào trong thiết kế phần mềm?
12. So sánh và đối chiếu các mẫu hành vi phổ biến như Strategy, Observer, Command, Template Method và State. Trong những tình huống nào thì mỗi mẫu này được áp dụng phù hợp?

13. Mẫu Observer giúp giải quyết vấn đề tương tác giữa các đối tượng như thế nào và nó khác với mẫu Mediator ở điểm nào?

14. Khi nào thì việc sử dụng mẫu Strategy sẽ có lợi hơn so với việc sử dụng các câu lệnh điều kiện (if-else) để lựa chọn hành vi?

15. Mẫu Command giúp đóng gói yêu cầu thành một đối tượng như thế nào và những lợi ích nào mà nó mang lại?

Bài tập vận dụng:

Bài tập 1: Hệ thống Quản lý Đơn hàng Trực tuyến (Sử dụng Mẫu Strategy)

- Yêu cầu:
 - Thiết kế một hệ thống quản lý đơn hàng trực tuyến cho phép người dùng lựa chọn các phương thức vận chuyển khác nhau (vận chuyển nhanh, vận chuyển tiêu chuẩn, giao hàng tận nơi).
 - Sử dụng mẫu Strategy để triển khai các thuật toán tính phí vận chuyển khác nhau cho mỗi phương thức vận chuyển.
 - Hệ thống cần có khả năng thêm các phương thức vận chuyển mới một cách dễ dàng.

Bài tập 2: Hệ thống Thông báo Sự kiện (Sử dụng Mẫu Observer)

- Yêu cầu:
 - Thiết kế một hệ thống thông báo sự kiện cho phép các đối tượng khác nhau đăng ký nhận thông báo khi có sự kiện xảy ra.
 - Sử dụng mẫu Observer để triển khai cơ chế thông báo sự kiện.
 - Hệ thống cần hỗ trợ nhiều loại sự kiện khác nhau và nhiều đối tượng quan sát khác nhau.

Bài tập 3: Hệ thống Tạo Báo cáo (Sử dụng Mẫu Factory Method)

- Yêu cầu:
 - Thiết kế một hệ thống tạo báo cáo cho phép tạo ra các loại báo cáo khác nhau (báo cáo PDF, báo cáo Excel, báo cáo HTML).
 - Sử dụng mẫu Factory Method để tạo ra các đối tượng báo cáo dựa trên loại báo cáo được yêu cầu.

- Hệ thống cần có khả năng thêm các loại báo cáo mới một cách dễ dàng.

Bài tập 4: Hệ thống Giao diện Người dùng Đa nền tảng (Sử dụng Mẫu Adapter)

- Yêu cầu:
 - Thiết kế một hệ thống giao diện người dùng cho phép ứng dụng chạy trên nhiều nền tảng khác nhau (Windows, macOS, Linux).
 - Sử dụng mẫu Adapter để chuyển đổi giao diện người dùng của từng nền tảng thành một giao diện chung.
 - Hệ thống cần đảm bảo tính nhất quán về giao diện người dùng trên tất cả các nền tảng.

Bài tập 5: Hệ thống Quản lý Cấu hình Ứng dụng (Sử dụng Mẫu Singleton)

- Yêu cầu:
 - Thiết kế một hệ thống quản lý cấu hình ứng dụng cho phép truy cập và quản lý các thông số cấu hình của ứng dụng.
 - Sử dụng mẫu Singleton để đảm bảo rằng chỉ có một thể hiện duy nhất của lớp quản lý cấu hình được tạo ra.
 - Hệ thống cần đảm bảo tính nhất quán và an toàn khi truy cập các thông số cấu hình.